

What is Natural Language ?

- Natural language is a language that has developed naturally over the years.
- Also, it will be having its own history.
- Every language has its own grammars and symbols.
- As you know, more than 7000 languages are used by the people around the world.
- In fact, Most people are using English as their language globally.

Why NLP is important ?

- Need automation. Let me give you two simple reasons.
- First, We want to connect with more people globally. But, it is very difficult to learn all the languages immediately. So, we need automation for translation.
- Second, In order to help the society, we want to convert some important books written in other languages into our own language. It will help millions of people to improve their knowledge. So, we need automation for translation.

Definition of NLP

- By definition, Natural language processing (NLP) is a branch of artificial intelligence that helps computers to understand, interpret and manipulate human language.
- Here, NLP is a skill. Humans have so many such skills like driving, swimming, dressing sense, and etc.
- Intelligence is the collection of all such skills, talents, knowledge, futuristic ideas, plans, executing styles and etc.,
- Hence, NLP is a branch of Artificial Intelligence.

Recall the terminologies

- **Natural Intelligence** : As you know, all the living beings such as human, animal, birds and even insects having this Natural Intelligence.
- **Artificial Intelligence** : Yes, we are trying to give this intelligence into a machine, is known as Artificial Intelligence.
- In this respect, It is said that NLP is the subset of AI.

Categories of NLP

- First category NLP is which is working with the Written Form such as books, scanned documents and hand written materials.
- Second category NLP that is working with Audio Form such as voice inputs and translating their language into an another language.

Components of NLP

- There are two components in NLP. First is called **Natural Language Understanding** and the second one is called **Natural Language Generation**.
- First component is responsible for understanding the original content, it may be written or voice form. The process is same.
- Second component is responsible for generating the original content into some other language as per our interest. Together jointly become a NLP.

Challenges in Working with NLP

- The first challenge in NLP is incompleteness. Because, some times, statement may not be having the complete information.
- Look at the first word and the last word in the given example .
- “Few Horses are standing outside”.
- There is no count and There is no exact location.

Challenges in Working with NLP

- Next, some statement may be giving different meaning. It depends on reader's perceptive. This is the second challenge in NLP.
- Please have a look at the given example.
- “Alan is poor”.
- How much poor he is ?
- Here, every reader will understand differently about Alan.

Ambiguity and Uncertainty

- Ambiguity means confusion. Consequently, confusion creates uncertainty. There are three ways of ambiguity. Let me give you one by one with examples.
- First, Lexical Ambiguity
- Second, Syntactic Ambiguity
- Third, Referential Ambiguity

Lexical Ambiguity

- Let us discuss the first one. Lexical Ambiguity.
- Here one word is going to be the reason for confusion.
- Have a look at the example :

“Manya is looking for a match.”
- Try to answer my question. What is referred by the word match ?

Syntactic Ambiguity

- Let us discuss on the second one. Syntactic ambiguity,
- Have a look, at this example.

“I saw the girl with the binocular”.

- Now, try to answer my question. Who is having that binocular ?
- Here, the confusion is coming because of the writing style – the arrangements of the words. That’s called syntactic ambiguity.

Referential Ambiguity

- Here is the third one. Referential ambiguity. Pro-noun is making the confusion.
- Please read the example.

“Kiran went to Sunita. She said, “I am hungry.”
- Now the question is, Who is hungry ?
- Kiran or Sunita. Ambiguity.... Yes.

Real life examples with NLP

- The first one is, **Email Spam filters**
- Let us take an example of Gmail, an email service provider.
- It is categorizing all the mails that comes to us, into different sections, such as primary, social, promotions and spam.
- So, this process make our life comfortable in checking the mails quickly.

Real life examples

- The second one is **Google Translators**
- We can search for the term “Google Translate”. This website facilitates the language translation as per our requirement.
- Also, Amazon also came with their product called “Amazon Translate”. Frankly speaking, this list is unlimited.

Real life examples

- Chatbot Applications : Chatbot is the software which will do the conversation through chat messages just look-like a human being. It will reply with ready-made messages.
- Companies are giving 24x7 customer support. But, most of the customers raises only the fundamental questions. For this, Chatbot are installed to understand the question and reply them immediately with ready-made answers. Across the domain, this application is used.

Real life examples

- Personal Assistant Applications in Mobiles.
- Let us take the Smart assistants for example such as Siri and Alexa.
- Siri is the product of Apple Company and Alexa is the product of Amazon.
- This will understand the human speech and reply them instantly.

Real life examples

- Sentiment Analysis / Opinion mining.
- Usually, this analysis is performed on the textual data.
- Companies take their required data from many social medias and feedbacks that are posted in the e-commerce websites.
- This will help them to understand the people's overall opinion on something of their interest.

What is Phases of NLP ?

- Phases may be called as Steps.
- There are five steps to solve any NLP problem.
- Hence, it is known as Phases of NLP.

Example : document translation

- Before begin this discussion, let us have a small example. – Document Translation.
- Consider you have just one page written material.
- Let us assume, this document is in English.
- Now, we try to translate into another language, anything but make it different language for time being.
- Yes, here you go, what to do first, how to start... what next... here comes the phases of Natural Language Processing.

Challenges in Translation

Mixed Languages

- The documents may have few words from other languages. If you see the social media – you know this.

Research Articles

- Graphs, Mathematical Expressions and Formula exist in between the documents. Scientific / Biological terms too will come.

Social Media

- People have started to use their own languages on social media. This will increase the language count.

Sentiments

- The emotions/feelings – what the writer wants to deliver. Need to be understood.

Phases of NLP

- Lexical Analysis
- Syntactic Analysis
- Semantic Analysis
- Discourse Integration
- Pragmatic Analysis

Phases of NLP

While we are solving any NLP problem, these phases should be followed in the sequence, the same order as I have told.

Lexical, Syntactic, Semantic, Discourse Integration, and lastly the Pragmatic Analysis.

Lexical Analysis

- This is the first step. Take the doc, and have a look at it.
- Paragraphs, Sentences, Words and Symbols are completely analyzed in this phase. [HOW MANY...]
- This phase stores all such information separately.
- Each division of such information is called Tokens.
- This process is called Tokenization.
- These tokens are sent to the next phase for further processing.

Example – Lexical Analysis

- Consider this statement :
- “I am giving a lecture now on Phases of Natural Language Processing”.
- Tokenization process will give 12 tokens :
- They are :
 - { I, am, giving, a, lecture, now, on, Phases, of, Natural, Language, Processing }
- Each word becomes one token. Hence it comes 12.

Syntactic Analysis

- The Output of the Lexical Analysis is used as input.
- Now, we are checking for mistakes in the words and mistakes in the writing style or grammar.
- Every statement in English has some format. We know there are some terms called, noun, verb, adjective, pronoun, subject, etc.
- This phase is primarily checking the grammar, word arrangements, and the relationship among the words.
- This process is called Parsing.
- All the mistakes are corrected here. After correction, goes to the next phase.

Example - Syntactic Analysis

**“I ate an
apple”.**

Correct as per the language rules.

**“I an apple
ate”.**

Not correct as per the rule.

Semantic Analysis

- The output of the Syntactic Phase is taken as input.
- This phase concerns the meaning of the statements.
- Statements may be correct grammatically. But, it may **not** be always correct as per the real meaning.
- Such statements are called unrealistic statements. They are completely ignored from further processing. Because, it is not going to help us anyway.
- **Rest of the statements will be sent to the next phase.**

Example - Semantic Analysis

- Example 1: “Horse is going to Factory”.
- Example 2 : “Factory is going to Horse”.
- Which is correct ? Both are correct as per grammar.
- But, meaning ?
- Example 3 : Birds sit on the Trees.
- Example 4. Trees sit on the Birds.

Discourse Integration

- This is the fourth phase.
- Usually, all the statements may not have complete information in it. We can recall the term incompleteness, one of the NLP challenges.
- Some details may be missing. You can't understand by analyzing an individual sentences.
- That is why, this phase is integrating / joining the related statements together for effective understanding.

Example – Discourse Integration

Three statements are given here. Kindly go through this.

- Horse ran up the hill.
- It was very steep.
- It soon got tired.
- What does “It” refers to in 2nd and 3rd sentences?

Pragmatic Analysis

- This is the last phase.
- This phase tries to characterize the statements.
- Categorization... Yes.. Let me define like this,
- Statements can be a **request / an order / pleasing / with love / angry / suspicious / with interest / hating and etc.**
- If you understand it better, the emotions / mood / the way the statement is written, the translation will be perfect.

Example – Pragmatic Analysis

- Example 1: "Open the door".
- Can you understand the emotional thing in this statement ? No... So You need related statements again. We have to extract this information.
- Example 2: "My child is taking all my time".
- What kind of emotion, it is coming from this statement ? It is difficult, but anyway, we need it.

What is Parsing ?

- It is a process.
- It is used to check the statement whether it is written correctly as per the syntax / grammar rules or not.
- This process will help syntactic analysis, the 2nd phase of NLP, checks all the statements.

Example – Let us parse these statements...

- I am riding a motor bike. --- Correct Statement.
- Motor bike is riding me --- Correct as per the syntax.

NOTE :

Here, please understand that the process of parsing is checking only the syntax not the meaning.

What is a Parse Tree?

- This parse tree is the tree structure which is used for parsing – means we are going to check the given statement whether it is correct or not using this tree structure.
- Parsing will help in syntactic analysis phase, as I have already told you.
- Initially, we are extracting all the tokens from the given statement, during the Lexical Analysis Phase. Then those tokens are used here in the parsing.

Parse Tree – An Example

Let us take the statement for example.

“He studied”.

Let us give symbol S. Now we try to parse. For parsing, you need the grammar rules.

Yes, here we need to create our own grammar rules depending upon which language we are using for translation.

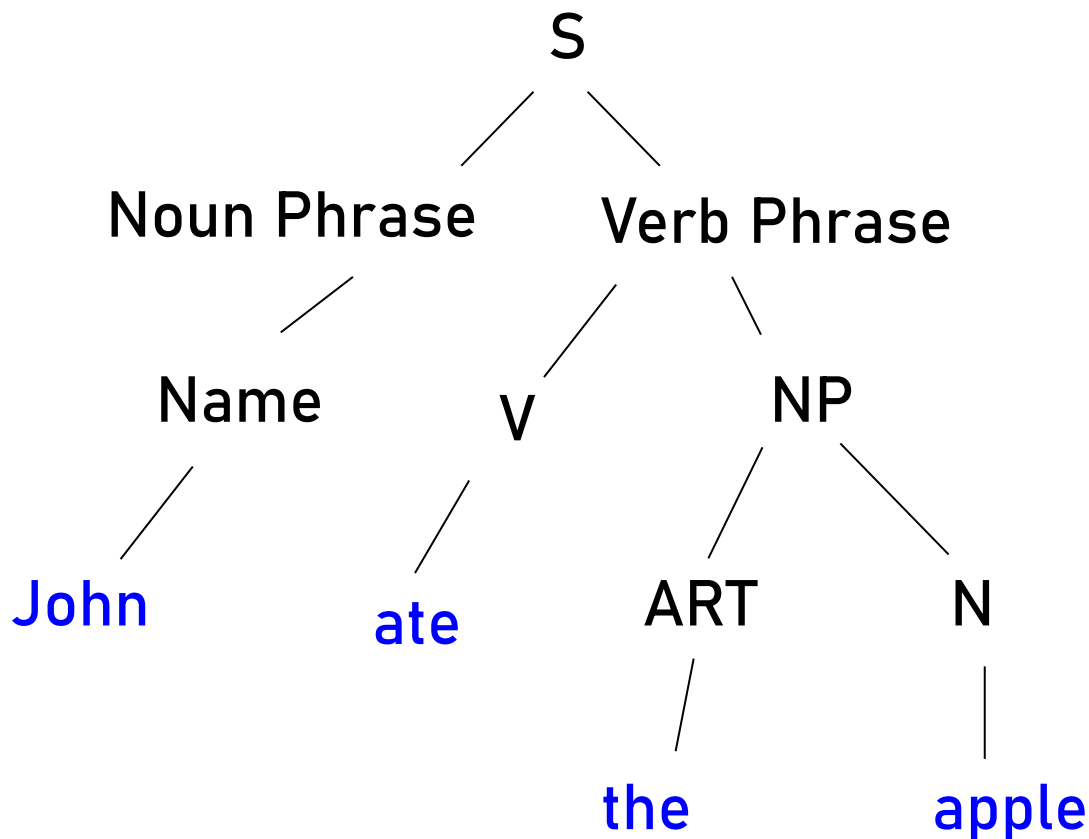
Hence, we can write our own Grammar Rules. Like this,
$$S \rightarrow NP \quad VP$$

Parse Tree – An Example

S = John ate the apple.

Grammar Rules

1. $S \rightarrow NP \ VP$
2. $VP \rightarrow V \ NP$
3. $NP \rightarrow NAME$
4. $NP \rightarrow ART \ N$

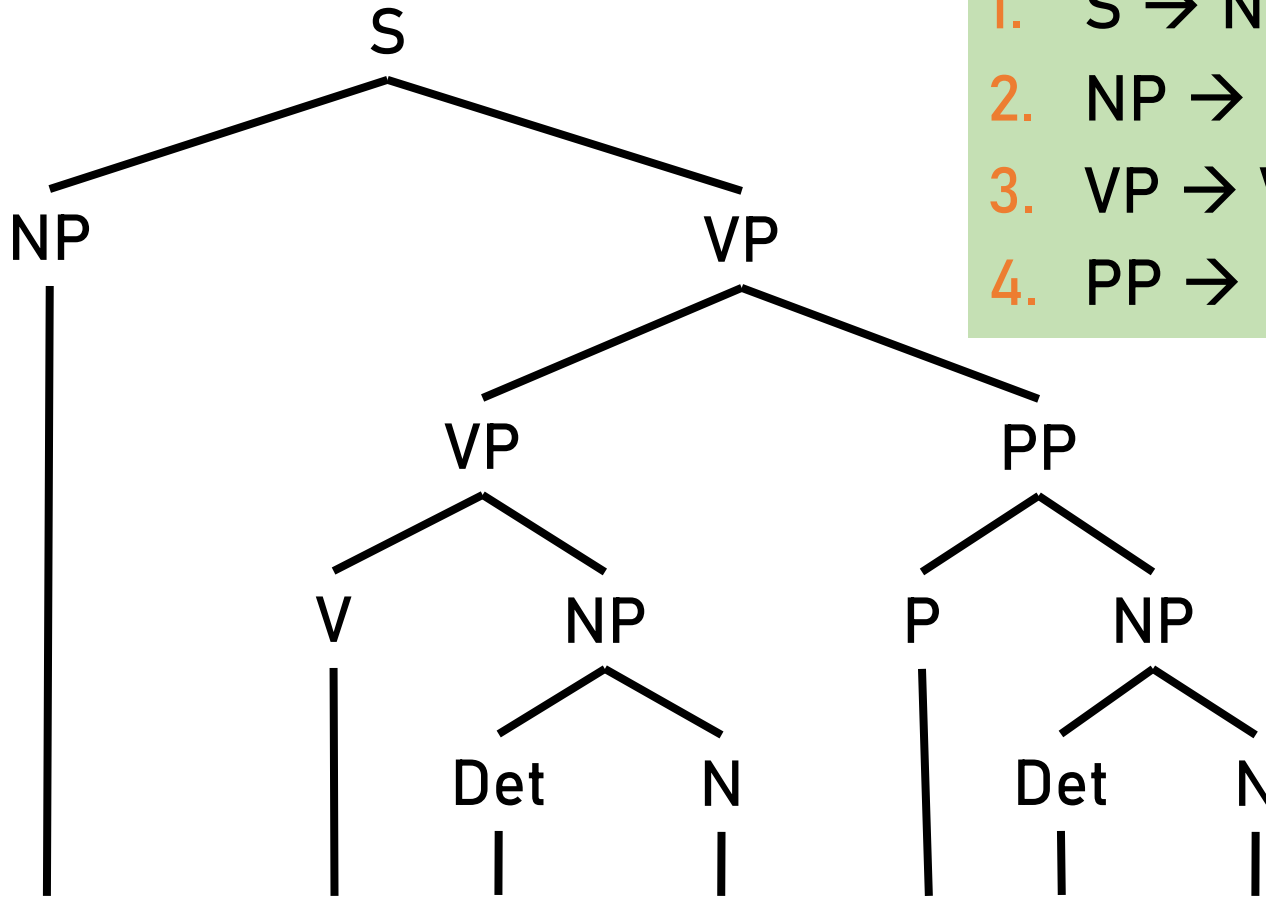


Examples

- S = “Brother gave a beautiful gift ”.
 1. $S \rightarrow NP \ VP$
 2. $VP \rightarrow V \ NP$
 3. $NP \rightarrow N \mid ART \ NP \mid Det \ NP \mid Art \ N$
- S = “Father ate an apple with a knife”.

Grammar Rules

1. $S \rightarrow NP VP$
2. $NP \rightarrow Det N \mid NP PP$
3. $VP \rightarrow V NP \mid VP PP$
4. $PP \rightarrow P NP$



Father ate an apple with a knife

Contd.

Now, you want to know, how to write our own grammar rules for a particular NLP problem.

This is not achievable in one or two statements. Also, we can not make perfect grammar rule at this time which will work for any statements, like a generalized one. It is not possible for now. TO DO THIS,

Collect 50 – 100 domain specific statements. Start making grammar rules for the first statement. Then do for the 2nd one... keep on updating the rules slowly with any rules you find new in that statement. Complete for all.

This is the only way – for building up our own grammar rules.

At this time, we assume that we already have grammar rules. Now, we are only generating the parse trees.

Parsing Components

Now, we should know, what are all the things we should get ready with and keep before doing this parsing.

- **Lexicon** – Yes, it has come from tokenization process.
- **Categorization** – You should know how to label each word.
- **Grammar Rules** – Of course., very important.

What is Lexicon ?

- Lexicon is the vocabulary of a language.
- Suppose, we want to process all the statements used in the educational institution.
- Means, we should have all the words (vocabulary) used in the educational institution.
- That is called Lexicon.

Categorization – How to label the words?

Here, you should know... what are all the words come in noun category. What are all th words come under Pro-noun category and etc.

For example, I hope... you know all these categories.

- Noun ; Pronoun ; Verb ; Preposition
- Adverb ; Adjectives ; Determinants ; Conjunctions and etc.

Yes, we should be ready with this categorization, with respect to your specific domain.

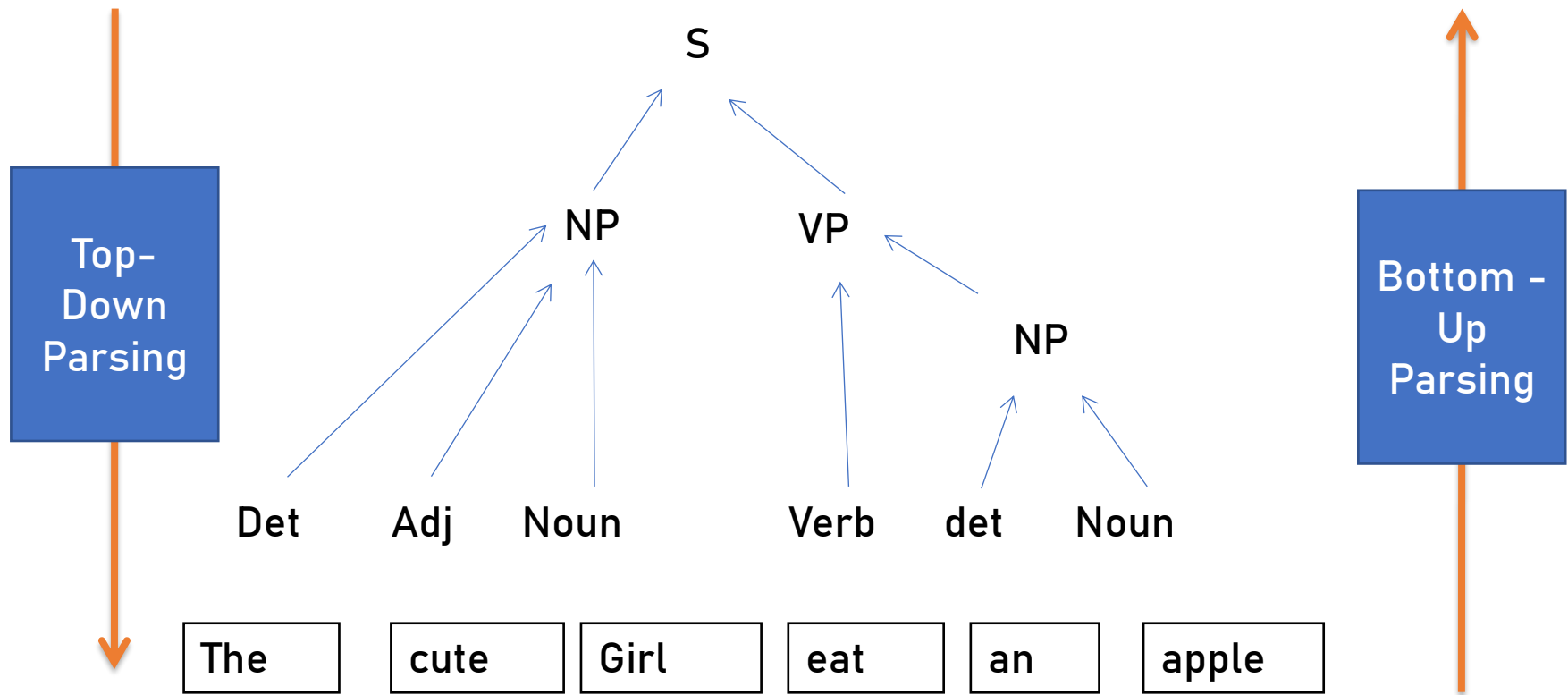
Types of Parsing

We came to the last part of parsing.

- Top Down Parsing
- Bottom Up Parsing

Types of Parsing

S = The cute girl ate an apple



Top down and bottom up parsing

We can understand this better with respect to grammar rules.
It will be easy also.

If you choose the grammar rule from LEFT to RIGHT – it is
Top Down Parsing.

$$S \rightarrow VP NP$$

If you choose the grammar rule from RIGHT to LEFT – it is
Bottom Up Parsing.

$$S \leftarrow VP NP$$

What is Tokenization ?

The Tokenization is the process – Used in Lexical Analysis Phase. Actually, it is dividing the sentence into pieces, such as extracting individual words or terms.

Hence, Tokenization is essentially splitting a phrase, sentence, paragraph, or an entire text document into smaller units. Each of these smaller units are called tokens.

What can be in the Tokens ?

We can't predict the things exactly. Because, document can be any category. Sometimes, it can be full of characters, means, only the words. Sometimes, it will be having some numbers like date of birth, year, some other quantitative measures used to present in the document. Also, it may be having some formula. So, symbols and operators also will come into the tokenization. And lastly, question marks, exclamatory symbol, single / double quotes, apostrophe, full-stop also may come in the document.

Here I am giving you few standard category of tokens, for example.

- Words ; Numbers ; Punctuation Marks ; Symbols

Example on How Tokenization works ?

- Let us consider the statement. "Cat eats two rats"
- We do the tokenization process in this statement. Please look at the result now. It is given as, four words.
- Tokens = { "Cat", "eats", "two", "rats" }
- Extraction of individual units is done and hence, we are getting this output. Here we have four tokens with us. And, all are belongs to word category. Means all are having only characters.
- We need to find some way how to get this words after we receive any statement. What is the logic or is there any library to help in tokenization ? Let us find out.

Example - Tokenization

Let us consider an another example. The statement is here. Please read the statement once for better understanding.

The statement says :

“My daughter got 80 percent in mathematics. She was born in the year 2015.”

- Now, try to complete tokenization and check out our output with this. The result is here..

Tokens =

{ “My”, “daughter”, “got”, “80”, “percent”, “in”,
“mathematics”, “.”, “She”, “was”, “born”, “the”, “year”, “2015” }

Total number of tokens is 14.

This is the logic for Sentence tokenization

- Means, this logic is going to help in extracting only the sentences one by one from the document... , like sentence by sentence, not like word by word. Here, we are going to use the python function known as Split function.
- Let us have an example. Store one paragraph in a variable S.
- S = "Lion is the king of jungle. Monkeys are having long tail. All are enjoying the rain".
- Please remember that I want to divide this into pieces... the piece should be a sentence.
- SentToken = S.split (".") This will give our required output.
- SentToken = { "Lion is the king of jungle", "Monkeys are having long tail", "All are enjoying the rain" }

Methods for Word Tokenization

- Yes, here, we are going to see the logic to do tokenization for word by word process.
- Like the same way we did on sentence, we can use the same function also. It will work. But, there is a parameter in function how the splitting should be performed.
- During sentence tokenization, we used full-stop as parameter.
- Because we know, full-stop is the key to identify a sentence.
- But, for the word, yes..... SPACE is the key. There are spaces in between every word. Here we go. Go through the example. And see how we are calling the split function.
$$S = \text{"cat eats many rats ; } T = S.\text{split ()}; T = \{\text{cat,eats,many,rats }\}$$
- **By default.... No parameter means spaces.**

Tokenization using NLTK (Natural Language Tool Kit)

- NLTK is a open source library for Python. This library is used mainly for Natural Language Processing.
- We can install this and import into our python program. We can use it very easily.
- Here, I am sharing with you the two functions from this NLTK.
- First one is `word_tokenize ( )` function
- The second one is `sent_tokenize ( )` function
- How to use them. Let us see this one by one,

word_tokenize () method

We have a sample document and store it in a variable.

Here, my variable name is TEXT. Go through the content.

```
text = "We are friends since 2020. This will go for a long."
```

```
from nltk.tokenize import word_tokenize  
Print ( word_tokenize(text) )
```

- We get our output and it will be displayed in the screen.
- OR if we want.... we can store the result in a separate variable... as per our interest.

sent_tokenize () method

Now, we are discussing on sentence tokenize function.

As usual, here is the example... My variable name is TEXT.

```
text = "We are friends since 2020. This will go for a long."
```

Importing of the required functionalities are given here....

```
from nltk.tokenize import sent_tokenize
```

```
Print ( sent_tokenize (text) )
```

Get the output... as printed... or use the variable to store it.

Why Stemming and lemmatization ?

- A word can be used in many forms because of the grammatical reasons.
- This example will help you to understand.
- Example 1 : come, came, coming, comes
- When you do the tokenization, you will get multiple tokens for the same base word, i.e., come
- Example 2 : organize, organizes, organizing,
organizable, organized
- Here, in second example. There are 5 different words for one base word i.e., organize.

Why Stemming and lemmatization ?

- Here we go. Why stemming and lemmatization ?
- Stemming and lemmatization is helping to reduce all the different forms of the words into one single common base form.

Difference between Stemming and lemmatization

- Stemming usually refers to a crude heuristic process that chops off... means it cuts the ending-words.
- This root word is called as the Stem.
- Lemmatization usually use the proper vocabulary and do the things properly and return the base form which is known as the lemma.
- Lemma usually will be from dictionary form of a word,
- But in stemming, you will not get dictionary form of the word always.... That's the main difference.

Difference between Stemming and lemmatization

- Lemmatization is taking more time. Because it finds dictionary related word.
- Lemmatization will be useful for document understanding.
- Stemming is taking less time. Because, It removes few letters at the end of the word and making a root / base word.
- Stemming will be useful where the dataset is large. Hence, it works comparatively faster than lemmatization.

Algorithms in Python NLTK

- We have two algorithms for Stemming. The names are ;;;
 - Porter Stemmer
 - Lancaster Stammer

For Lemmatization, we have WordNetLemmatizer

This is using Wordnet database, which is a publicly available lexical database of over 200 languages. It is one of the earliest and most commonly used lemmatize technique.

How to use Porter Stemmer ?

We assume that tokenization is completed. And now we are doing only Stemming. Import the required functionalities like this.

```
from nltk.stem import PorterStemmer
```

Create an object for PORTER STEMMER.

```
ps = PorterStemmer ( )
```

```
words = [ "program", "programs", "programmer",  
          "programming", "programmers"]
```

```
for w in words :
```

```
    print(w, " : ", ps.stem(w))
```

How to use Lancaster Stemmer ?

Similarly, here is the discussion on how Lancaster is used.

As usual, how to import the functionalities...

```
from nltk.stem import LancasterStemmer
from nltk.tokenize import word_tokenize
ps = LancasterStemmer ()
words = ["program", "programs", "programmer",
"programming", "programmers"]
for w in words:
    print(w, " : ", ps.stem(w))
```

How to use WordNet Lemmatizer ?

Lastly, we have come to lemmatization using WordNet.

```
from nltk.stem import WordNetLemmatizer
ps = WordNetLemmatizer()

words = ["program", "programs", "programmer",
"programming", "programmers"]
for w in words:
    print(w, " : ", ps.lemmatize(w))
```

With this, I am sure that you can understand the importance of stemming and lemmatization. You can also understand... where to use... what to use ?

What is Bag of Words ?

- A bag-of-words is a representation of text that describes the occurrence of words within a document.
- The collection of words is called Bag of Words.

Detecting category of Document

Sports Category :

Ashwin's day end figures read 28-5-70-1 as skipper Rory Burns, who is also an England opener, used him from both ends of the Oval ground. Somerset finished the opening day at 280 for 6 in 98 overs with skipper James Hildreth scoring 107 off 212 balls.

Banking Category :

The restructured book of non-bank financial companies (NBFCs) and housing finance companies (HFCs) is expected to double to 3.1-3.3% of their assets in March 2022 from 1.6% in March 2021, rating agency Icra said.

Classification Sport and Banking Documents

Assume you have 100 documents. 50 documents represents sports category. 50 documents represents banking category

DOC								TARGET
D1								1
D2								1
..								..
D50								1
D51								0
D52								0
D53								0
...								..
D100								0

Text classification – An Example

- Document-level
- Paragraph-level
- Sentence level
- Sub-sentence level

Managing Vocabulary

- As the vocabulary size increases, it is reflected in the vector representation of documents also.
- The length of the document vector is equal to the number of known words.

Text cleaning

- Ignoring case
- Ignoring punctuation
- Ignoring stop words i.e., frequent words that don't contain much information, like "a," "of," "in", etc.

Step 1: Collect the Data

- Consider this example having four lines as four documents.
- It was the best of times,
- it was the worst of times,
- it was the age of wisdom,
- it was the age of foolishness,

Step 2: Design the Vocabulary

- Now we can make a list of all of the words in our model vocabulary.
- The unique words here (ignoring case and punctuation) are:
- “it”, “was”, “the”, “best” , “of” , “times” , “worst”, “age”, “wisdom”, “foolishness”
- That is a vocabulary of 10 words from a corpus containing 24 words.

Step 3: Create Document Vectors

- We can use a fixed-length document representation of 10, with one position in the vector to score each word.
- In this step, we need to score the words in each document.
- The simplest scoring method is to mark the presence of words as a boolean value, 0 for absent, 1 for present.

Line 1 – Result of scoring method

- The scoring of the document would look as follows for the first line:
- It was the best of times,
- “it” = 1 ; “was” = 1 ; “the” = 1 ; “best” = 1, “of” = 1 ; “times” = 1 ; “worst” = 0 ; “age” = 0 ; “wisdom” = 0 ; “foolishness” = 0

Line 2 – Result of scoring method

- The scoring of the document would look as follows for the second line:
- it was the worst of times,
- “it” = 1 ; “was” = 1 ; “the” = 1 ; “best” = 0, “of” = 1 ; “times” = 1 ; “worst” = 1 ; “age” = 0 ; “wisdom” = 0 ; “foolishness” = 0

Line 3 – Result of scoring method

- The scoring of the document would look as follows for the third line:
- it was the age of wisdom,
- “it” = 1 ; “was” = 1 ; “the” = 1 ; “best” = 0, “of” = 1 ; “times” = 0 ; “worst” = 0 ; “age” = 1 ; “wisdom” = 1 ; “foolishness” = 0

Line 4 – Result of scoring method

- The scoring of the document would look as follows for the fourth line:
- it was the age of foolishness,
- “it” = 1 ; “was” = 1 ; “the” = 1 ; “best” = 0, “of” = 1 ; “times” = 0; “worst” = 0 ; “age” = 1; “wisdom” = 0 ; “foolishness” = 1

Binary Vector Representation

- Vocabulary
- “it”, “was”, “the”, “best”, “of”, “times”, “worst”, “age”, “wisdom”, “foolishness”
- Sentence : It was the best of times,
- Binary vector is = [1, 1, 1, 1, 1, 1, 0, 0, 0, 0]

Binary Vector Representation

- Vocabulary
- “it”, “was”, “the”, “best”, “of”, “times”, “worst”, “age”, “wisdom”, “foolishness”
- Sentence : it was the worst of times,
- Binary Vector = [1, 1, 1, 0, 1, 1, 1, 0, 0, 0]

Binary Vector Representation

- Vocabulary
- “it”, “was”, “the”, “best”, “of”, “times”, “worst”, “age”, “wisdom”, “foolishness”
- Sentence : it was the age of wisdom,
- Binary Vector is = [1, 1, 1, 0, 1, 0, 0, 1, 1, 0]

Binary Vector Representation

- Vocabulary
- “it”, “was”, “the”, “best”, “of”, “times”, “worst”, “age”, “wisdom”, “foolishness”
- Sentence : it was the age of foolishness,
- Binary Vector is = [1, 1, 1, 0, 1, 0, 0, 1, 0, 1]

Fixing misspelled words

- We need to correct the spelling before tokenization.

Decreasing Size of Vocabulary

- Reducing words to their stem (e.g. “play” from “playing”) using stemming algorithms.

Group of Words as Vocabulary

- A more sophisticated approach is to create a vocabulary of grouped words.
- In this approach, each word or token is called a “gram”.
- Creating a vocabulary of two-word pairs is, in turn, called a bigram model.
- Creating a vocabulary of n-word pairs is, in turn, called an n-gram model.

Additional simple scoring methods

- Counts : Count the number of times each word appears in a document.
- Frequencies : Calculate the frequency that each word appears in a document out of all the words in the document.
 - TERM FREQUENCY
 - DOCUMENT FREQUENCY

Term Frequency–Inverse Document Frequency

- Term Frequency (TF): Measures how often a word appears in a document. A higher frequency suggests greater importance. If a term appears frequently in a document, it is likely relevant to the document's content.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

Term Frequency–Inverse Document Frequency

- Inverse Document Frequency (IDF): Reduces the weight of common words across multiple documents while increasing the weight of rare words. If a term appears in fewer documents, it is more likely to be meaningful and specific.

$$\text{IDF}(t, D) = \log \frac{\text{Total number of documents in corpus } D}{\text{Number of documents containing term } t}$$

Example

- Document 1: "The cat sat on the mat."
- Document 2: "The dog played in the park."
- Document 3: "Cat and dogs are great pets."

Calculate Term Frequency (TF)

- Document 1: "The cat sat on the mat."
- The word "cat" appears 1 time.
- The total number of terms in Document 1 is 6 ("the", "cat", "sat", "on", "the", "mat").
- So, $TF(\text{cat}, \text{Document 1}) = 1/6$

Calculate Term Frequency (TF)

- Document 2: "The dog played in the park."
- The word "cat" does not appear.
- So, $TF(\text{cat}, \text{Document 2}) = 0$.

Calculate Term Frequency (TF)

- Document 3:
- The word "cat" appears 1 time.
- The total number of terms in Document 3 is 6 ("cats", "and", "dogs", "are", "great", "pets").
- So $TF(\text{cat}, \text{Document 3}) = 1/6$

Calculate Inverse Document Frequency (IDF)

- Total number of documents in the corpus (D): 3
- Number of documents containing the term "cat": 2 (Document 1 and Document 3).

$$IDF(cat, D) = \log \frac{3}{2} \approx 0.176$$

Calculate TF-IDF

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

- $\text{TF-IDF}(\text{cat}, \text{Document 1}, D) \rightarrow 0.167 * 0.176 = 0.029$
- $\text{TF-IDF}(\text{cat}, \text{Document 2}, D) \rightarrow 0 \times 0.176 = 0$
- $\text{TF-IDF}(\text{cat}, \text{Document 3}, D) \rightarrow 0.167 \times 0.176 = 0.029$

BoW Model - Recall

DOC								TARGET
D1								1
D2								1
..								..
D50								1
D51								0
D52								0
D53								0
...								..
D100								0

SEE YOU IN NEXT CLASS !!!

INT344

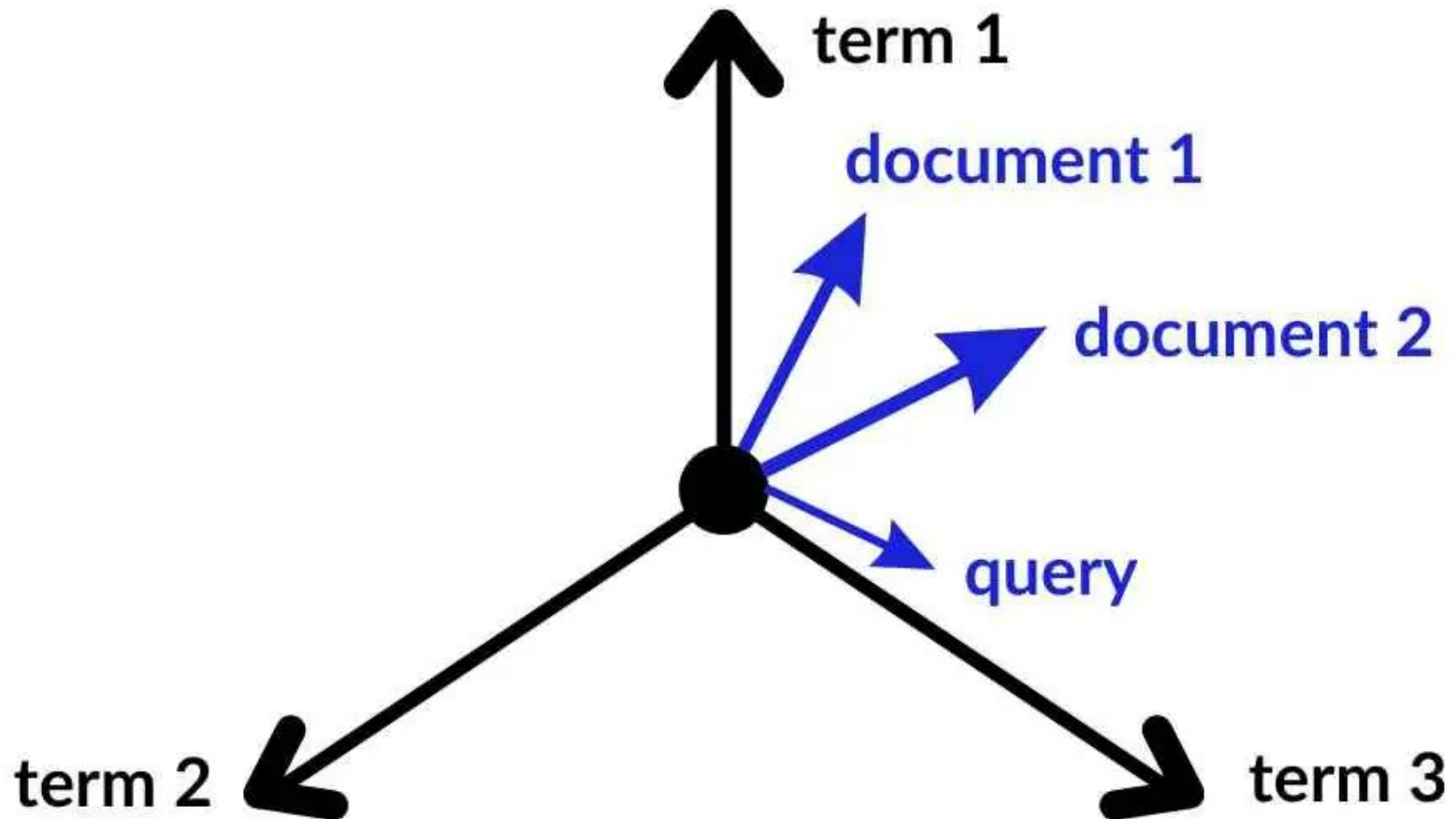
Natural Language Processing

UNIT- 2

- **Vector Space Models :**

- Vector Space Models, Capture Semantic Meaning, Continuous Bag-of-Words (CBOW), Relationships between Words, Capture Dependencies between Words, Visualize the Relationships in Two Dimensions Using PCA, Machine Translation and Document Search, Transform Word Vectors.

Vector Space Model



What is Vector Space Model ?

- Vector space model or term vector model is an algebraic model for representing text documents (or more generally, items) as vectors such that the distance between vectors represents the relevance between the documents.

Vector Space Model

- The **Vector Space Model (VSM)** is used primarily in **information retrieval** and **text mining** because it provides a way to represent text documents and queries as vectors in a multi-dimensional space.

Vector Space Model

- **Vector Space Model (VSM)** represents documents and queries as vectors in a multi-dimensional term space.
- **Each dimension** corresponds to a unique term in the corpus, forming the basis for vector representation.
- **Document-Term Matrix (DTM)** stores term frequencies for each document, forming the raw input for VSM.
- **TF-IDF weighting** highlights important terms in documents and reduces the impact of common words.
- **TF (Term Frequency)** counts how often a term appears in a document.
- **IDF (Inverse Document Frequency)** reduces weight for terms appearing in many documents.
- **Vectorization** transforms documents and queries into fixed-length numeric vectors.
- **Document vectors** have dimensions equal to the number of unique terms in the corpus.

Vector Space Model

- **Cosine similarity** measures the angle between vectors to assess similarity between documents or queries.
- **Similarity score** ranges from -1 (opposite) to 1 (identical); higher scores indicate greater similarity.
- **Efficient retrieval** is possible by ranking documents based on similarity to a query vector.
- **Noise reduction** is achieved by down-weighting frequent, non-informative terms.
- **Sparse representation** results due to most documents using only a subset of total terms.
- **Used in IR systems**, search engines, and recommender systems for text matching.
- **Scalable approach** suitable for large corpora using efficient matrix and vector operations.

How is Cosine Similarity Calculated?

The formula for calculating cosine similarity between two vectors A and B is as follows:

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

$A \cdot B$ represents the dot product of vectors A and B.

$\|A\|$ and $\|B\|$ represent the Euclidean norms (magnitudes) of vectors A and B, respectively.

The cosine similarity value ranges from -1 (completely dissimilar) to 1 (completely similar). A higher cosine similarity score indicates greater similarity between the two vectors.

Vector Space Model example

Let's walk through a simple example of the Vector Space Model (VSM) using a small corpus of documents and a query. In this example, we'll represent documents and a query as vectors and calculate cosine similarity to retrieve relevant documents based on the query.

•Step 1: Corpus and Query

Let's start with a small corpus of three documents and a query:

Document 1: "The quick brown fox jumps over the lazy dog."

Document 2: "A brown dog chased the fox."

Document 3: "The dog is lazy."

Query: "brown dog"

Step 2: Create the Document-Term Matrix (DTM)

We create a DTM where rows represent documents and columns represent terms. We'll use TF-IDF values for each term in the matrix:

	a	brown	chased	dog	fox	is	jumps	lazy	over	quick	the
Doc 1	0	0.29	0	0.29	0.29	0	0.29	0.29	0.29	0.29	0.58
Doc 2	0.41	0.29	0.41	0.29	0.29	0	0	0	0	0	0.41
Doc 3	0	0	0	0.41	0	0.41	0	0.41	0	0	0.41
Query	0	0.71	0	0.71	0	0	0	0	0	0	0

Here, we've calculated TF-IDF values for each term in the documents and the query. You can use different formulas for TF-IDF, but this is common.

Step 3: Vectorize the Query

The query is also represented as a vector. In this case, it's a simple binary vector where 1 represents the presence of a term and 0 represents the absence:

	a	brown	chased	dog	fox	is	jumps	lazy	over	quick	the
Query	0	1	0	1	0	0	0	0	0	0	0

Step 4: Calculate Cosine Similarity

Now, we calculate the cosine similarity between the query vector and each document vector. The formula for cosine similarity is: cosine similarity formula applied to documents
Using this formula, we calculate the cosine similarity between the query and each document:

$$\text{Cosine Similarity} = \frac{\text{Query Vector} \cdot \text{Document Vector}}{\|\text{Query Vector}\| \cdot \|\text{Document Vector}\|}$$

Cosine Similarity(Query, Doc 1) $\approx$ 0.58

Cosine Similarity(Query, Doc 2) $\approx$ 0.29

Cosine Similarity(Query, Doc 3) $\approx$ 0.41

Step 5: Rank Documents by Similarity

The documents are ranked by their cosine similarity values in descending order:

Document 1: Cosine Similarity $\approx$ 0.58

Document 3: Cosine Similarity $\approx$ 0.41

Document 2: Cosine Similarity $\approx$ 0.29

Vector Model Solution using Documents

The method of performing the operations and the formulas required for the computation is present in the previous document that is part 1. Consider the following collection of documents.

document1 = 'one two '

document2 = 'three two four '

document3 = 'one two three '

document4 = 'one two '

The formulas used

Some terms appear thrice, twice and sometimes only once in the document. The total number of documents $N=4$. Therefore, the IDF values of the terms are:

one --> $\log_2(4/3) = 0.4147$

two --> $\log_2(4/4) = 0$

three --> $\log_2(4/2) = 1$

four --> $\log_2(4/1) = 2$

Representation in boolean model

	one	two	three	four
document1	1	1	0	0
document2	0	1	1	1
document3	1	1	1	0
document4	1	1	0	0

Calculation of term frequency

one --> $3/4 = 0.75$

two --> $4/4 = 1$

three --> $2/4 = 0.5$

four --> $1/4 = 0.25$

Calculation of weights ($tf * idf$)

weight(one) --> $0.75 * 0.4147 = 0.3110$

weight(two) --> $1 * 0 = 0$

weight(three) --> $0.5 * 1 = 0.5$

weight(four) --> $0.25 * 2 = 0.5$

Representation of vector model in terms of weights

	one	two	three	four
document1	0.3110	0	0	0
document2	0	0	0.5	0.5
document3	0.3110	0	0.5	0
document4	0.3110	0	0	0

QUERY: Document containing ‘ one three three ‘

Calculation of weights for query terms(term frequency)

weight(one) $\rightarrow 1/3 = 0.333$

weight(three) $\rightarrow 2/3 = 0.667$

Vector representation

- Document

$$\vec{d}_j = \{0.3110, 0, 0.5, 0.5\}$$

- Query

$$\vec{q} = \{0.333, 0, 0.667, 0\}$$

$$\text{sim}(d1, q) = \frac{0.3110 \cdot 0.333 + 0 \cdot 0 + 0 \cdot 0.667 + 0 \cdot 0}{\sqrt{(0.3110^2 + 0^2 + 0^2 + 0^2)} \cdot \sqrt{(0.333^2 + 0^2 + 0.667^2 + 0^2)}} = 0.4466$$

$$\text{sim}(d2, q) = \frac{0 \cdot 0.333 + 0 \cdot 0 + 0.5 \cdot 0.667 + 0.5 \cdot 0}{\sqrt{(0^2 + 0^2 + 0.5^2 + 0.5^2)} \cdot \sqrt{(0.333^2 + 0^2 + 0.667^2 + 0^2)}} = 0.4001$$

$$\text{sim}(d3, q) = \frac{0.3110 \cdot 0.333 + 0 \cdot 0 + 0.5 \cdot 0.667 + 0 \cdot 0}{\sqrt{(0.3110^2 + 0^2 + 0.5^2 + 0^2)} \cdot \sqrt{(0.333^2 + 0^2 + 0.667^2 + 0^2)}} = 0.9086$$

$$\text{sim}(d4, q) = \frac{0.3110 \cdot 0.333 + 0 \cdot 0 + 0 \cdot 0.667 + 0 \cdot 0}{\sqrt{(0.3110^2 + 0^2 + 0^2 + 0^2)} \cdot \sqrt{(0.333^2 + 0^2 + 0.667^2 + 0^2)}} = 0.4466$$

Ranking of the documents, (for ranking we have followed the method in statistics for the case of allocating same rank to two different items)

document1	2nd
document2	4th
document3	1st
document4	2nd

Since the similarity between document 3 is greater than the similarities between the other documents, 3rd document is more relevant to the query.

Capture Semantic Meaning

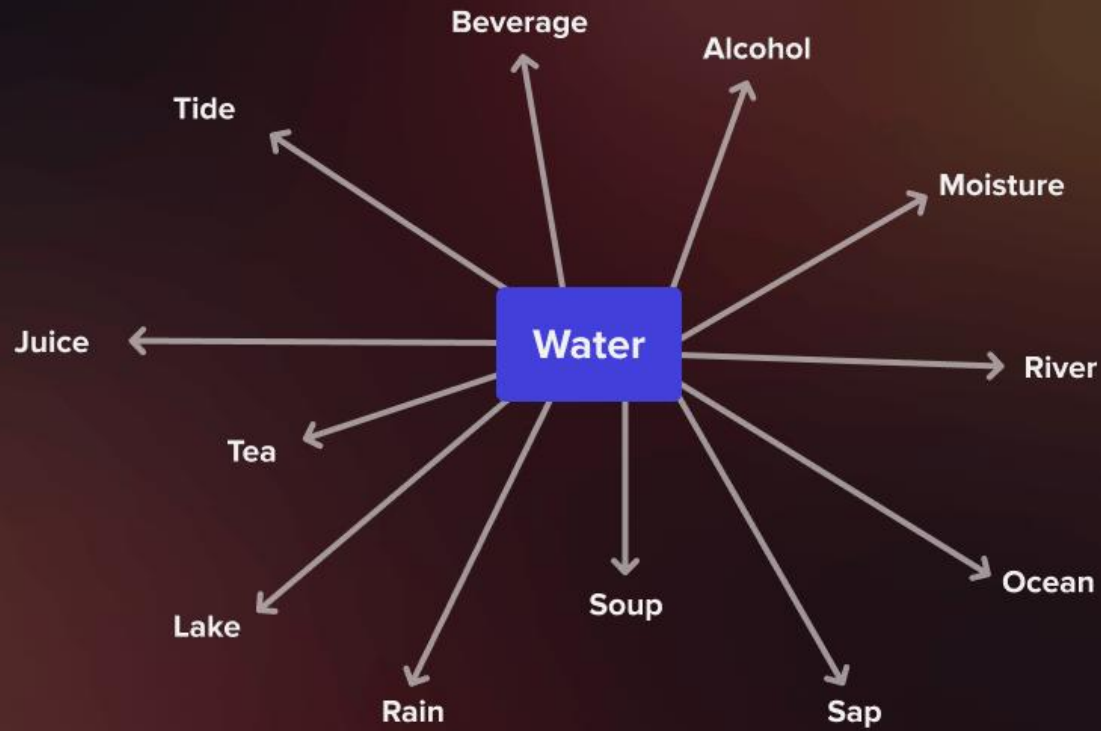
Word embedding is an alternative technique in NLP, whereby words or phrases from the vocabulary are mapped to vectors of real numbers in a low-dimensional space relative to the vocabulary size, and the similarities between the vectors correlate with the words' semantic similarity

For example, let's take the words *woman*, *man*, *queen*, and *king*. We can get their vector representations and use basic algebraic operations to find semantic similarities. Measuring similarity between vectors is possible using measures such as cosine similarity. So, when we subtract the vector of the word *man* from the vector of the word *woman*, then its cosine distance would be close to the distance between the word *queen* minus the word *king*.

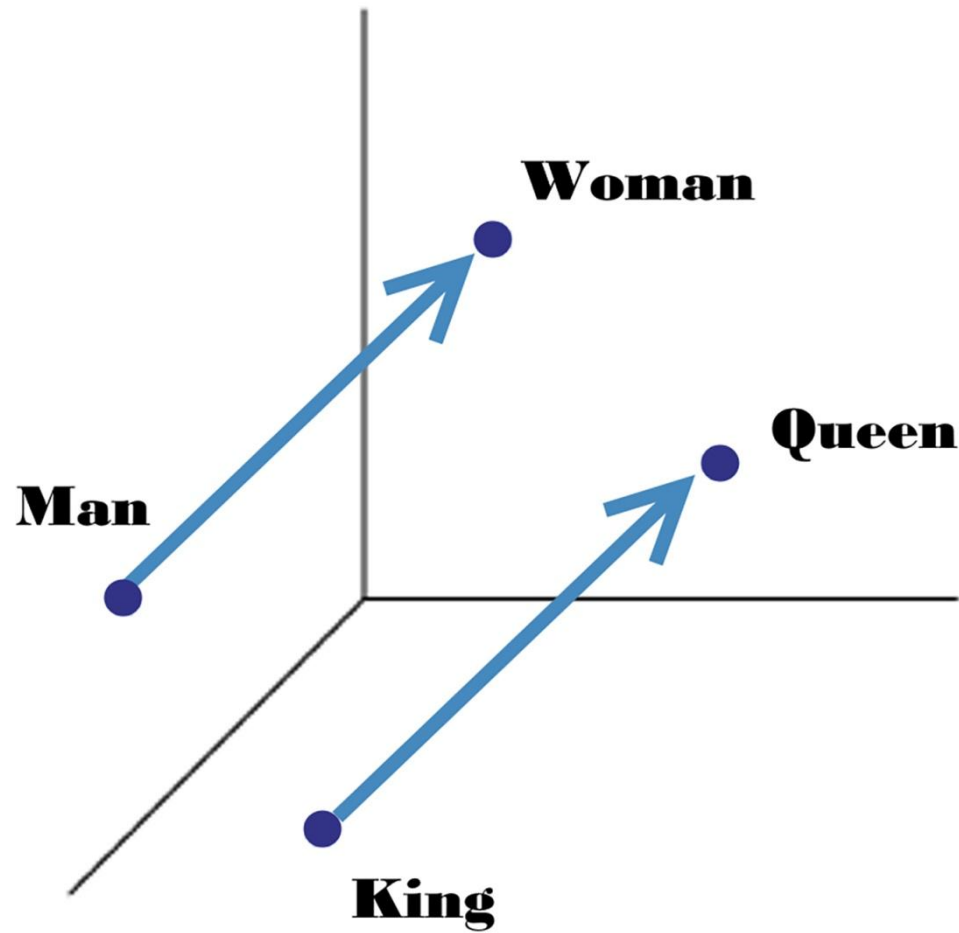
What is a word embedding?

If you ask someone which word is more similar to “king” – “ruler” or “worker” – most people would say “ruler” makes more sense, right? But how do we teach this intuition to a computer? That’s where word embeddings come in handy.

A word embedding is a representation of a word used in text analysis. It usually takes the form of a vector, which encodes the word’s meaning in such a way that words closer in the vector space are expected to be similar in meaning. Language modeling and feature learning techniques are typically used to obtain word embeddings, where words or phrases from the vocabulary are mapped to vectors of real numbers.

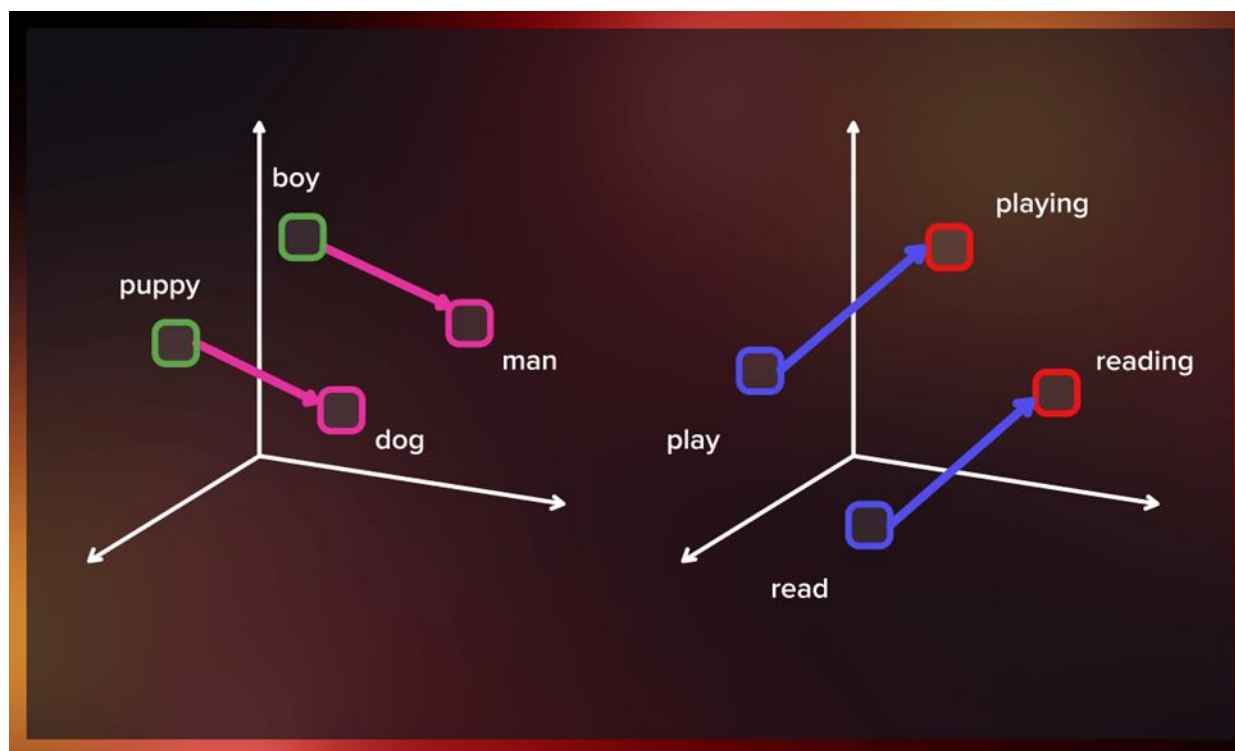


Capture Semantic Meaning



Word2Vec takes a large corpus of text as input and generates a vector space with hundreds of dimensions. Each unique word in the corpus is assigned a vector in this space.

The development of word to vector also involved analyzing the learned vectors and exploring how they can be manipulated using vector analysis. For instance, subtracting the “man-ness” from “King” and adding “women-ness” would result in the word “Queen,” which captures the analogy of “king is to queen as man is to woman.”



How is Word2Vec trained?

Word to vector is trained using a neural network that learns the relationships between words in large databases of texts. To represent a particular word as a vector in multidimensional space, the algorithm uses one of the two modes: continuous bag of words (CBOW) or skip-gram.

Continuous bag-of-words (CBOW)

The continuous bag-of-words model predicts the central word using the surrounding context words, which comprises a few words before and after the current word.

Bag-of-words vector

Raw text

Peter Piper Picked
a Peck of Pickled
Peppers



Peter	1
dog	0
picked	1
and	0
apple	0
cat	0
flower	0
peppers	1
pickled	1
...	...

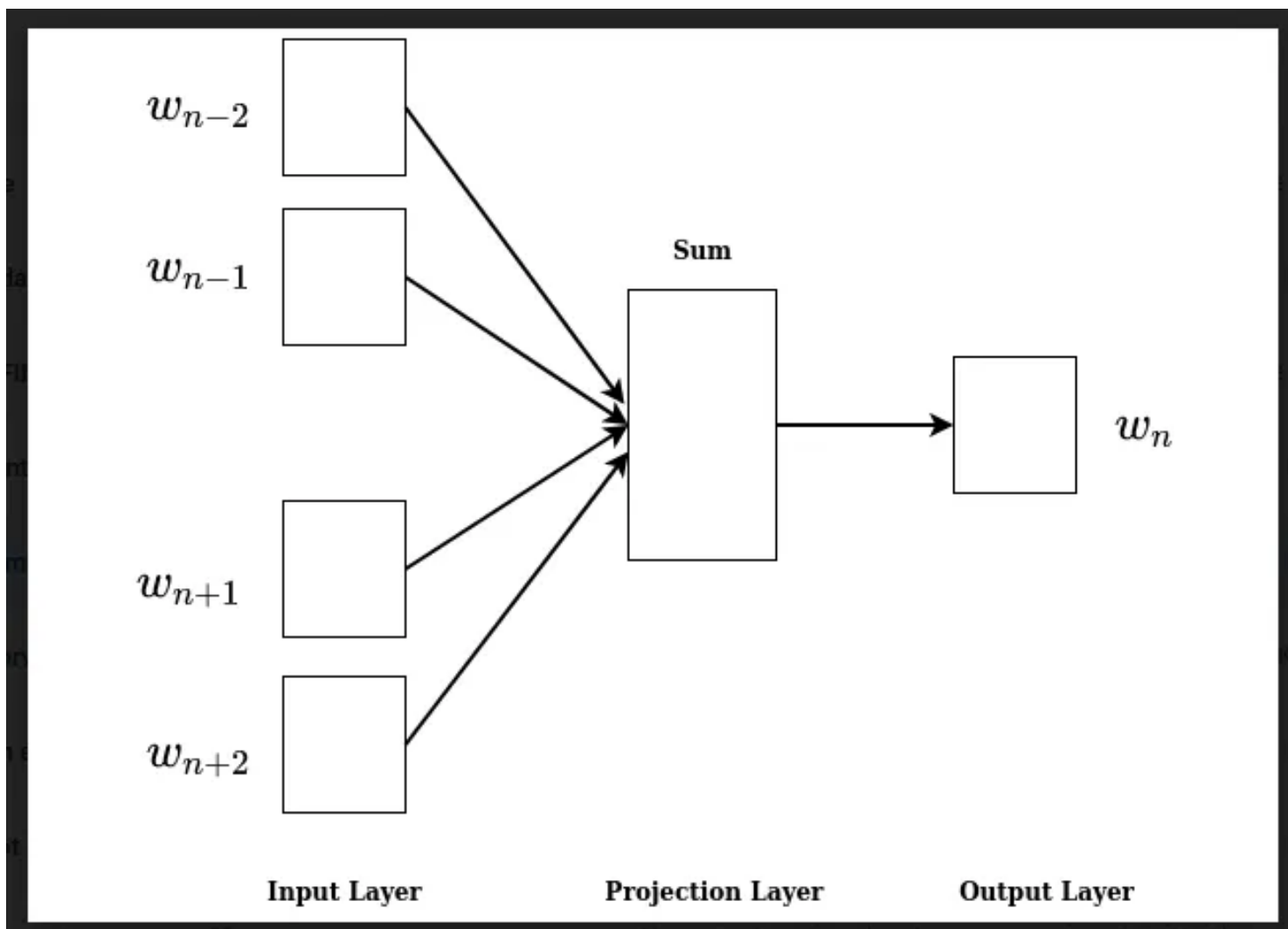
Are you going to
the theater with
Alex tomorrow?
What time does
the play begin?

1	tomorrow
0	today
0	friday
0	go
0	went
0	move
1	play
:	:
0	movie
0	great

Continuous Bag of Words

The Continuous Bag of Words (CBOW) model is a popular method for training word embeddings, which are representations of words in a numerical vector space. The goal of CBOW is to predict a target word given the context words in a sentence. This is in contrast to the skip-gram model, which predicts context words given a target word.

Understanding the Continuous Bag of Words (CBOW) Model: Architecture



Understanding the Continuous Bag of Words (CBOW) Model: Architecture

The architecture of the CBOW model is relatively simple, consisting of an input layer, a hidden layer, and an output layer. The input layer is used to represent the context words, the hidden layer is used to learn the word embeddings, and the output layer is used to predict the target word.

The input layer is typically represented by a one-hot encoded vector, where each element in the vector corresponds to a specific word in the vocabulary. For example, if the vocabulary contains 10,000 words, the input layer will have 10,000 elements.

Understanding the Continuous Bag of Words (CBOW) Model: Architecture

The hidden layer is where the word embeddings are learned. It is a dense layer, with each neuron representing a specific word in the vocabulary. The number of neurons in the hidden layer is the same as the number of words in the vocabulary.

The output layer is also a dense layer, with each neuron representing a specific word in the vocabulary. The number of neurons in the output layer is the same as the number of words in the vocabulary.

The working mechanism of CBOW can be mathematically represented as follows

1. Let $x(1), x(2), \dots, x(n)$ be the context words, where $x(i)$ is a one-hot encoded vector.

Let $W(1)$ be the weight matrix connecting the input layer to the hidden layer, and $W(2)$ be the weight matrix connecting the hidden layer to the output layer.

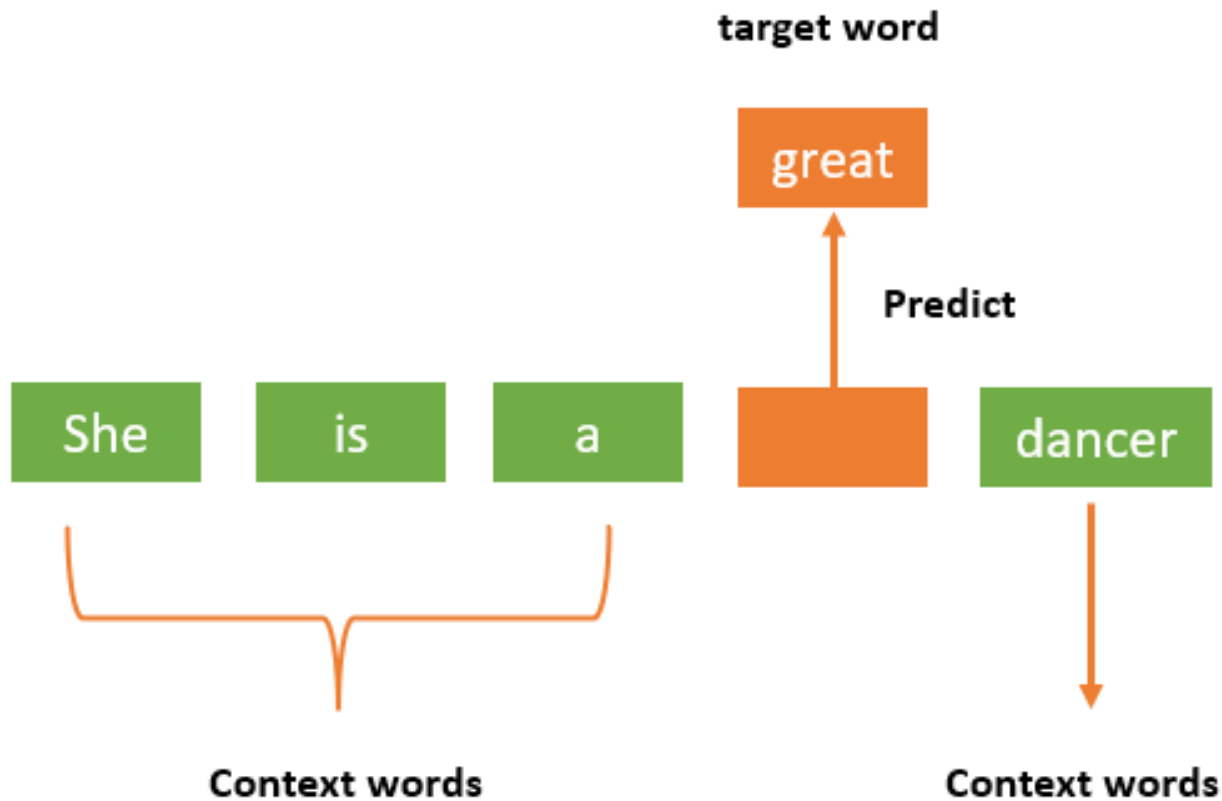
2. Let h be the hidden layer, which is the average of the input vectors,
$$h = 1/n * (x(1) + x(2) + \dots + x(n))$$

3. Let y be the output layer, which is the probability distribution over the vocabulary,

$$y = \text{softmax}(W(2) * h)$$

The target word is selected as the word with the highest probability in y .

Continuous Bag of Words Example



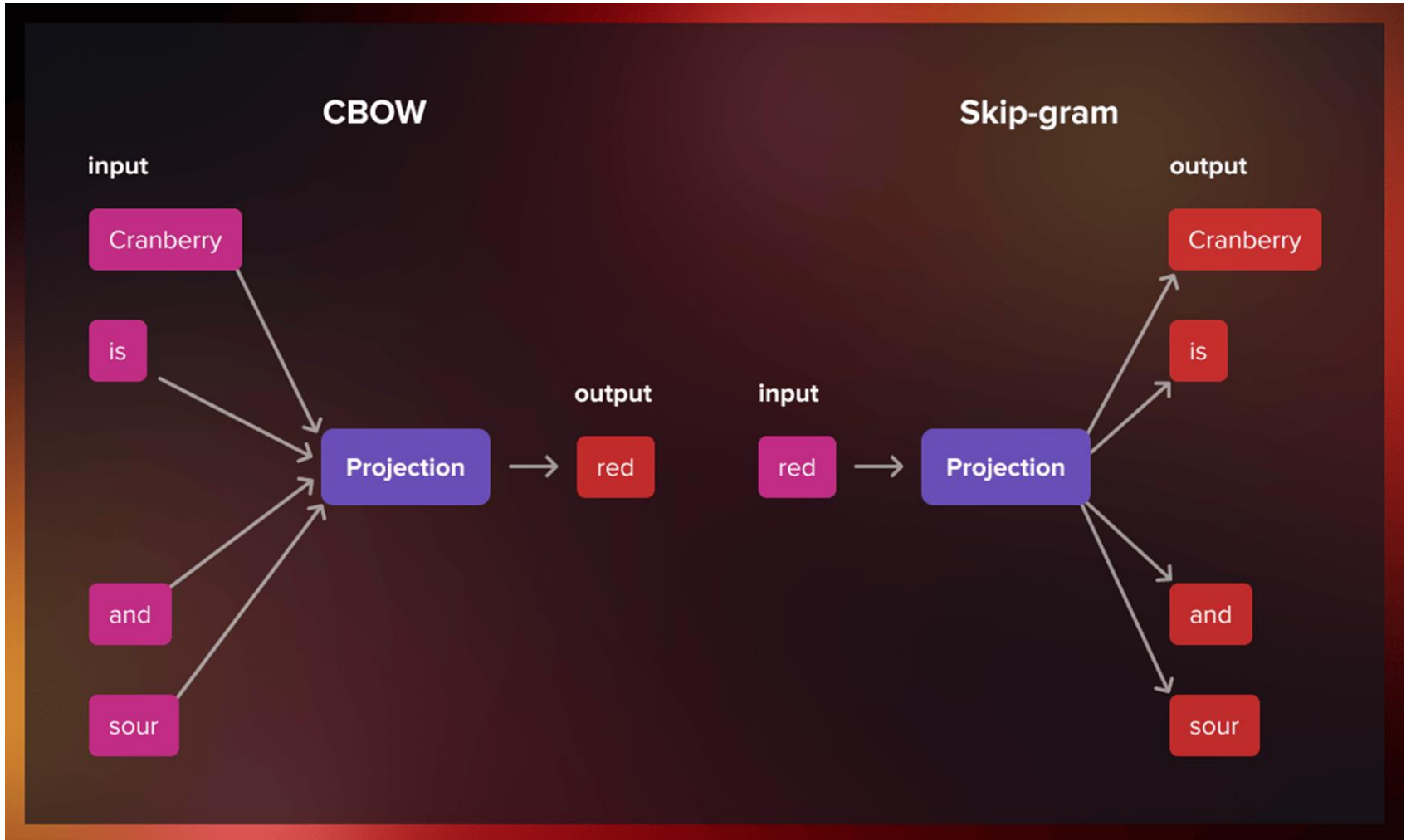
Is there any difference between Bag-of-Words (BoW) model and the Continuous Bag-of-Words (CBOW)?

1. The Bag-of-Words model and the Continuous Bag-of-Words model are both techniques used in natural language processing to represent text in a computer-readable format, but they differ in how they capture context.
2. The BoW model represents text as a collection of words and their frequency in a given document or corpus. It does not consider the order or context in which the words appear, and therefore, it may not capture the full meaning of the text. The BoW model is simple and easy to implement, but it has limitations in capturing the meaning of language.
3. In contrast, the CBOW model is a neural network-based approach that captures the context of words. It learns to predict the target word based on the words that appear before and after it in a given context window. By considering the surrounding words, the CBOW model can better capture the meaning of a word in a given context.

Skip-gram

The skip-gram model architecture is designed to achieve the opposite of the CBOW model. Instead of predicting the center word from the surrounding context words, it aims to predict the surrounding context words given the center word.

The choice between the two approaches depends on the specific task at hand. The skip-gram model performs well with limited amounts of data and is particularly effective at representing infrequent words. In contrast, the CBOW model produces better representations for more commonly occurring words.



Relationships & Dependencies Between Words in NLP

1. Syntactic Dependencies Definition: Structural relationships based on grammar.

Example: In “The cat sat on the mat”, “sat” is the verb, and “cat” is its subject.

Used in: Dependency parsing, POS tagging.

Tools: spaCy, Stanford Parser.

2. Semantic Relationships Definition: Relationships based on meaning.

Types:

Synonymy: Words with similar meaning (big $\leftrightarrow$ large).

Antonymy: Opposite meanings (hot $\leftrightarrow$ cold).Hypernymy/Hyponymy:

General/specific (animal $\leftrightarrow$ dog).

Meronymy: Part-whole (wheel $\leftrightarrow$ car).

Used in: WordNet, semantic similarity, question answering.

REFERENCES

1. [Vector Space Model Made Simple With Examples & Tutorial](#)
2. [Lecture 5 — Vector Space Model Basic Idea | UIUC](#)
3. [5 in 1 Benefits with Axis Max Life Term Plan. Watch to Know More!](#)
4. [PY SD VS74 UV 16x9](#)
5. [PY SD VS82 UV 16x9](#)
6. [Continuous bag of words \(CBOW\) in NLP – GeeksforGeeks](#)
7. [\(PDF\) Two/Too Simple Adaptations of Word2Vec for Syntax Problems](#)
8. [\(PDF\) Extending Thesauri Using Word Embeddings and the Intersection Method](#)
- 9.

INT344

Natural Language Processing

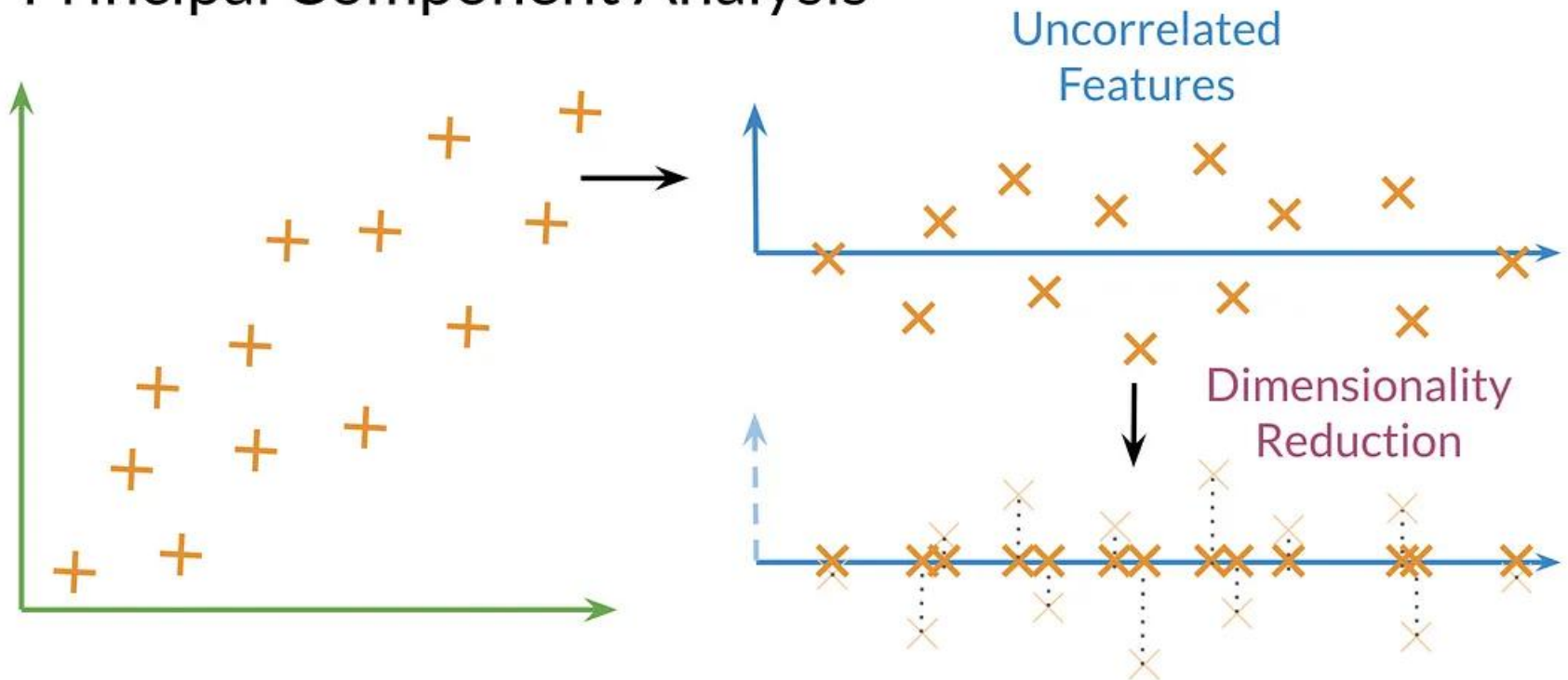
UNIT- 2(Part-II)

Principal Component Analysis (PCA)

1. It's a common practice of reducing the dimension, PCA is an unsupervised learning algorithm that is commonly used for this.
2. Visualization of data is also very important and when we have data in several dimensions, it becomes much more difficult to store large data, perform computation, and visualize and that's why we need to reduce dimensions.
3. The true purpose is mainly to decrease the complexity of the model and to simplify the model while keeping relevance and performance.

Principal Component Analysis (PCA)

Principal Component Analysis



Principal Component Analysis (PCA)

PCA is used for a wide range of applications like:

- i. Image Compression
- ii. Anomaly Detection
- iii. Noise filtering
- iv. Feature Extraction and many more...

Some terms we need to understand

Eigenvector — Uncorrelated features for your data

Eigenvalue — the amount of information retained by each feature

Variance — simply measures how spread the data is

Steps for PCA:

1. Standardization / Mean normalize the data — Mean normalize is a way to make values adjusted to a common/fixed scale {Process is as subtracting every value with the mean of that feature and dividing by the range of values (i.e. max-min)}.

Standardization is when we use standard deviation in the denominator.

2. Obtain covariance matrix

- if positive then: the two variables increase or decrease together (correlated)
- if negative then: One increases when the other decreases (Inversely correlated).

Check this out “[How to calculate covariance matrix](#)”.

Steps for PCA:

3. Obtain Eigenvector and Eigenvalue from covariance matrix or correlation matrix.
4. Now sort the Eigenvalue in decreasing order and get the index (you can use `np.argsort()` to get the index and for decreasing order of array $\rightarrow [::-1]$)
5. Sort Eigenvector simultaneously with Eigenvalue with help of indexes you have got (i.e `eigen_vecs[:,idx_sorted_decreasing]`)
6. Select the dimensions in sorted Eigenvector matrix you want to convert it in (i.e `n_components`).
7. Do product of Transposed Eigenvector Matrix with `n_component` and Transposed Mean normalized matrix and return Transpose of the result.

Machine Translation (MT) in NLP

Definition:

MT is the process of automatically converting text or speech from one language to another using computer algorithms.

Core Functionality:

NLP models are trained to analyze the source language, understand its meaning, and then generate equivalent text or speech in the target language, considering factors like context and nuances.

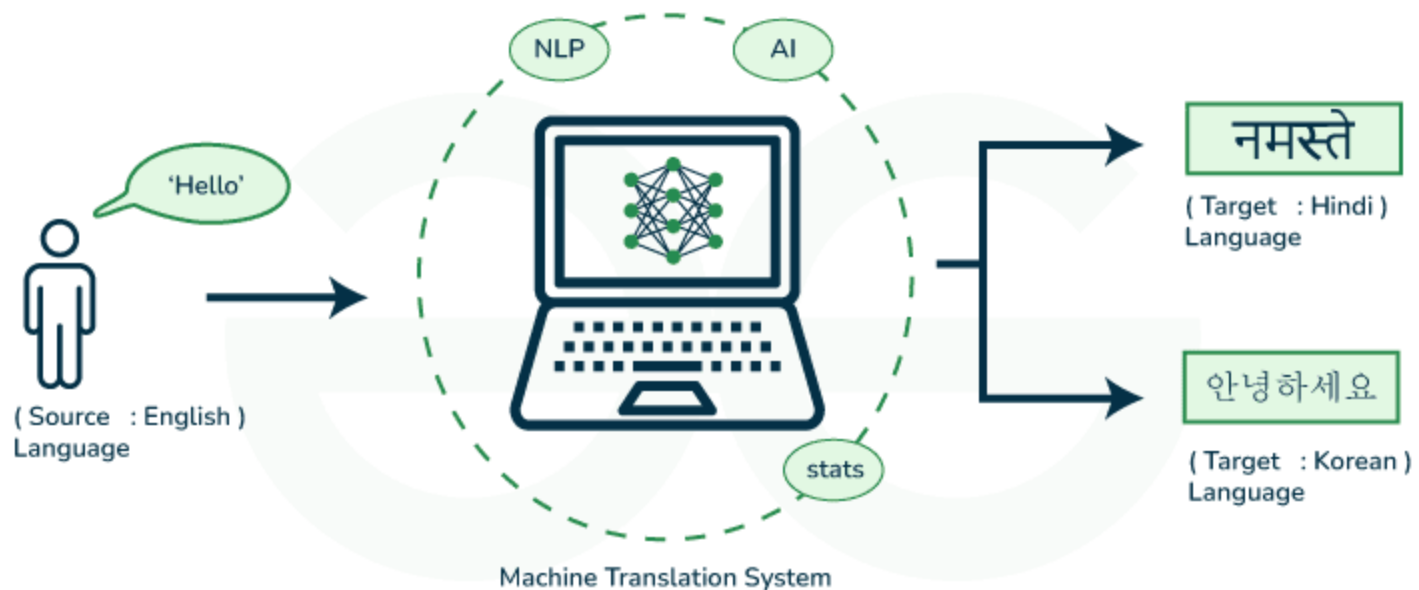
Evolution of MT:

MT has evolved from rule-based systems to statistical machine translation and now to neural machine translation (NMT), with NMT leveraging deep learning to achieve more accurate and human-like translations.

Applications:

MT is used in various applications, including real-time speech translation, website localization, multilingual customer support, and language learning platforms.

Machine Translation (MT) in NLP



Document Search in NLP

- **Core Functionality:**

- Document search in NLP involves using NLP techniques to understand the meaning of both the user's query and the content of documents, allowing for more accurate and relevant search results.

- **NLP Techniques Used:**

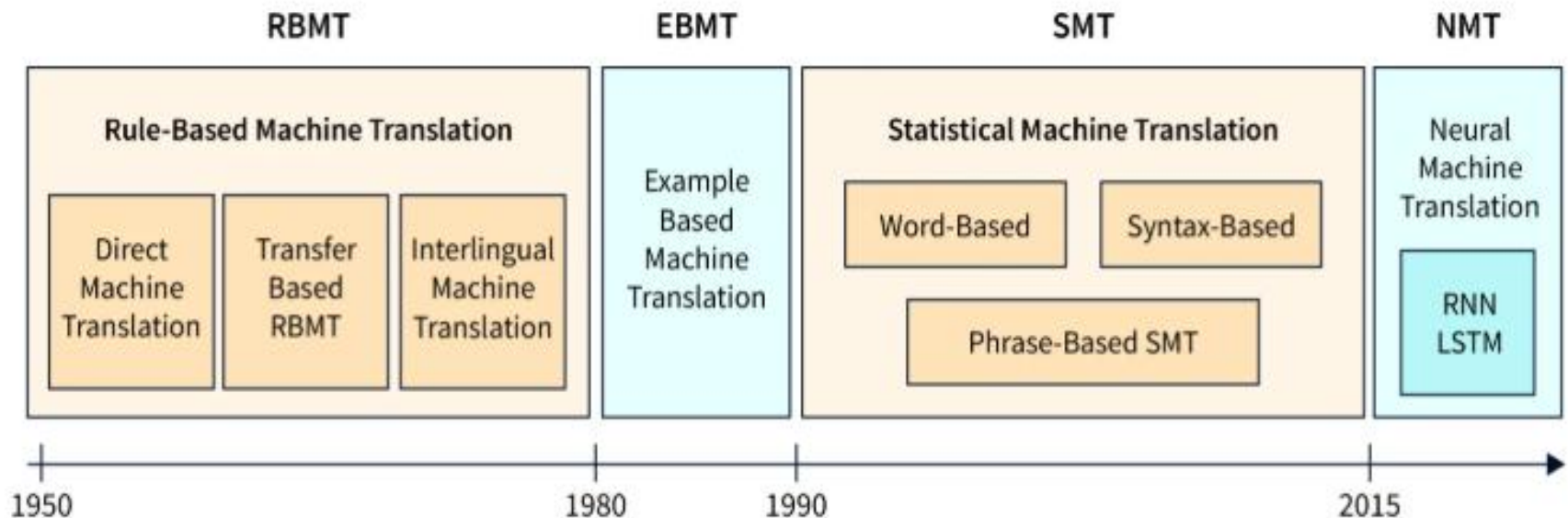
- **Tokenization:** Breaking down text into smaller units (tokens) like words or phrases.

- **Text Cleaning:** Removing irrelevant characters and formatting to improve accuracy.

- **Keyword Extraction:** Identifying important keywords and concepts within documents.

- **Semantic Analysis:** Understanding the meaning and relationships between words and phrases.

Evolution of Machine Translation



Evolution of Machine Translation

- 1950 to 1980: The major interest and support for machine translation in the decades from the 1950s to the 1980s was fueled by the need for high-speed, high-quality translation of arbitrary texts, which are heavily funded mostly by military and intelligence communities during the cold war era.
 - The systems during this time were mainly rule-based, and almost all research and development centred around improving the performance of individual use cases.
- 1980-1990: In the late 1980s, example-based machine translation models were employed for some time till the early 1990s.
 - The main idea is to provide the system with a set of sentences within the source language and then the corresponding translations in the target language. Both examples serve as examples for translations.

Evolution of Machine Translation

- 1990-2015: Machine translation systems based on statistical methods were introduced in the 1990s, replacing the earlier rule-based and example-based systems.
 - The statistical-based methods come under the case of corpus-based machine translation methods.
- 2015-Present: Deep neural network-based models were developed for automatic machine translation by almost all technology companies for industrial applications, which achieved state-of-the-art results in the last few years recently among all NLP techniques.
 - This recent utility of neural net-based models made the field named neural machine translation.

Different types of Machine Translation Methods

1. Rule-based Machine Translation or RBMT

Also called **knowledge-based machine translation**, these are the earliest set of classical methods used for machine translation.

- These translation systems are mainly based on **linguistic information** about the source and target languages that are derived from dictionaries and grammar covering the characteristic elements of each language separately.
- Once we have input sentences in some source languages, RBMT systems generally generate the translation to output language based on the morphological, syntactic, and semantic analysis of both the source and the target languages involved in the translation tasks.
- The sub-approaches under RBMT systems are the direct machine translation approach, the interlingual machine translation approach, and the transfer-based machine translation approach.

2. Corpus-based Machine Translation Approach or CBMT

Also called data **driven machine translation**, these methods majorly overcome the problem of knowledge acquisition problem of rule-based machine translation approaches. The Corpus-based machine translation methods can handle linguistic issues and generally find it difficult to translate cultural items and concepts.

- Corpus-Based Machine Translation is a set of methods that uses a **bilingual parallel corpus** to obtain knowledge for new incoming translation.
- This approach uses a large amount of raw data in the form of parallel corpora. This raw data contains text and their translations. These corpora are used for acquiring translation knowledge.
- Corpus-based approaches are classified into **two sub-approaches**: Statistical Machine Translation and the Example-based Machine Translation approach.

Different types of Machine Translation Methods

3. Example-based Machine Translation or EBMT

Example-based machine translation systems are trained from **bilingual parallel corpora** where the sentence pairs contain sentences in one language with their translations into another.

- The foundational idea of example-based machine translation is the concept of translation by analogy.
 - The principle of translation by analogy is encoded to the example-based machine translation through the example translations that are used to train the system.
- The bilingual corpus with parallel texts serves as the main knowledge for an Example-based machine translation system in which translation by analogy.

Simple Example MT

I'm going to the ground = ^{Already familiar example} मैं मैदान के लिए जा रहा हूँ

I'm going to the **movies**. = मैं फिल्मों के लिए जा रहा हूँ


4. Statistical Machine Translation or SMT

Statistical machine translation (SMT) models are generated with the use of statistical models whose parameters are derived from the analysis of bilingual text corpora. The ideas for SMT come from information theory.

- The principle theory of SMT-based models is that they rely on the **Bayes Theorem** to perform the analysis of parallel texts.
 - The central idea is that every sentence in one language is a **possible translation** of any sentence in the other language, but the most appropriate is the translation that is assigned the highest probability by the system.
- The SMT methods need millions of sentences in both the source and target languages for analysis to collect the relevant statistics for each word.
 - In the early set of models, the abstracts of the European Parliament and the United Nations Security Council meetings, which were available in the languages of all member countries, were taken for getting the required probabilities.
- SMT-based methods are further **classified** into Word-based SMT, Phrase-based SMT, and Syntax based SMT approaches.

5. Hybrid Machine Translation or HMT

- Hybrid Machine Translation approaches take advantage of both statistical and rule-based translation methodologies which were proven to have better efficiency in the area of machine translation systems using NLP.
 - These techniques are better than the previous methods and have more power, flexibility, and control in translation tasks in general.
 - Most governmental and industrial-based machine translation systems use the hybrid-based approach to develop translation from source to target language, which is based on both rules and statistics.

Hybrid-based approaches combine the best of both worlds such that they can be used in several different ways.

In cases where translations are performed in the first stage using a rule-based approach, they can be followed by adjusting or correcting the output using statistical information.

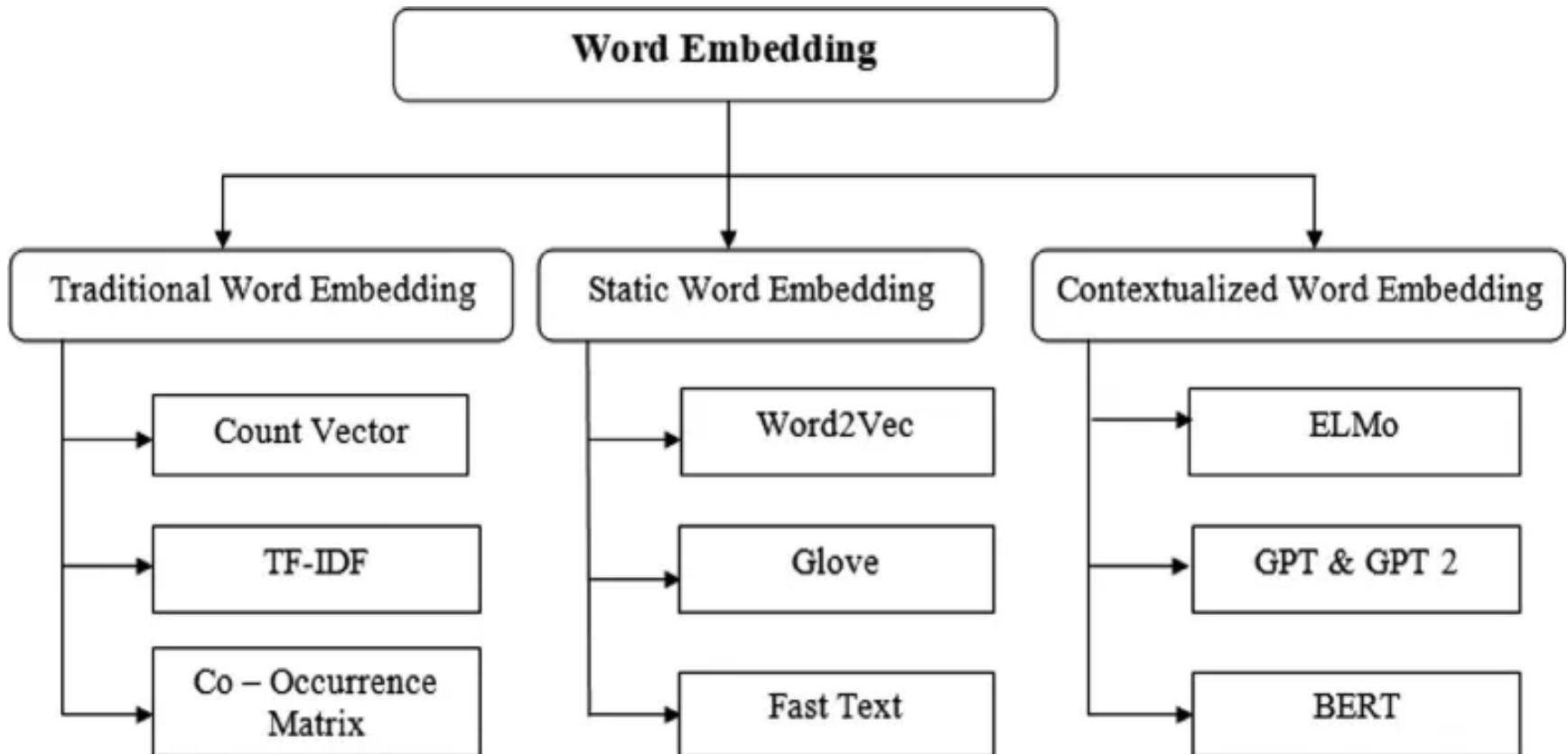
On the other way, rules are used to pre-process the input data as well as post-process the statistical output of a statistical-based translation system.

6. Neural Machine Translation or NMT

Neural Machine Translation methods in NLP employ artificial intelligence techniques and, in particular, rely upon neural network models which were originally based on the human brain to build machine learning models with the end goal of translation and also further learning languages, and improving the performance of the machine translation systems constantly.

- Neural Machine Translation systems do not need any specific requirements that are regular to other machine translation systems like statistical-based methods in general.
- Neural Machine Translation works by incorporating training data, which can be generic or custom depending on the user's needs.
 - **Generic Data:** Includes the total of all the data learned from translations performed over time by the machine translation engines employed in the entire realm of translation.
 - This data also serves as a generalized translation tool for various applications, including text, voice, and documents.

Transform Word Vectors



REFERENCES

1. [Principal Component Analysis \(PCA\) - Scaler Topics](#)
2. [Principal Component Analysis \(PCA\) \[NLP, Python\] | by Yash Jain | Medium](#)
3. [Mod-01 Lec-30 Principal Component Analysis \(PCA\)](#)
4. [Lec 26: Dimensionality Problem and Principal Component Analysis](#)
5. [NPTEL : NOC:Applied Natural Language Processing \(Computer Science and Engineering\)](#)
6. [Machine Translation in NLP](#)
7. [A Comprehensive Guide to Word Embeddings in NLP | by Harsh Vardhan | Medium](#)

THANK YOU

NLP Practical Programs

Unit – 2

Vector Space Models

Extracting Binary Vectors

```
from sklearn.feature_extraction.text import  
CountVectorizer  
  
docs = ["the cat sat on the mat", "the dog sat on the  
floor", "cats and dogs"]  
  
vec = CountVectorizer()  
  
X = vec.fit_transform(docs)  
  
print("Doc-term matrix:\n", X.toarray())  
  
print("Features:", vec.get_feature_names_out())
```

Extracting TF-IDF vectors

```
from sklearn.feature_extraction.text import  
TfidfVectorizer  
  
docs = ["the cat sat on the mat", "the dog sat on the  
floor", "cats and dogs"]  
tfidf = TfidfVectorizer()  
X = tfidf.fit_transform(docs)  
print("TF-IDF matrix:\n", X.toarray().round(2))  
print("Features:", tfidf.get_feature_names_out())
```

Cosine similarity between documents

```
from sklearn.feature_extraction.text import  
TfidfVectorizer
```

```
from sklearn.metrics.pairwise import  
cosine_similarity
```

```
docs = ["the cat sat on the mat", "the dog sat on  
the floor", "cats and dogs"]
```

```
tfidf = TfidfVectorizer().fit_transform(docs)
```

```
sim = cosine_similarity(tfidf)
```

```
print(sim)
```

Semantic search — query → best document

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
docs = ["Deep learning for NLP", "Introduction to algorithms",
        "Natural language processing with transformers"]
tfidf = TfidfVectorizer().fit_transform(docs).toarray()
query = "NLP transformers"
qv = TfidfVectorizer(vocabulary =
TfidfVectorizer().fit(docs).vocabulary_ )
qv = qv.fit(docs).transform([query])
sim = cosine_similarity(qv, tfidf)
print(sim)
```

N-grams (bigrams) with CountVectorizer

```
from sklearn.feature_extraction.text import  
CountVectorizer  
  
docs = ["new york city", "new york times", "city  
of new york"]  
  
vec = CountVectorizer(ngram_range=(2,2))  
X = vec.fit_transform(docs)  
print(X.toarray())  
print("Bigrams:", vec.get_feature_names_out())
```

TF-IDF Feature Extraction using Bigram Model

```
from sklearn.feature_extraction.text import  
TfidfVectorizer  
  
import pandas as pd  
  
docs = [ "the cat sat on the mat", "the dog barked  
loudly", "the cat chased the mouse", "the dog sat near  
the cat", "the mouse ran away quickly" ]  
  
vec = TfidfVectorizer(ngram_range=(2, 2))  
  
X = vec.fit_transform(docs)  
  
df = pd.DataFrame(X.toarray(), columns=  
vec.get_feature_names_out())  
  
print("TF-IDF Feature Matrix => ", df.round(2))
```

Simple Word2Vec (CBOW) for a single token

```
from gensim.models import Word2Vec  
corpus = [  
    ["the", "cat", "sat", "on", "the", "mat"],  
    ["the", "dog", "barked"],  
    ["the", "cat", "chased", "the", "mouse"]]  
model = Word2Vec(sentences=corpus, vector_size = 40,  
window=2, min_count=1, sg=0) # CBOW  
print("Vector shape for 'cat':", model.wv['cat'].shape)  
print(model.wv.most_similar('cat', topn=5))
```


Simple Word2Vec (Skip-gram)

```
from gensim.models import Word2Vec
corpus = [
    "king queen prince princess man woman".split(),
    "the king sat on his throne".split(),
    "the queen ruled the kingdom".split()
]
model_sg = Word2Vec(sentences=corpus,
vector_size=40, window=2, min_count=1, sg=1) #
Skip-gram
print("Most similar to 'king':",
model_sg.wv.most_similar('king', topn=5))
```

Word Analogy (king – man + woman ≈ ?)

```
from gensim.models import Word2Vec
sentences = [
    "king queen man woman prince princess".split(),
    "king is a male ruler".split(), "queen is a female
ruler".split(), "man woman boy girl".split() ]
m = Word2Vec(sentences, vector_size=50, window=2,
min_count=1, sg=0)
print(m.wv.most_similar (positive = ['king', 'woman'],
negative=['man'], topn=3))
```

See you in the next class !!!

TEXT PRE-PROCESSING USING NLTK

How to install nltk ?

- `pip install nltk`
- `!pip install nltk`

Check your NLTK version

- `import nltk`
- `print(nltk.__version__)`

Important downloads

- `nltk.download('punkt_tab')` # Tokenizer
- `nltk.download('stopwords')` # Stopwords
- `nltk.download('wordnet')` # Lemmatizer

Important downloads

- `nltk.download('punkt_tab')` # Tokenizer
- punkt is a **pretrained unsupervised sentence tokenizer model**

Word Tokenization

```
import nltk
nltk.download('punkt_tab')
text = "We are studying INT344 now and doing
practical"
s = nltk.word_tokenize(text)
print(s)
```

Remove duplicates in tokens

```
tokens = ["the", "cat", "sat", "on", "the", "mat",  
"mat", "sat"]  
unique_tokens = list(set(tokens))  
print(unique_tokens)
```

Case Normalization

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')
documents = ['The cat sat on the mat', 'The dog chased
the cat', 'The dog and cat are friends']
documents_sent = [ ]
for doc in documents :
    documents_sent.append(doc.lower())

print(documents_sent)
```

Stop Words Removal

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')
nltk.download('stopwords')
text = "This is a practical class on NLP subject"
tokens = word_tokenize(text)
stop_words = stopwords.words('english')
filtered_tokens = [ ]
for w in tokens :
    if w.lower() not in stop_words :
        filtered_tokens.append(w)
print(filtered_tokens)
```

Stemming

```
import nltk
from nltk.stem import PorterStemmer
ps = PorterStemmer()
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')
text = "organize organized organizing organizes"
tokens = word_tokenize(text)
stemmed_token = [ ]
for w in tokens :
    t = ps.stem(w)
    stemmed_token.append(t)

print( stemmed_token )
```

Lemmatization

```
import nltk
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
lem = WordNetLemmatizer()
text = "organizes"
output = lem.lemmatize(text, pos='v')
print(output)
```

Parsing Vs Parts of Speech

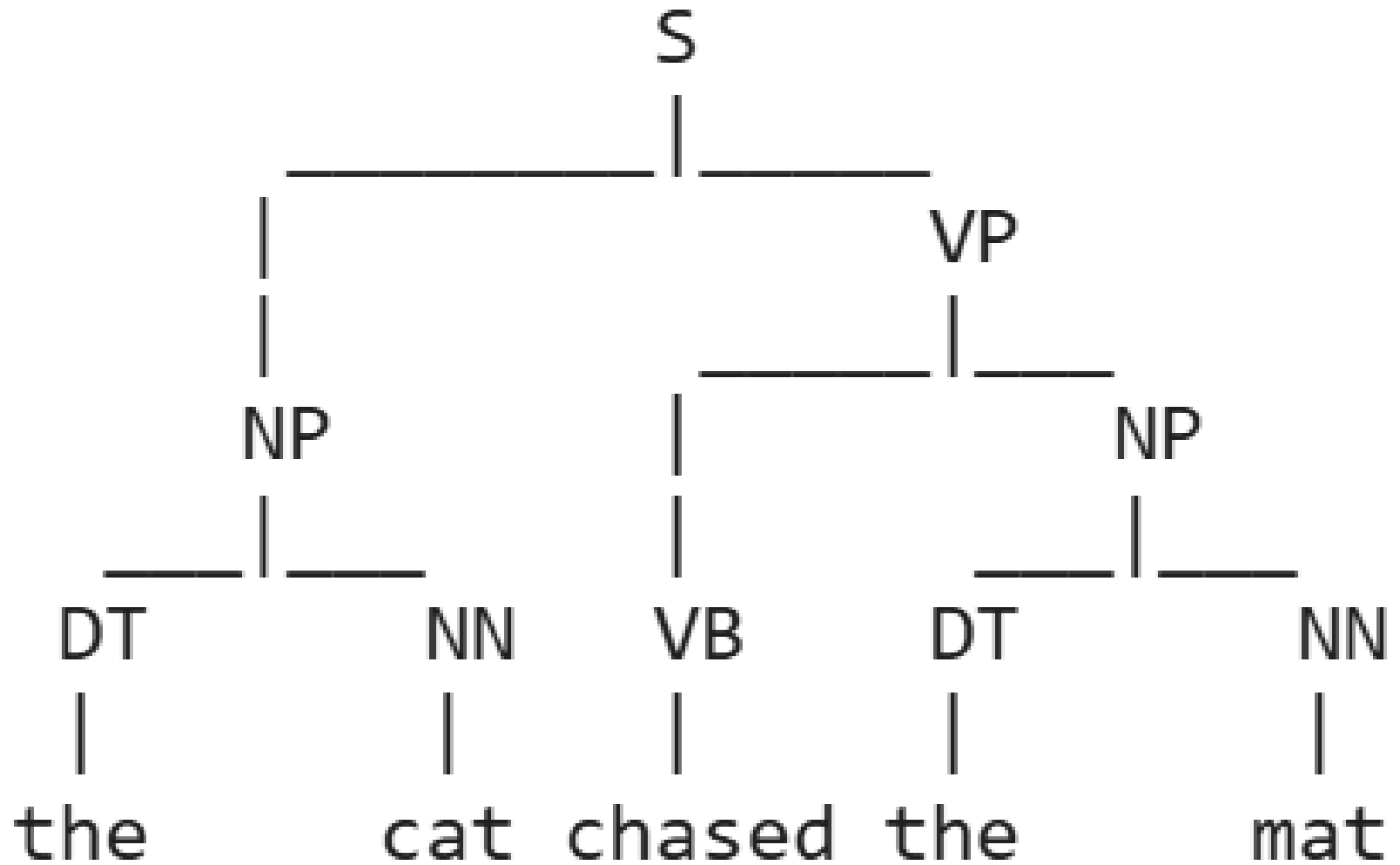
Task	Purpose	Helps With...
Parsing	Understand sentence structure	Meaning extraction, semantic analysis, QA systems
POS Tagging	Identify word roles (noun, verb, etc.)	Word sense disambiguation, grammar checks

POS

- **Disambiguation:** Helps distinguish between different meanings of a word.
- Example:
- “*Book a flight*” → “book” is a **verb**
- “*Read a book*” → “book” is a **noun**

Parsing

```
import nltk
from nltk import CFG
grammar = CFG.fromstring(""" S -> NP VP
NP -> DT NN    VP -> VB NP  DT -> 'the'
NN -> 'cat' | 'mat' VB -> 'chased' """)
parser = nltk.ChartParser(grammar)
sentence = ['the', 'cat', 'chased', 'the', 'mat']
for tree in parser.parse(sentence):
    tree.pretty_print()
```



Morphology in NLP

- It is the study of how words are formed from smaller units of meaning called morphemes.
- Morphological analysis involves breaking down words into their constituent parts, such as roots, prefixes, and suffixes, to understand their structure and meaning.

Types of Morphology in NLP

Type	Description	Example
Inflectional	Changes form without changing meaning or word class	walk → walks, walked
Derivational	Changes form and meaning or word class	happy → unhappy, happy → happiness

Example word: unhappiness

Segments (morphemes): un + happy + ness

morpheme segmentation

BoW Model

Text Data



Bag of words

	cat	cute	dog	small
	-----	-----	-----	-----
'small dog'	0	0	1	1
'cute cute cat'	1	2	0	0
'cute dog'	0	1	1	0

Vector Representation

Document D1	<i>The child makes the dog happy</i> the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	<i>The dog makes the child happy</i> the: 2, child: 1, makes: 1, dog: 1, happy: 1



	child	dog	happy	makes	the	BoW Vector representations
D1	1	1	1	1	2	[1,1,1,1,2]
D2	1	1	1	1	2	[1,1,1,1,2]

Binary Vector Representation

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

What does TF in TF-IDF stand for?

- A) Total Frequency
- B) Term Frequency
- C) Text Frequency
- D) Token Frequency

What does IDF in TF-IDF stand for?

- A) Inverse Document Frequency
- B) Indexed Document Factor
- C) Important Document Frequency
- D) Internal Document Feature

The purpose of IDF is to:

- A) Increase the weight of common terms
- B) Penalize rare terms
- C) Reduce the weight of frequent terms
- D) Ignore stop words

The purpose of IDF is to:

- A) Increase the weight of common terms
- B) Penalize rare terms
- C) Reduce the weight of frequent terms 
- D) Ignore stop words

Which of the following best describes the TF-IDF score?

- A) Measures how often a term appears across all documents
- B) Measures the importance of a term in one document relative to all documents
- C) Measures the frequency of all words
- D) Measures similarity between documents

Which of the following best describes the TF-IDF score?

- A) Measures how often a term appears across all documents
- B) Measures the importance of a term in one document relative to all documents 
- C) Measures the frequency of all words
- D) Measures similarity between documents

If a term appears in every document, its IDF is:

- A) Very high
- B) Zero
- C) One
- D) Undefined

If a term appears in every document, its IDF is:

- A) Very high
- B) Zero 
- C) One
- D) Undefined

TF is usually calculated as:

- A) $\log(1 + \text{term count})$
- B) $\text{term count} / \text{total terms in document}$
- C) $\text{total terms} / \text{term count}$
- D) $\text{number of documents} / \text{term frequency}$

TF is usually calculated as:

- A) $\log(1 + \text{term count})$
- B) $\text{term count} / \text{total terms in document}$ 
- C) $\text{total terms} / \text{term count}$
- D) $\text{number of documents} / \text{term frequency}$

What happens to TF-IDF score for stopwords like "the", "is", etc.?

- A) It increases
- B) It remains the same
- C) It becomes very low or zero
- D) It becomes very high

What happens to TF-IDF score for stopwords like "the", "is", etc.?

- A) It increases
- B) It remains the same
- C) It becomes very low or zero 
- D) It becomes very high

- **Which of these libraries has built-in TF-IDF support in Python?**
- A) matplotlib
- B) nltk
- C) sklearn
- D) pandas

- **Which of these libraries has built-in TF-IDF support in Python?**
- A) matplotlib
- B) nltk
- C) sklearn 
- D) pandas

- **In Scikit-learn, which class is used to compute TF-IDF values?**
- A) CountVectorizer
- B) TfidfTransformer
- C) TfidfVectorizer
- D) TextVectorizer

- **In Scikit-learn, which class is used to compute TF-IDF values?**
- A) CountVectorizer
- B) TfidfTransformer
- C) TfidfVectorizer 
- D) TextVectorizer

- **TF-IDF is an example of which type of model?**
- A) Probabilistic
- B) Statistical
- C) Vector Space Model
- D) Neural Model

- **TF-IDF is an example of which type of model?**
- A) Probabilistic
- B) Statistical
- C) Vector Space Model 
- D) Neural Model

- **If a term appears more frequently in a document, its TF:**
- A) Decreases
- B) Increases
- C) Becomes constant
- D) Is ignored

- **If a term appears more frequently in a document, its TF:**
- A) Decreases
- B) Increases 
- C) Becomes constant
- D) Is ignored

- In a document of 100 words, the word "**data**" appears 5 times. What is the **Term Frequency (TF)** of the word "data"?

- There are **10 documents** in a corpus. The word "**science**" appears in **2 of them**. What is the **Inverse Document Frequency (IDF)** ?

You have two terms in the same document:

"learning": appears 4 times, document length = 200

"AI": appears 2 times, document length = 200

Corpus has 100 documents:

"learning" appears in 20 documents

"AI" appears in 2 documents

Calculate TF-IDF for both and identify which is higher.

- In a corpus of 500 documents, the word "the" appears in all 500 documents. What is its IDF score?

Do explore more in NLTK ...

Unit-3

Natural Language Processing with Probabilistic Models

Natural Language Processing with Probabilistic Models : Autocorrect: Minimum Edit Distance, Spellchecker to Correct Misspelled Words, Part of Speech Tagging and Hidden Markov Models: About Markov Chains and Hidden Markov Models, Part-Of-Speech Tags using a Text Corpus, Autocomplete and Language Models: N-gram Language Models work by Calculating Sequence Probabilities, Autocomplete Language Model using a Text Corpus, Word Embeddings with Neural Networks: Word Embeddings, Semantic Meaning of Words.

Introduction

- NLP with Probabilistic Models
- Autocorrect, HMMs, POS Tagging, N-grams, Embeddings
- Foundation of classical and modern NLP

Autocorrect & Minimum Edit Distance

- Typing errors: insertion, deletion, substitution
- Levenshtein Distance
- Dynamic Programming approach
- Used in spellcheckers and search engines

Autocorrect & Minimum Edit Distance

- Example 1: 'spel'ing' → 'spelling'
- Edit distance = 1 (insert 'l')
- Example 2: 'hte' → 'the'
- Edit distance = 1 (substitute 'h'↔'t')

Probabilistic Spell Checking

- Noisy Channel Model
- Correct Word = $\operatorname{argmax} P(w) * P(\text{error} \mid w)$
- Uses word frequency + error likelihood

Probabilistic Spell Checking

- Example 1: 'definatly' → 'definitely'
- Higher $P(\text{word})$ + small edit distance
- Example 2: 'recieve' → 'receive'
- Noisy channel: $P(\text{error} | \text{word})$ considered

Noisy Channel Model

- A probabilistic model used to recover the original message after noise corrupts it.
- Used in: Spell checking, Speech recognition, OCR, Machine translation.

Core Idea

- User intends a correct word.
- Noise occurs (typing, hearing, scanning errors).
- We observe a distorted word.
- Goal: Predict the most likely original word.

Noisy Channel Formula

- We choose the word w that maximizes:
- $P(w) \times P(\text{observed} \mid w)$
- $P(w)$: How common the word is (language model).
- $P(\text{observed} \mid w)$: How likely this error is (error model).

Diagram

- Original word (w) $\rightarrow$ [Channel with Noise] $\rightarrow$ Observed word (x)
- Decoding direction:
- Observed word (x) $\rightarrow$ Predict original word (w)

Spell Checker Example

- Observed word: 'recieve'
- Possible original words: receive, recipe, recite, reprice
- $P(\text{receive})$ is high, $P(\text{recieve} \mid \text{receive})$ is also high
→ likely correction
- Chosen result: 'receive'

Common Example

- User types: 'teh'
- Common intended words: 'the', 'tech', 'ten'
- $P(\text{the}) \times P(\text{teh} \mid \text{the})$ is highest $\rightarrow$ Correct output: 'the'

Why Called Noisy Channel?

- From Information Theory: messages get corrupted by noise.
- We try to reconstruct the original message.
- NLP applies this logic to language errors.

Summary

- Noisy Channel helps reverse noisy inputs.
- Decision rule: $\text{Original} = \text{argmax } P(w) \times P(\text{observed} \mid w)$.
- Foundation of classical spell checking systems.

Markov Chains

- Next state depends only on current state
- Used for sequential modeling
- Basis for HMMs and N-grams

Markov Chains

- Example 1: Weather: Rainy $\rightarrow$ Sunny (0.3),
Rainy $\rightarrow$ Rainy (0.7)
- Next state depends only on current state
- Example 2: After 'the', nouns often follow
- Probabilistic next-state prediction

Hidden Markov Models (HMM)

- Hidden states: POS tags
- Observations: words
- Transition & emission probabilities
- Viterbi algorithm for decoding

Hidden Markov Models (HMM)

- Example 1: Word 'flies' → noun or verb
- HMM decides based on transition probabilities
- Example 2: 'lead' (metal/verb)
- HMM selects best tag based on context

POS Tagging with HMM

- Assign syntactic categories to words
- Train using tagged corpora
- Resolve ambiguity using probabilities

POS Tagging with HMM

- Example 1: 'Time flies fast'
- HMM tags: Noun Verb Adjective
- Example 2: 'Book the flight'
- HMM tags: Verb Det Noun

N-gram Language Models

- Estimate $P(\text{word sequence})$
- Unigram, Bigram, Trigram models
- Data sparsity & smoothing

N-gram Language Models

- Example 1: Bigram: $P(\text{'dog'} | \text{'the'})$
- Computed from corpus counts
- Example 2: Trigram: $P(\text{'to'} | \text{'want'}, \text{'I'})$
- Predict next word using context

Goal of the Example

- We want to compute the bigram probability:
- $P(\text{cat} \mid \text{the})$
- Meaning: Probability that the next word is 'cat' given previous word 'the'.

Simple Corpus Used

- 1. the cat sleeps
- 2. the cat eats
- 3. the dog barks
- 4. the dog eats
- Tiny corpus so calculations are easy.

Extracted Bigrams

- From 'the cat sleeps' → (the, cat), (cat, sleeps)
- From 'the cat eats' → (the, cat), (cat, eats)
- From 'the dog barks' → (the, dog), (dog, barks)
- From 'the dog eats' → (the, dog), (dog, eats)

Counting Occurrences

- Count bigrams starting with 'the':
- (the, cat) = 2 times
- (the, dog) = 2 times
- Total = 4

Count Specific Bigram

- Count (the, cat):
- (the, cat) appears = 2 times

Bigram Probability Calculation

- Formula: $P(y \mid x) = \text{Count}(x, y) / \text{Count}(x, *)$
- $P(\text{cat} \mid \text{the}) = 2 / 4 = 0.5$
- Final Answer: 0.5 (50%)

Meaning of Result

- In this tiny corpus:
- 50% of the time after the word 'the', the next word is 'cat'.
- This is how a bigram model learns probability of next words.

Autocomplete Systems

- Predict next word using N-grams
- Prefix matching, beam search
- Used in mobile keyboards & search engines

Autocomplete Systems

- Example 1: 'I want to ____'
- Predictions: eat, go, buy
- Example 2: 'New York is a ____'
- Predictions: city, place, state

Word Embeddings

- Dense vector representation of words
- Semantic similarity & analogies
- Word2Vec, GloVe, FastText

Word Embeddings

- Example 1: king - man + woman $\approx$ queen
- Semantic vector relationships
- Example 2: Similar words: walk, walking, walked
- Close vector distances

Modern NLP Evolution

- N-grams → RNNs → LSTMs → Transformers
- From static embeddings to contextual embeddings
- Foundation for LLMs like GPT

Closing Summary

- Autocorrect, HMMs, POS, N-grams, Embeddings
- Classical NLP forms base for deep learning NLP

HIDDEN MARKOV MODEL

HIDDEN MARKOV MODEL

- Markov Chains
- Hidden Markov Models
- POS Tagging

Introduction to Sequential Models

- Language is sequential
- Speech is sequential
- POS tags follow patterns

Sequential Models

- Models designed to handle data where order matters.
- Used in language, speech, time-series, video, DNA sequences.

Why Sequential Models?

- Meaning depends on order: 'Dog bites man' $\neq$ 'Man bites dog'.
- Captures dependencies across time or word positions.
- Essential for understanding grammar, context, and meaning.

Examples of Sequential Data

- Words in a sentence.
- Audio / speech waves.
- Sensor or stock market time-series.
- Video frames.

Key Sequential Models in NLP

- Markov Chains – next state depends on current state.
- Hidden Markov Models – hidden states emit observations.
- N-gram Models – predict next word using N-1 previous words.
- RNN / LSTM / GRU – neural sequence models.
- Transformers – modern sequence models based on attention.

Why Order Matters?

- Sequential models preserve context.
- Previous elements influence current meaning.
- Used in POS tagging, speech recognition, machine translation.

Why Sequential Models?

- Predict next events
- Understand structure
- Model uncertainty

Markov Concept

- Future depends only on present
- Memoryless property

Markov Examples

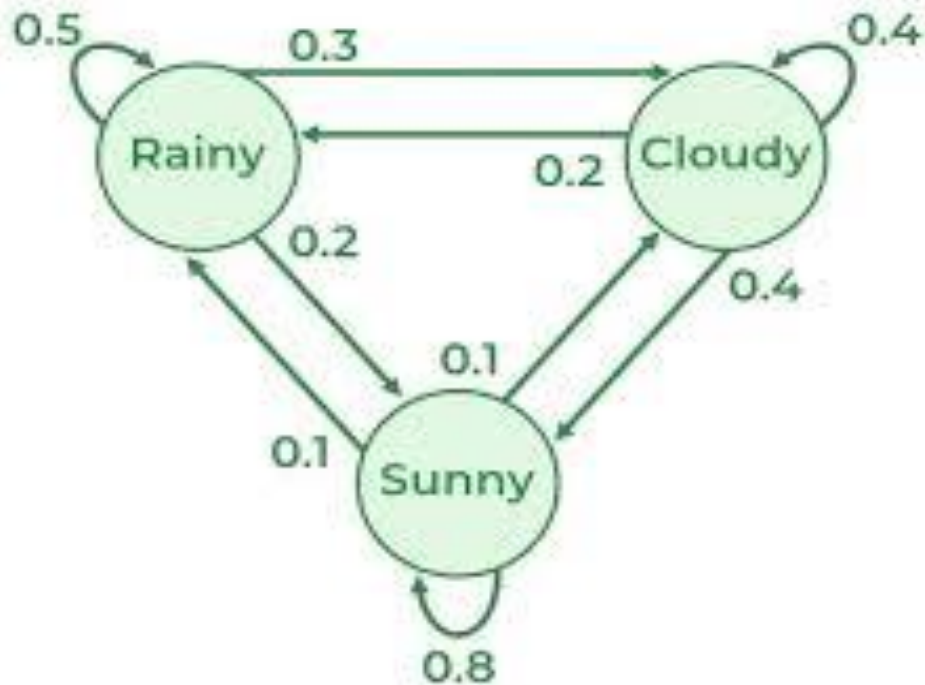
- Weather prediction
- Website navigation
- Machine states

States in Markov Chains

- Sunny/Rainy
- Hot/Cold
- Browsing/Checkout

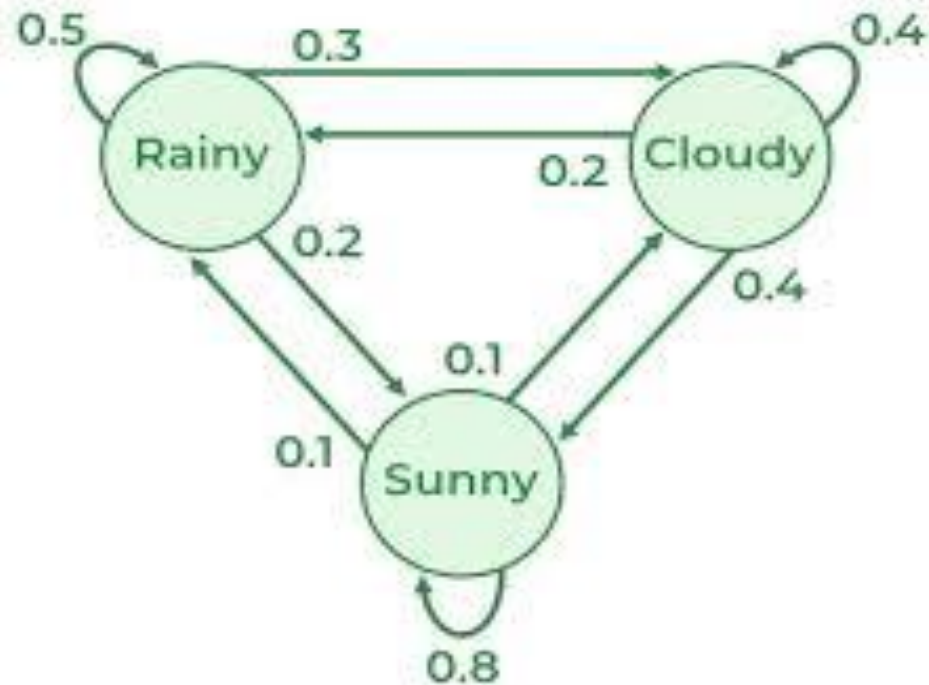
Transitions

- Move between states
- Each transition has probability



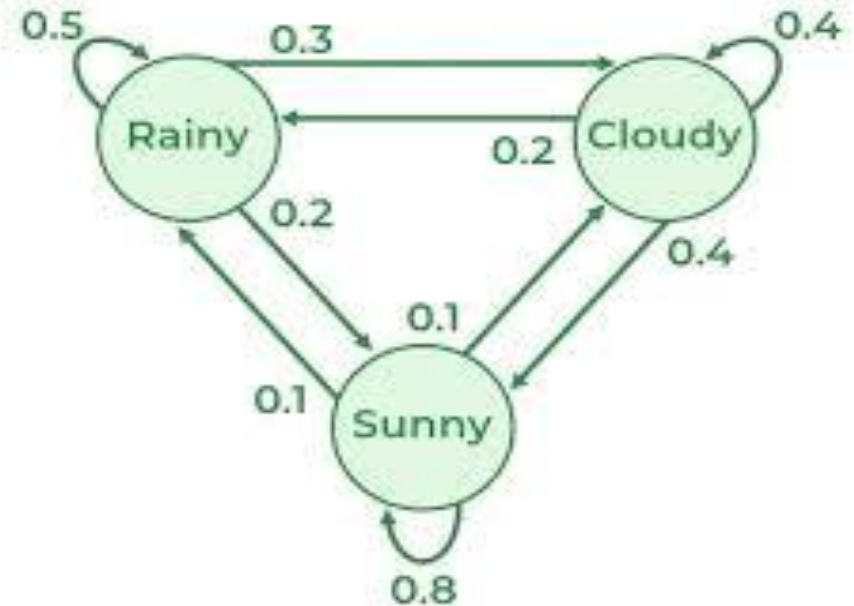
Transition Diagram

- Visual representation
- Arrows = transitions



Transition Matrix

- Rows = current state
- Columns = next state



Sequence Probabilities

- Compute path probability
- Multiply transitions

Sequence Example 1

- Sunny → Sunny → Rainy
- $0.6 \times 0.4 = 0.24$

Sequence Example 2

- Rainy $\rightarrow$ Sunny $\rightarrow$ Sunny
- $0.7 \times 0.6 = 0.42$

Why Markov Important?

- Basis of HMM
- Basis of N-grams

Limitations of Markov Chains

- Visible states only
- Cannot model hidden structure

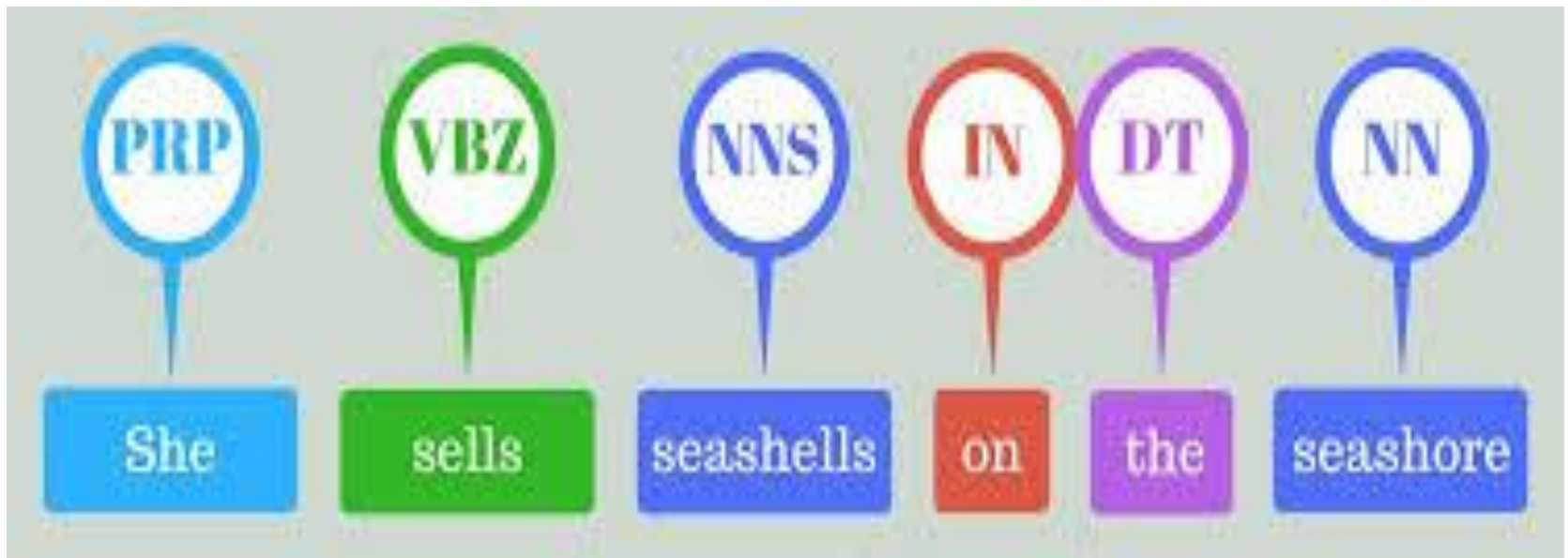
Introduction to HMM

- Hidden states + observed outputs



Why Hidden?

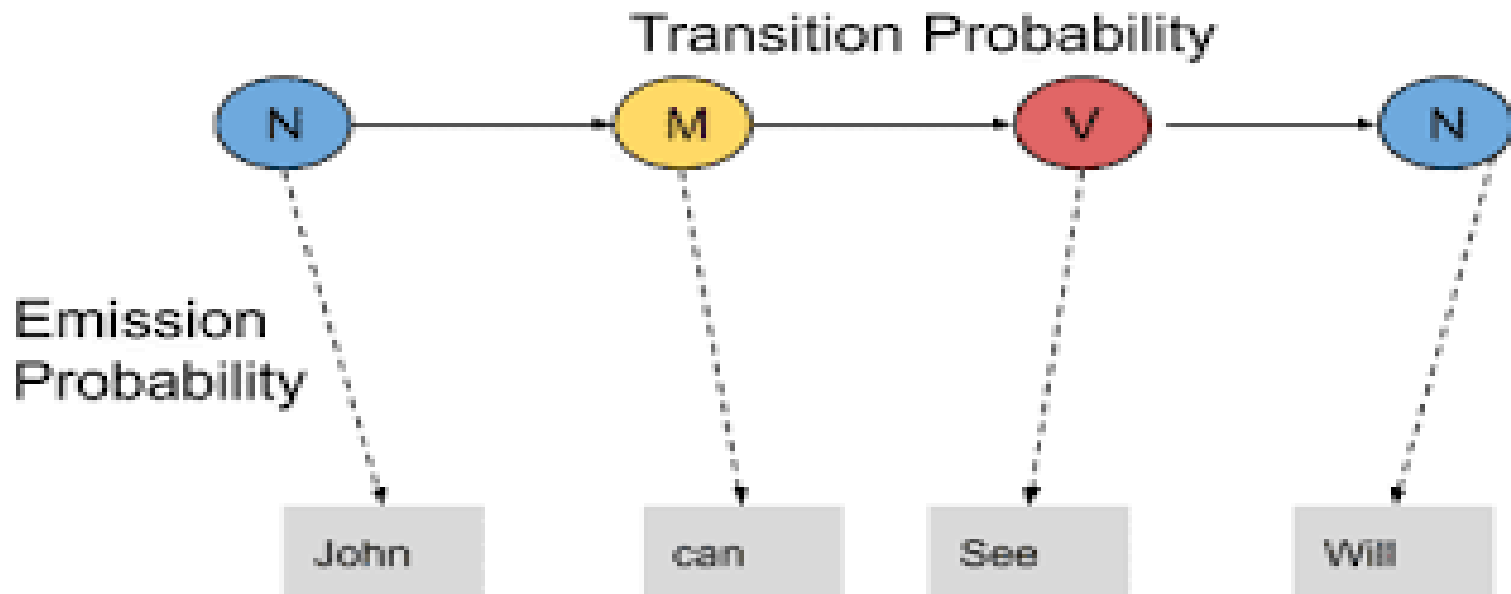
- Tags hidden
- Words visible



HMM Components

$$\pi_i = P(\text{state } S_i \text{ at time } t=1)$$

- Initial probabilities
- Transition probabilities
- Emission probabilities



Hidden States Example

- Noun
- Verb
- Adverb

Observed Example

- Words: time, flies, quickly

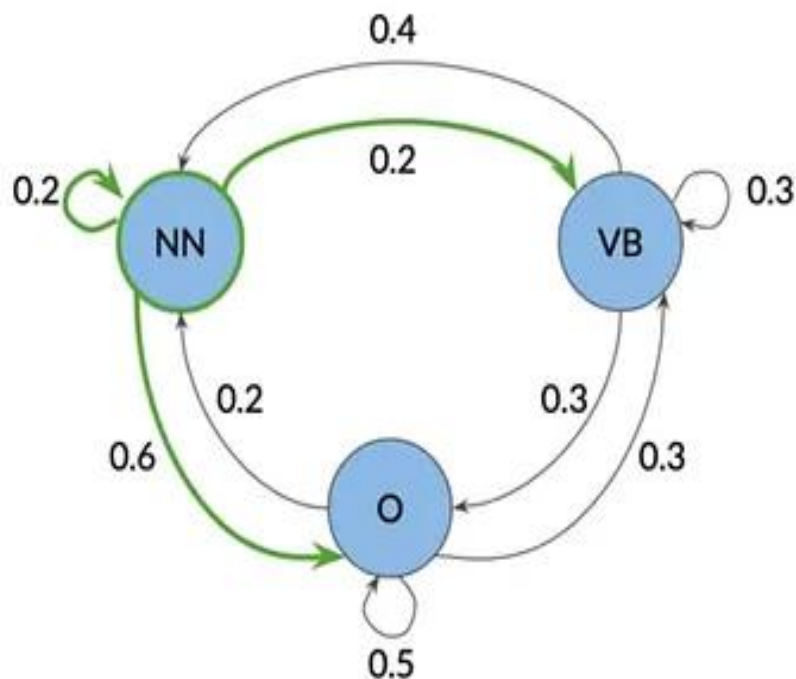
HMM Diagram

– Hidden layer $\rightarrow$ Observed layer

Transitions in HMM

- Between hidden states

The transition matrix



$A =$

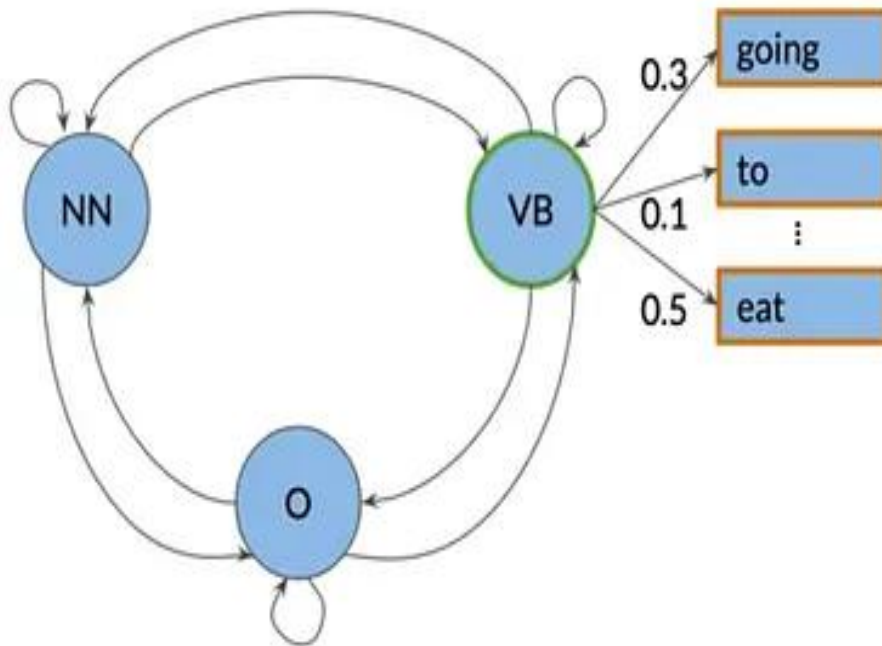
	NN	VB	O
NN (noun)	0.2	0.2	0.6
VB (verb)	0.4	0.3	0.3
O (other)	0.2	0.3	0.5

$$\sum_{j=1}^N a_{ij} = 1$$

Emissions in HMM

- From hidden state to word

Emission probabilities



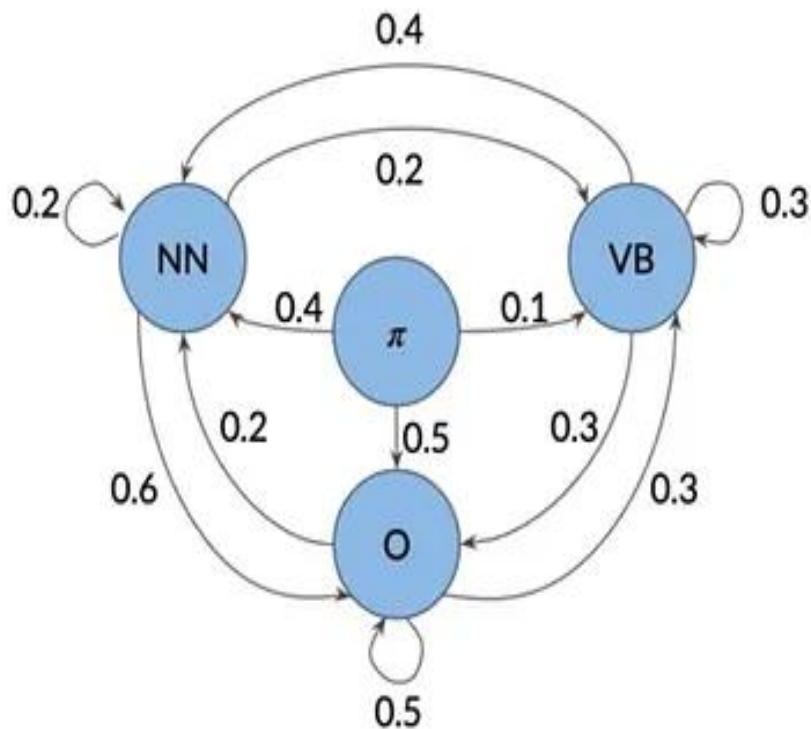
$B =$

	going	to	eat	...
NN (noun)	0.5	0.1	0.02	
VB (verb)	0.3	0.1	0.5	
O (other)	0.3	0.5	0.68	

Initial Probabilities

– Start-state distribution

Initial probabilities



$A =$

	NN	VB	O
π (initial)	0.4	0.1	0.5
NN (noun)	0.2	0.2	0.6
VB (verb)	0.4	0.3	0.3
O (other)	0.2	0.3	0.5

Toy HMM Example

- Sentence: time flies

Possible Tag Paths

– $N \rightarrow V$

– $V \rightarrow N$

Compute Path Probabilities

- Multiply initial $\times$ emissions $\times$ transitions

Choose Best Path

- Higher probability wins

POS Tagging Introduction

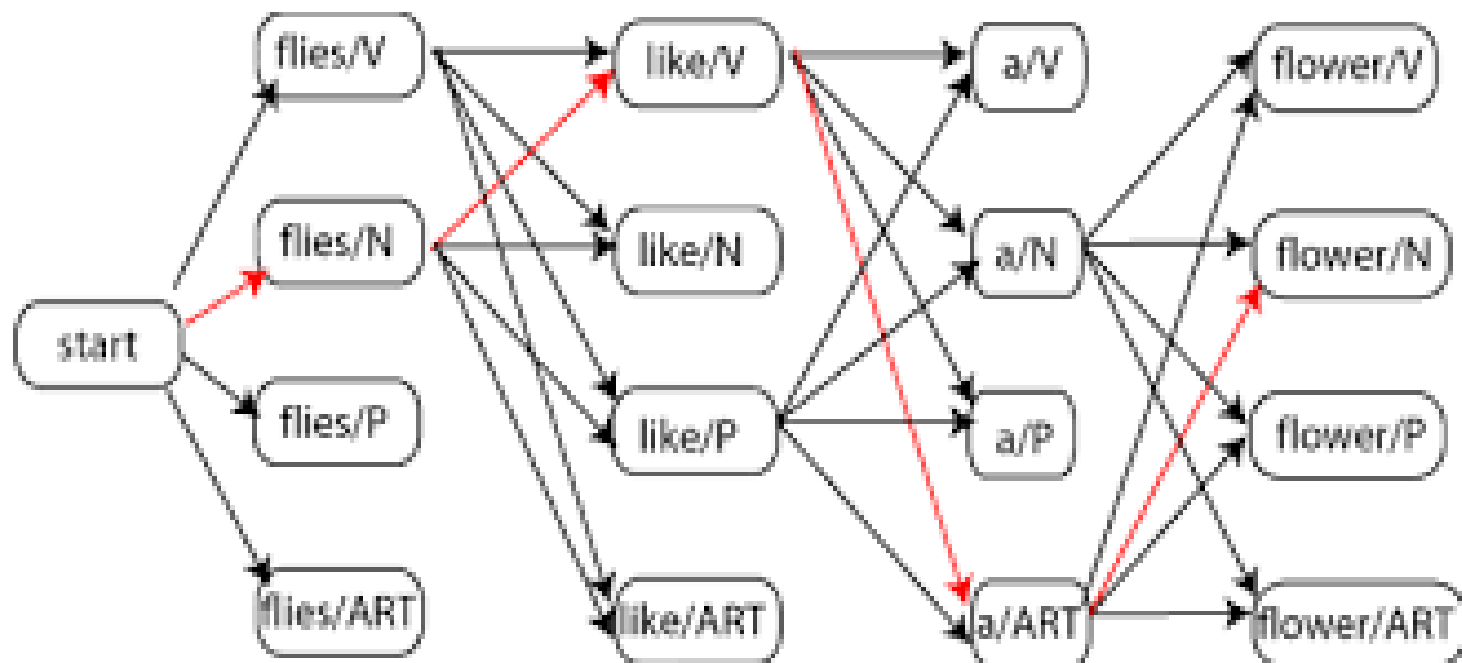
- Words = observed
- Tags = hidden

Why Use HMM for POS Tagging?

- Probabilistic inference
- Sequence-level reasoning

Viterbi Algorithm

- Dynamic programming
- Find best tag sequence



Viterbi Table Structure

- Rows = tags
- Columns = observed words
- The Viterbi table is a **dynamic programming grid** used to find the **most probable sequence of hidden states** (like POS tags) for a given sequence of observed words.

	time	flies	quickly

Noun	v11	v12	v13
Verb	v21	v22	v23
Adjective	v31	v32	v33

$$V[i,1] = \pi_i \times P(\text{word1} | \text{state}_i)$$

Best Path Storage

- Track maximum values

**max_all previous states[Viterbi(prev)×P(transition)×
P(emission)]**

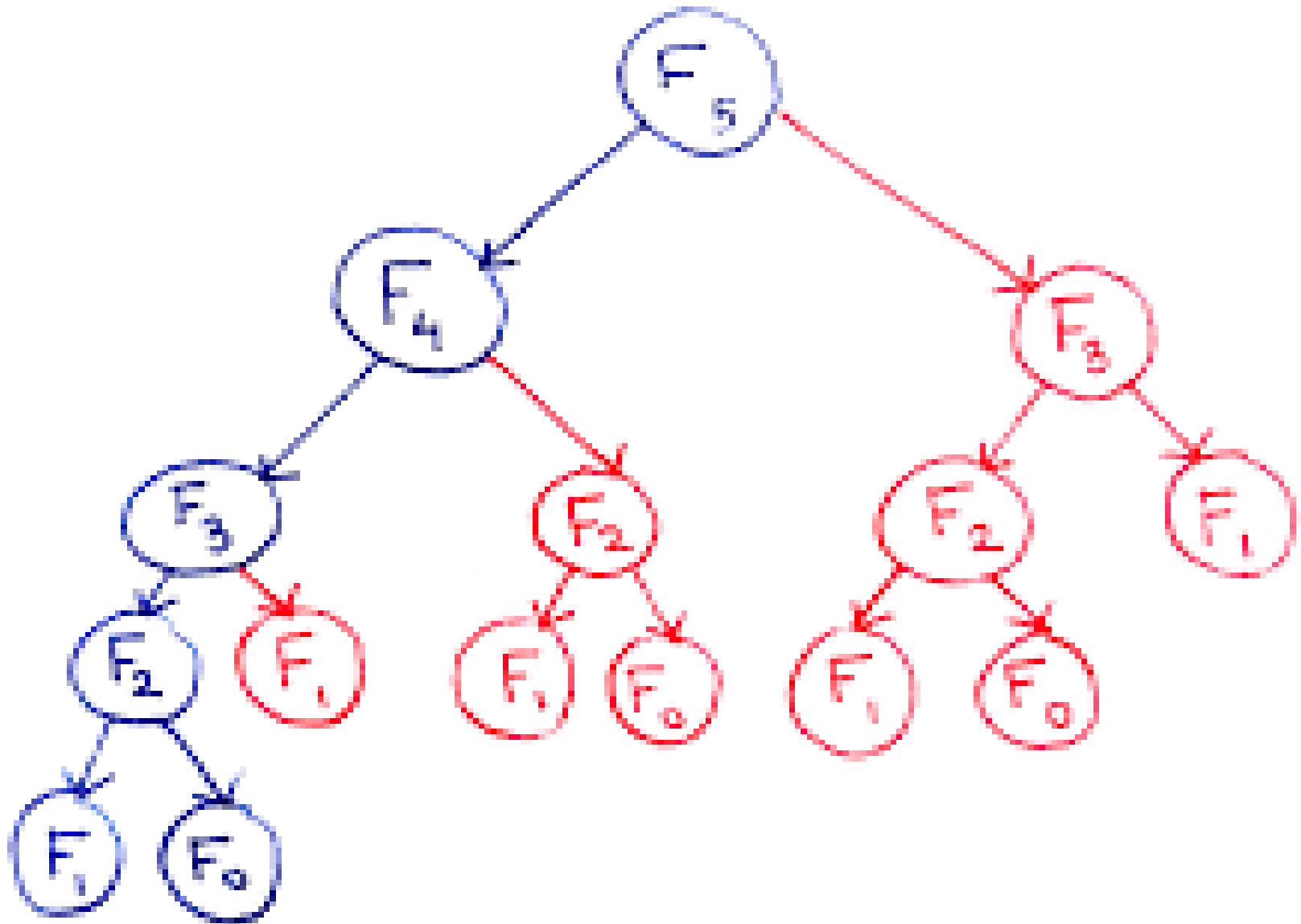
Strengths of HMM Tagging

- Simple
- Interpretable

Summary of Entire Model

– Markov → HMM → Viterbi → POS Tagging

Dynamic Programming



Dynamic Programming

Minimum Edit Distance

-	O	E	X	E	C	U	T	I	O	N
O	0	1	2	3	4	5	6	7	8	9
I	1	1	2	3	4	5	6	5	7	8
N	2	2	2	3	4	5	6	7	7	7
T	3	3	3	3	4	5	5	6	7	8
E	4	3	4	3	4	5	6	6	7	8
N	5	4	4	4	4	5	6	7	7	7
T	6	5	5	5	5	5	5	6	7	8
I	7	6	6	6	6	6	6	5	6	7
O	8	7	7	7	7	7	7	6	5	6
N	9	8	8	8	8	8	8	7	6	5

Logic ?

Minimum Edit Distance

```
pip install python-Levenshtein
```

```
import Levenshtein
```

```
Levenshtein.distance("kitten", "sitting")
```




How similar are two strings?

- Spell correction

- The user typed “graffe”

Which is closest?

- graf
 - graft
 - grail
 - giraffe

- Also for Machine Translation, Information Extraction, Speech Recognition



Edit Distance

- The minimum edit distance between two strings
- Is the minimum number of editing operations
 - Insertion
 - Deletion
 - Substitution
- Needed to transform one into the other



Minimum Edit Distance

- Two strings and their **alignment**:

I	N	T	E	*	N	T	I	O	N
*	E	X	E	C	U	T	I	O	N



Minimum Edit Distance

I N T E * N T I O N

| | | | | | | | | |

* E X E C U T I O N

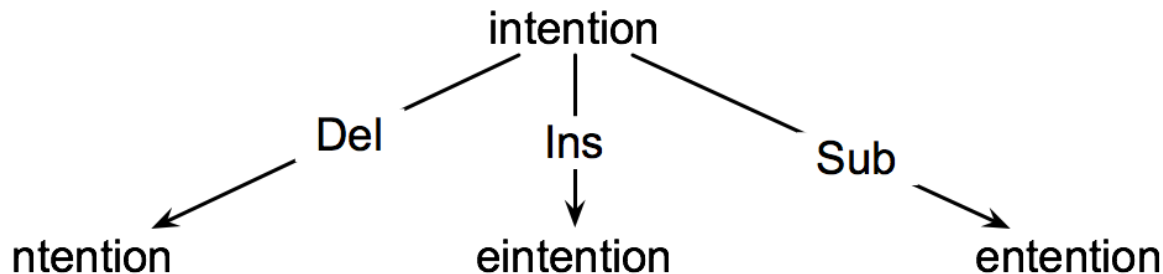
d s s i s

- If each operation has cost of 1
 - Distance between these is 5
- If substitutions cost 2 (Levenshtein)
 - Distance between them is 8



How to find the Min Edit Distance?

- Searching for a path (sequence of edits) from the start string to the final string:
 - **Initial state:** the word we're transforming
 - **Operators:** insert, delete, substitute
 - **Goal state:** the word we're trying to get to
 - **Path cost:** what we want to minimize: the number of edits





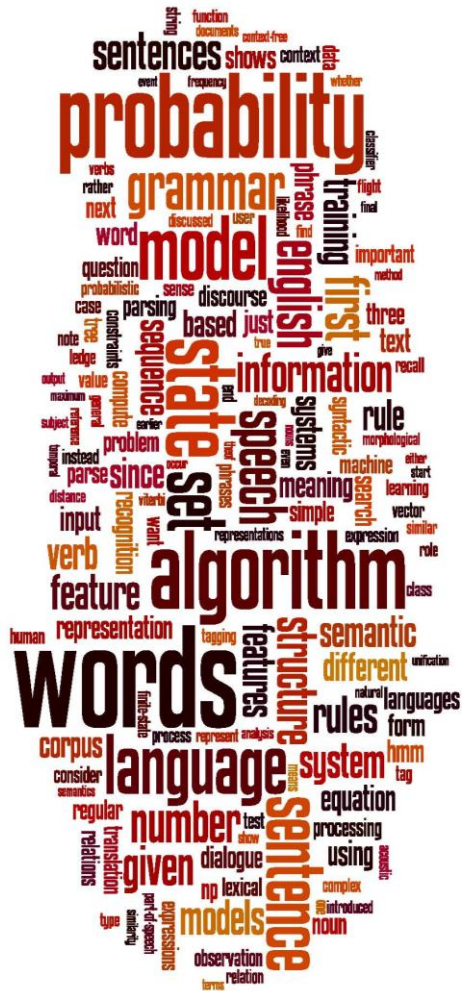
Minimum Edit as Search

- But the space of all edit sequences is huge!
 - We can't afford to navigate naïvely
 - Lots of distinct paths wind up at the same state.
 - We don't have to keep track of all of them
 - Just the shortest path to each of those revisited states.



Defining Min Edit Distance

- For two strings
 - X of length n
 - Y of length m
- We define $D(i,j)$
 - the edit distance between $X[1..i]$ and $Y[1..j]$
 - i.e., the first i characters of X and the first j characters of Y
 - The edit distance between X and Y is thus $D(n,m)$



Minimum Edit Distance

Computing Minimum Edit Distance



The Edit Distance Table

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N



The Edit Distance Table

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

 A red arrow points from the top right towards the cell containing 'I' in the row labeled 'I' and column '1' (the cell with value 1).



Edit Distance

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N



The Edit Distance Table

N	9	8	9	10	11	12	11	10	9	8
O	8	7	8	9	10	11	10	9	8	9
I	7	6	7	8	9	10	9	8	9	10
T	6	5	6	7	8	9	8	9	10	11
N	5	4	5	6	7	8	9	10	11	10
E	4	3	4	5	6	7	8	9	10	9
T	3	4	5	6	7	8	7	8	9	8
N	2	3	4	5	6	7	8	7	8	7
I	1	2	3	4	5	6	7	6	7	8
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N



Dynamic Programming for Minimum Edit Distance

- **Dynamic programming:** A tabular computation of $D(n,m)$
- Solving problems by combining solutions to subproblems.
- Bottom-up
 - We compute $D(i,j)$ for small i,j
 - And compute larger $D(i,j)$ based on previously computed smaller values
 - i.e., compute $D(i,j)$ for all i ($0 < i < n$) and j ($0 < j < m$)



Defining Min Edit Distance (Levenshtein)

- Initialization

$$D(i, 0) = i$$

$$D(0, j) = j$$

- Recurrence Relation:

For each $i = 1 \dots M$

For each $j = 1 \dots N$

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 \\ D(i, j-1) + 1 \\ D(i-1, j-1) + \begin{cases} 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} \end{cases}$$

$$X(i) = Y(j)$$

- Termination:

Sentiment Analysis

Introduction, Workflow, Models &
Examples

What is Sentiment Analysis?

- • NLP task to detect opinions/emotions
- • Classifies text as Positive, Negative, Neutral
- • Used in reviews, social media, feedback analysis

Why Sentiment Analysis?

- • Understand customer satisfaction
- • Track brand reputation
- • Analyze user-generated content
- • Automate feedback processing

Steps in Sentiment Analysis

- 1. Data Collection
- 2. Text Preprocessing
- 3. Feature Extraction (TF-IDF, embeddings)
- 4. Model Training (LR, NB, SVM, BERT)
- 5. Sentiment Prediction

Text Preprocessing

- • Tokenization
- • Lowercasing
- • Stopword removal
- • Lemmatization/Stemming
- • Removing punctuation

Feature Extraction

- • Bag of Words
- • TF-IDF
- • Word2Vec/GloVe embeddings
- • Transformer embeddings (BERT)

Machine Learning Models

- • Logistic Regression
- • Naïve Bayes
- • Support Vector Machines
- • LSTM, GRU
- • BERT, RoBERTa

Python Code Example

- `from sklearn.feature_extraction.text import TfidfVectorizer`
- `from sklearn.linear_model import LogisticRegression`
- `vectorizer = TfidfVectorizer()`
- `X = vectorizer.fit_transform(texts)`
- `model = LogisticRegression()`
- `model.fit(X, labels)`

Applications

- • Product review mining
- • Social media monitoring
- • Customer feedback analysis
- • Market research
- • Emotion tracking

Sample Program

- `import pandas as pd`
- `# Load the CSV file`
- `df = pd.read_csv`
`("mini_movie_reviews_dataset.csv")`

- # Check dataset size
- `print("Total rows:", len(df))`
- `for i, row in df.iterrows():`
 `print(i, "---", row["text"], "====>",`
 `row["sentiment"])`

- #preprocessing – lowercase
- `df["text"] = df["text"].str.lower()`

- `#preprocessing - removing punctuations`
- `import string`
- `df["text"] = df["text"].str.replace
(f"[{string.punctuation}]", "", regex=True)`

- #preprocessing – tokenization
- import nltkfrom nltk.tokenize
- import word_tokenize
- nltk.download("punkt_tab")
- df["text"] = df["text"].apply(lambda x:
word_tokenize(x))

- `#preprocessing - stop words removal`
- `from nltk.corpus import stopwords`
- `nltk.download("stopwords")`
- `stop_words = set(stopwords.words("english"))`
- `df["text"] = df["text"].apply(lambda tokens:
[word for word in tokens if word not in
stop_words])`

- #preprocessing – stemming
- from nltk.stem import PorterStemmer
- stemmer = PorterStemmer()
- df["text"] = df["text"].apply(lambda tokens: [stemmer.stem(word) for word in tokens])

- `X_text = df['text'].apply(lambda toks: ".join(toks)).astype(str)`

- # TF-IDF Extract
- from sklearn.feature_extraction.text import TfidfVectorizer
- vectorizer =
TfidfVectorizer(ngram_range=(1,1))
- X_tfidf = vectorizer.fit_transform(X_text)

- `print("Number of features:",
len(vectorizer.get_feature_names_out()))`
- `print("Name of the features:",
vectorizer.get_feature_names_out())`

- # prepare the labels for training
- `y = df["sentiment"].map({"positive": 1, "negative": 0})`

- # Prepare the Logistic Regression Model
- from sklearn.model_selection import train_test_split
- X_train, X_test, y_train, y_test = train_test_split(X_tfidf, y, test_size=0.2, random_state=42)

- `from sklearn.linear_model import LogisticRegression`
- `clf = LogisticRegression(max_iter=1000, random_state=42)`
- `clf.fit(X_train, y_train)`
- `y_pred = clf.predict(X_test)`

- `from sklearn.metrics import accuracy_score, confusion_matrix`
- `print("Accuracy:", accuracy_score(y_test, y_pred))`
- `print("\nConfusion matrix:\n", confusion_matrix(y_test, y_pred))`

Try with some other datasets

Unit-5

INTRODUCTION

Unit-5

Natural Language Processing with Sequence and Attention Models : Introduction to Sequence Models, Recurrent Neural Networks and their limitations, Long Short-Term Memory (LSTM), Applications, POS Tagging, Named Entity Recognition, Neural Machine Translation, Shortcomings of a Traditional seq2seq Model, Introduction to Attention Mechanism.

Prerequisites – Neural Networks

- Neuron: weighted sum + activation
- Weight: importance of input
- Bias: shifting the activation
- Loss function: error measurement

Prerequisites – Text as Data

- Text is treated as a sequence of words
- Order of words matters
- Each word represented as a vector

What is Sequential Data?

- Data where order is important
- Output depends on previous inputs

Examples:

- Sentence
- Speech
- Time-series

Sequential Data

$\text{Word}_1 \rightarrow \text{Word}_2 \rightarrow \text{Word}_3 \rightarrow \text{Word}_4$

Past information influences future prediction.

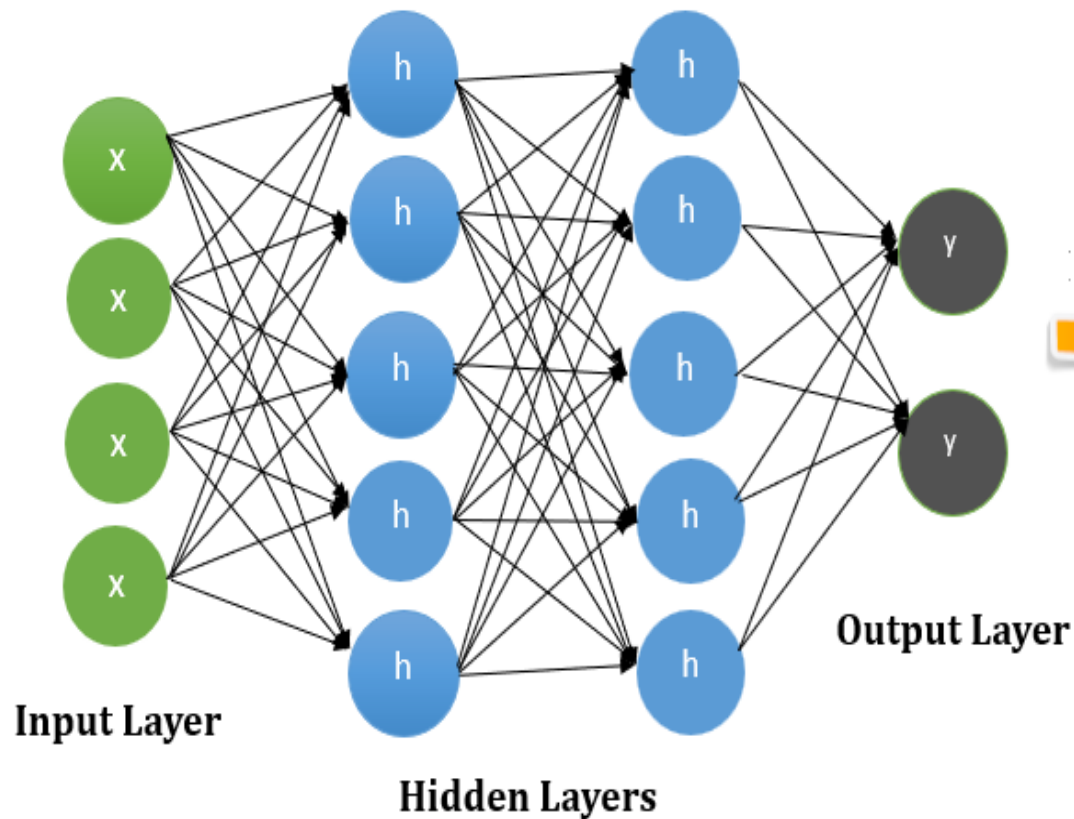
Why Sequence Models?

- Traditional NN has no memory
- Cannot handle variable-length input
- Sequence models maintain context

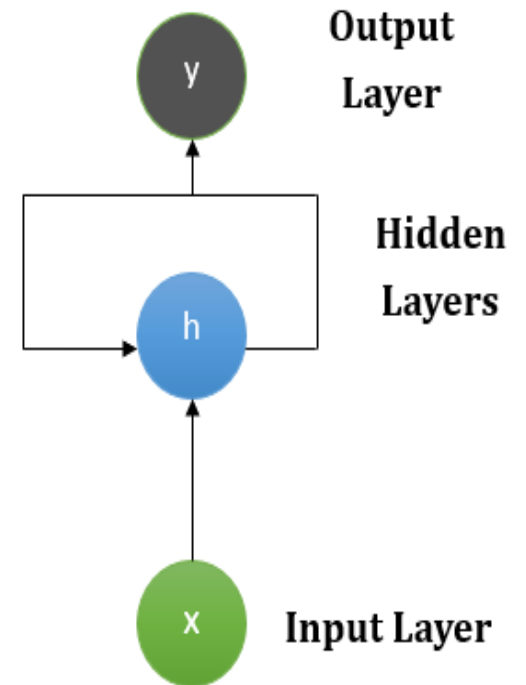
Recurrent Neural Network (RNN)

Definition:

- A neural network designed for sequential data
- Uses hidden state as memory



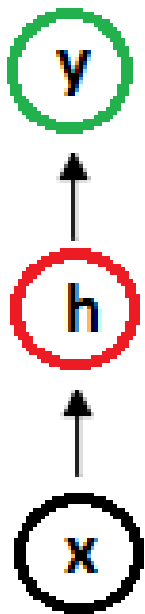
Feedforward Neural Network (FNN)



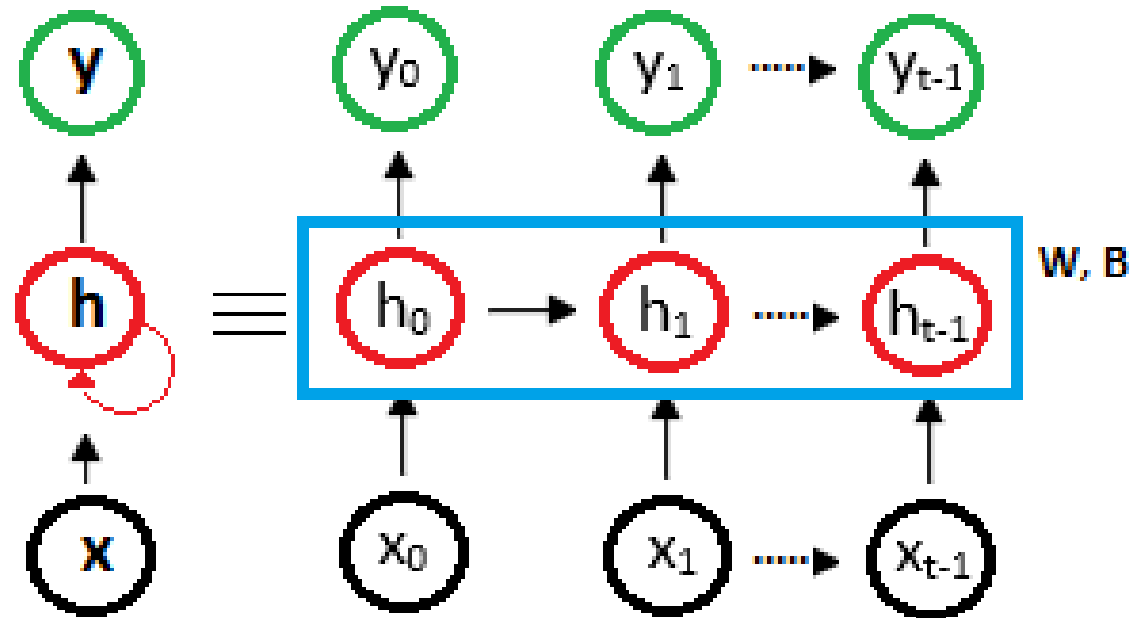
Recurrent Neural Network (RNN)

RNN Architecture

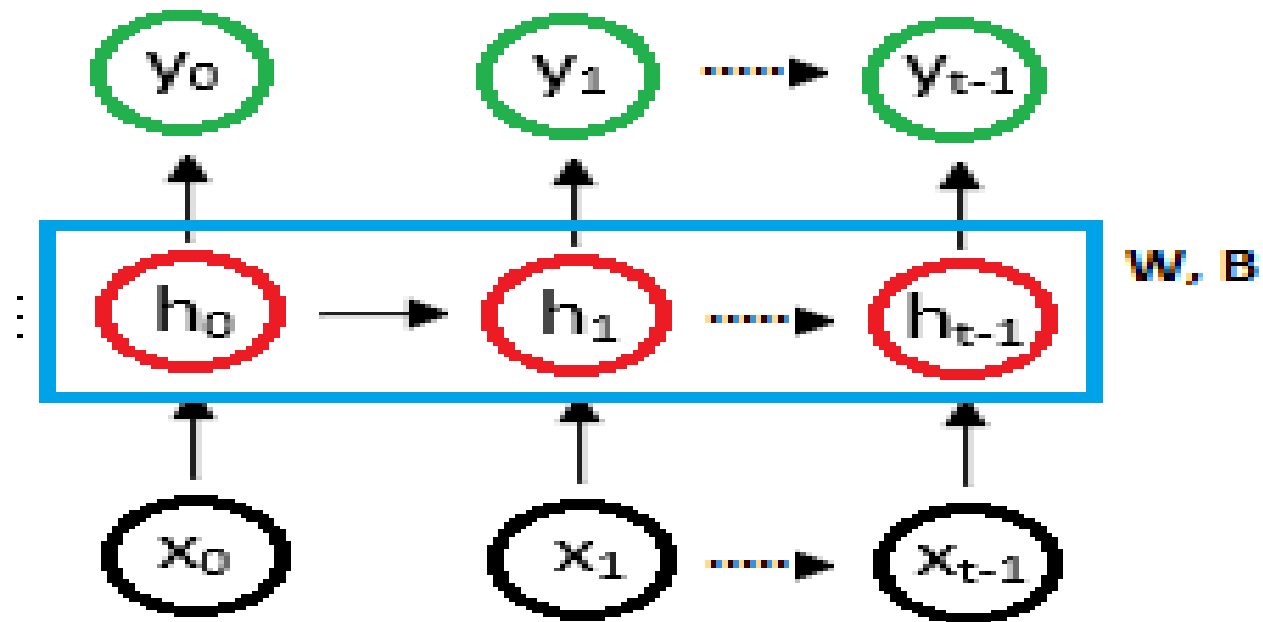
ANN



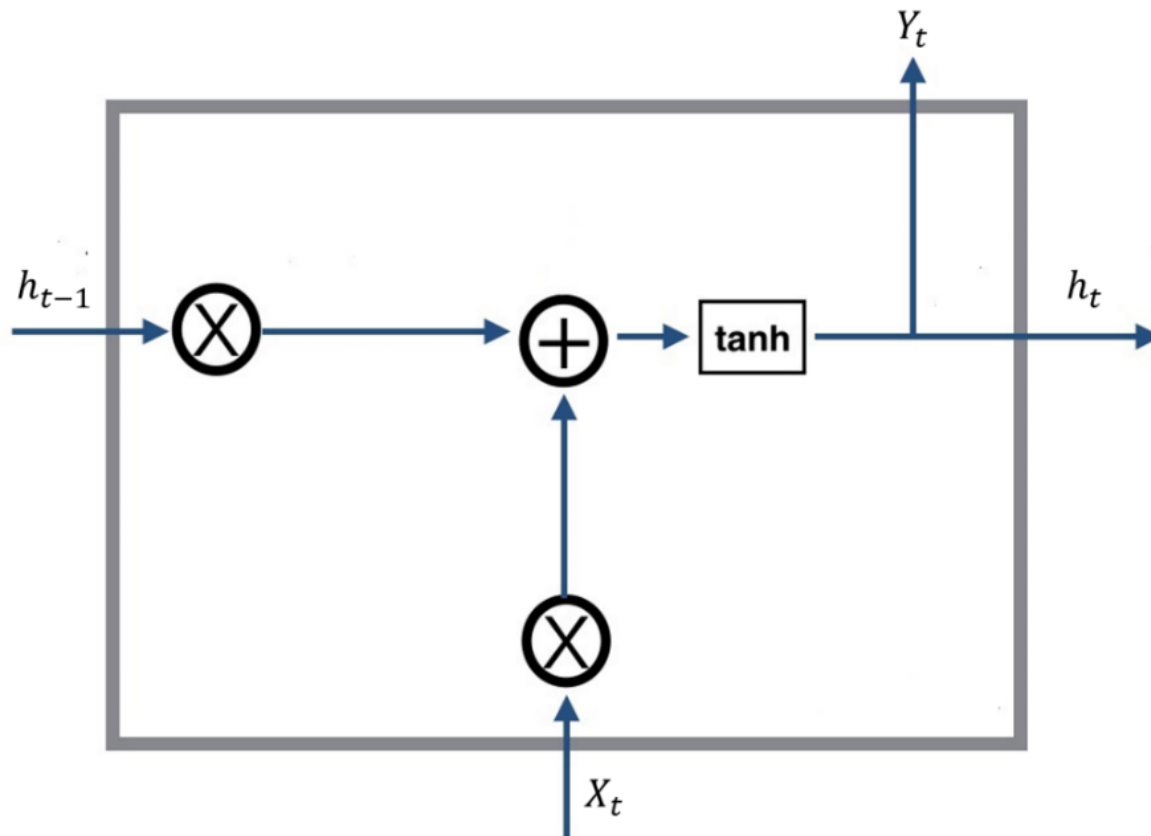
RNN Architecture



RNN Cell vs RNN Layer



Weights and Bias



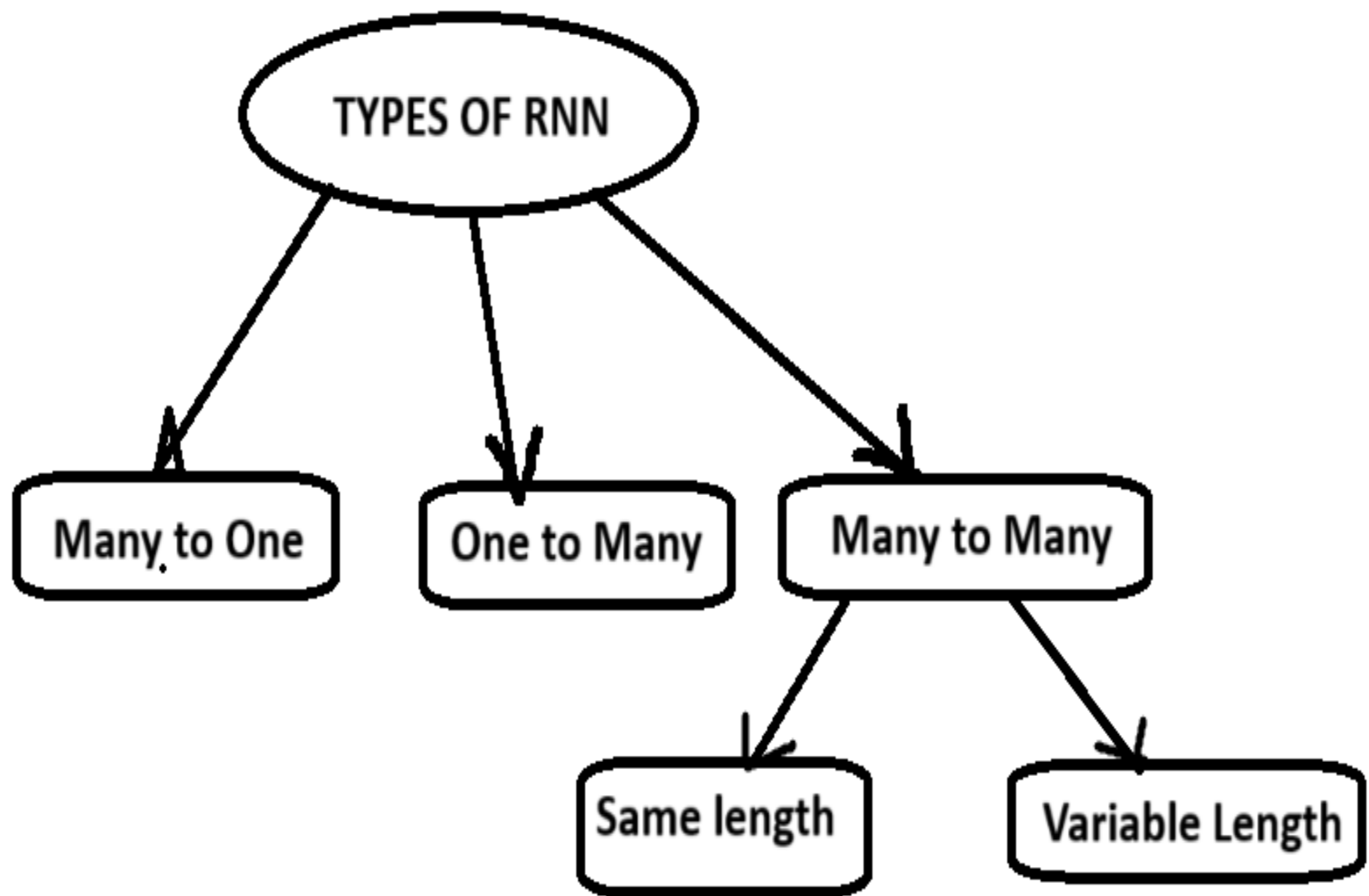
How RNN Works

- Input at time t
- Combine with previous hidden state
- Output + new hidden state

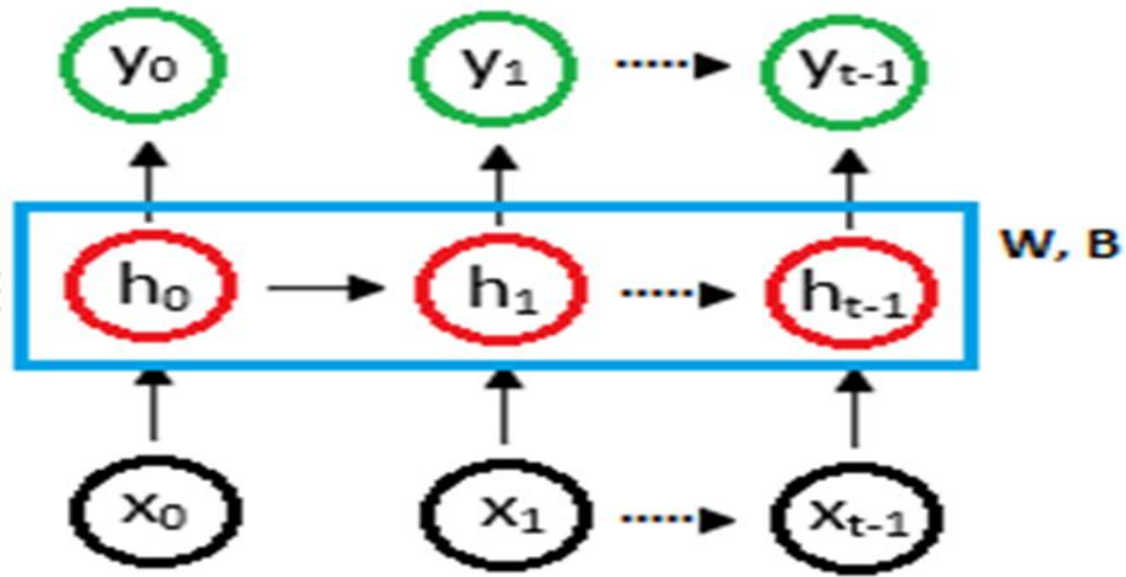
Types of RNN

(Based on input / output mapping)

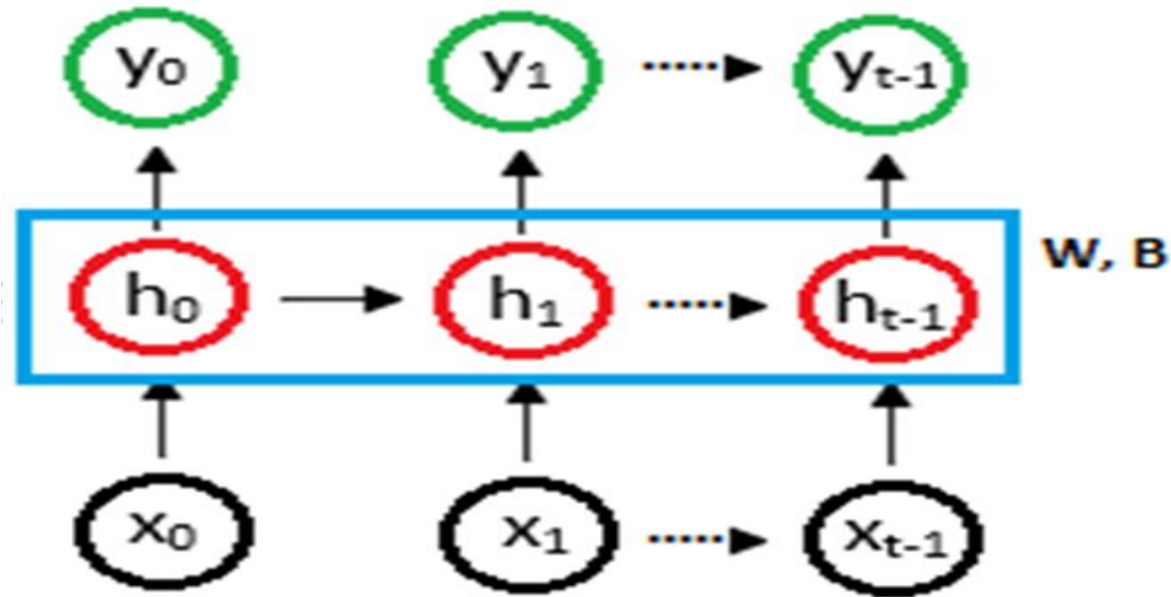
- One-to-One RNN
- One-to-Many RNN (Vector-to-Sequence)
- Many-to-One RNN
- Many-to-Many RNN (Same Length)
- Many-to-Many RNN (Sequence-to-Sequence)



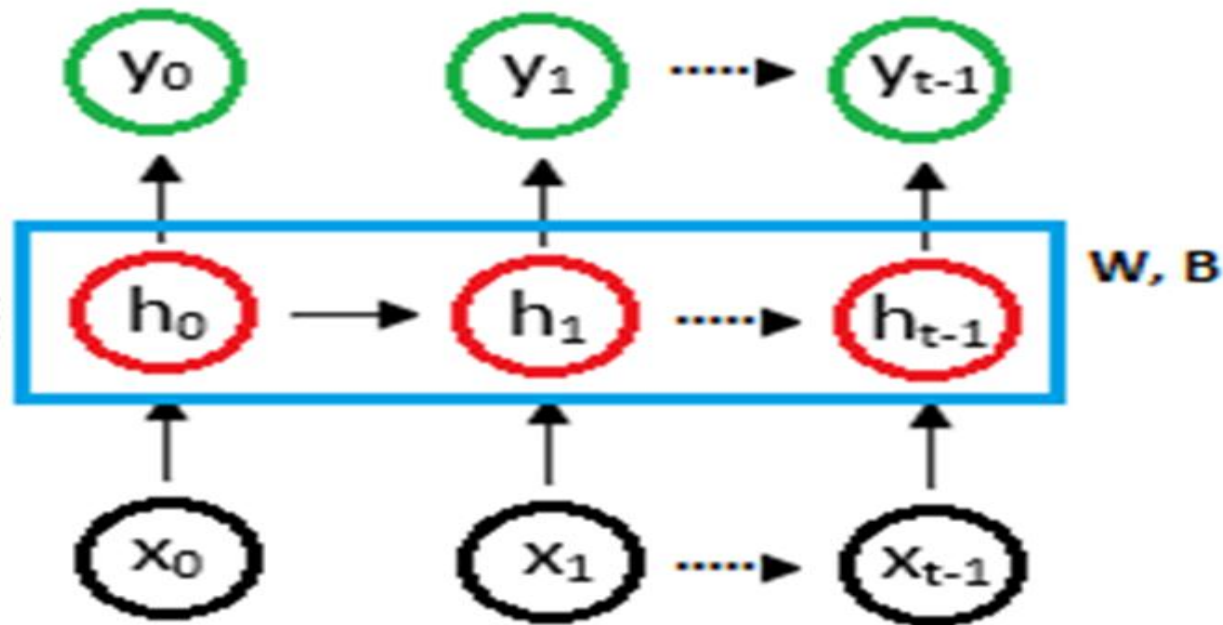
One-to-One RNN



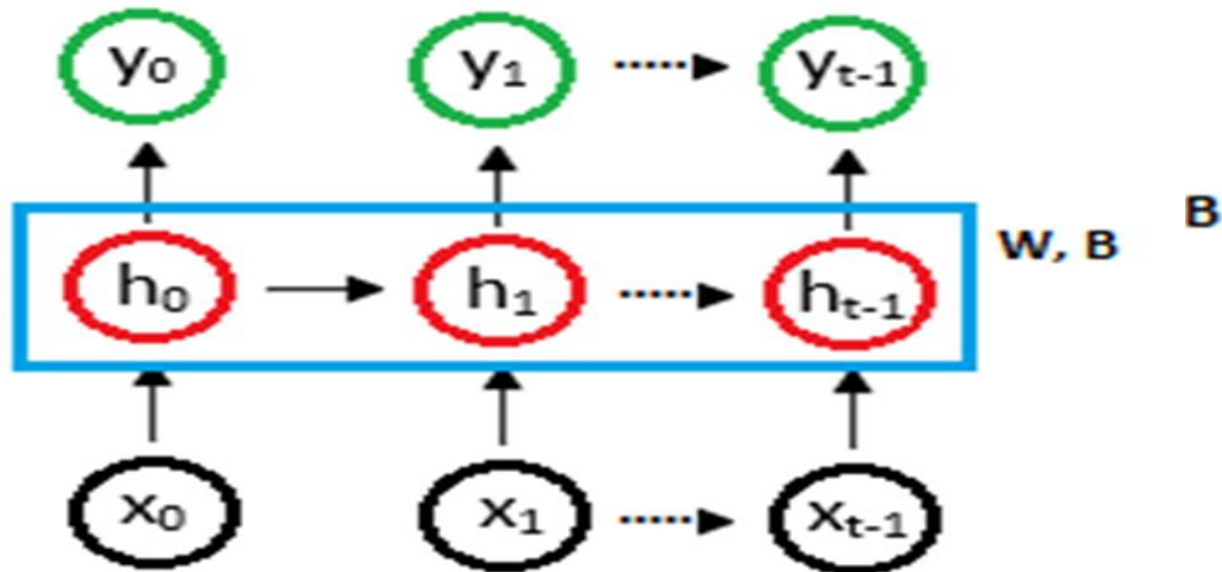
One-to-Many RNN (Vector-to-Sequence)



Many-to-One RNN



Many-to-Many RNN



Types of RNN

(Based on Architecture)

- Vanilla RNN
- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit (GRU)
- Bidirectional RNN (Bi-RNN)
- Deep (Stacked) RNN

Limitations of RNN

- Vanishing gradient problem
- Exploding gradient problem
- Cannot capture long-term dependencies

Vanishing Vs Exploding Gradient

Vanishing gradient is a problem where gradients become extremely small during backpropagation, preventing earlier layers from learning.

$$\text{Vanishing gradient} \quad \left\| \frac{\partial h_i}{\partial h_{i-1}} \right\|_2 < 1$$

$$\text{Exploding gradient} \quad \left\| \frac{\partial h_i}{\partial h_{i-1}} \right\|_2 > 1$$

- Long sequence $\rightarrow$ gradients shrink
- Earlier words lose importance

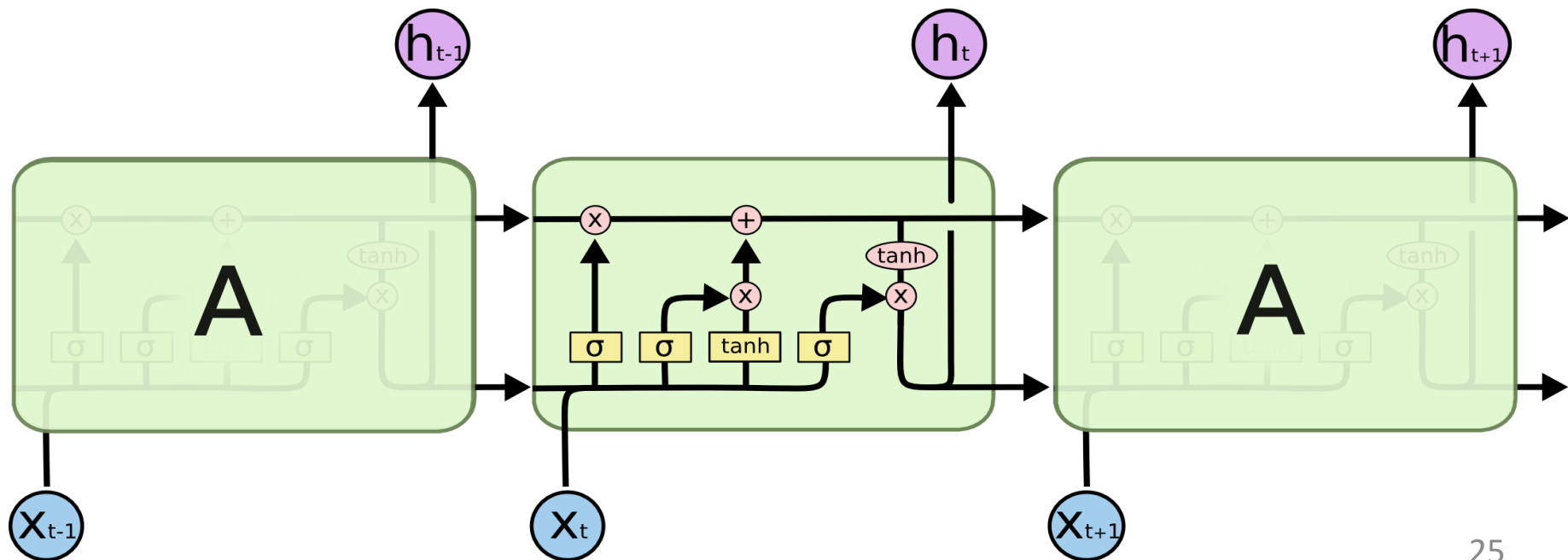
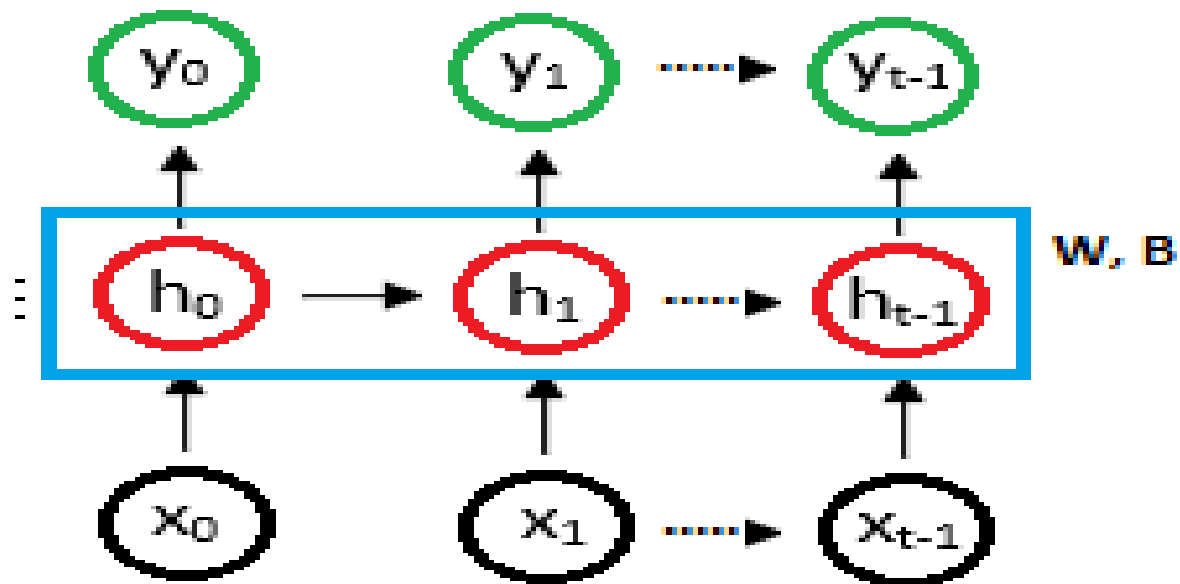
Why LSTM?

- Designed to overcome RNN limitations
- Maintains long-term memory
- Uses gates

Long Short-Term Memory (LSTM)

Definition:

- Special RNN with memory cell
- Controls information flow



Forget Gate

- Decides what information to remove
- Removes irrelevant past information

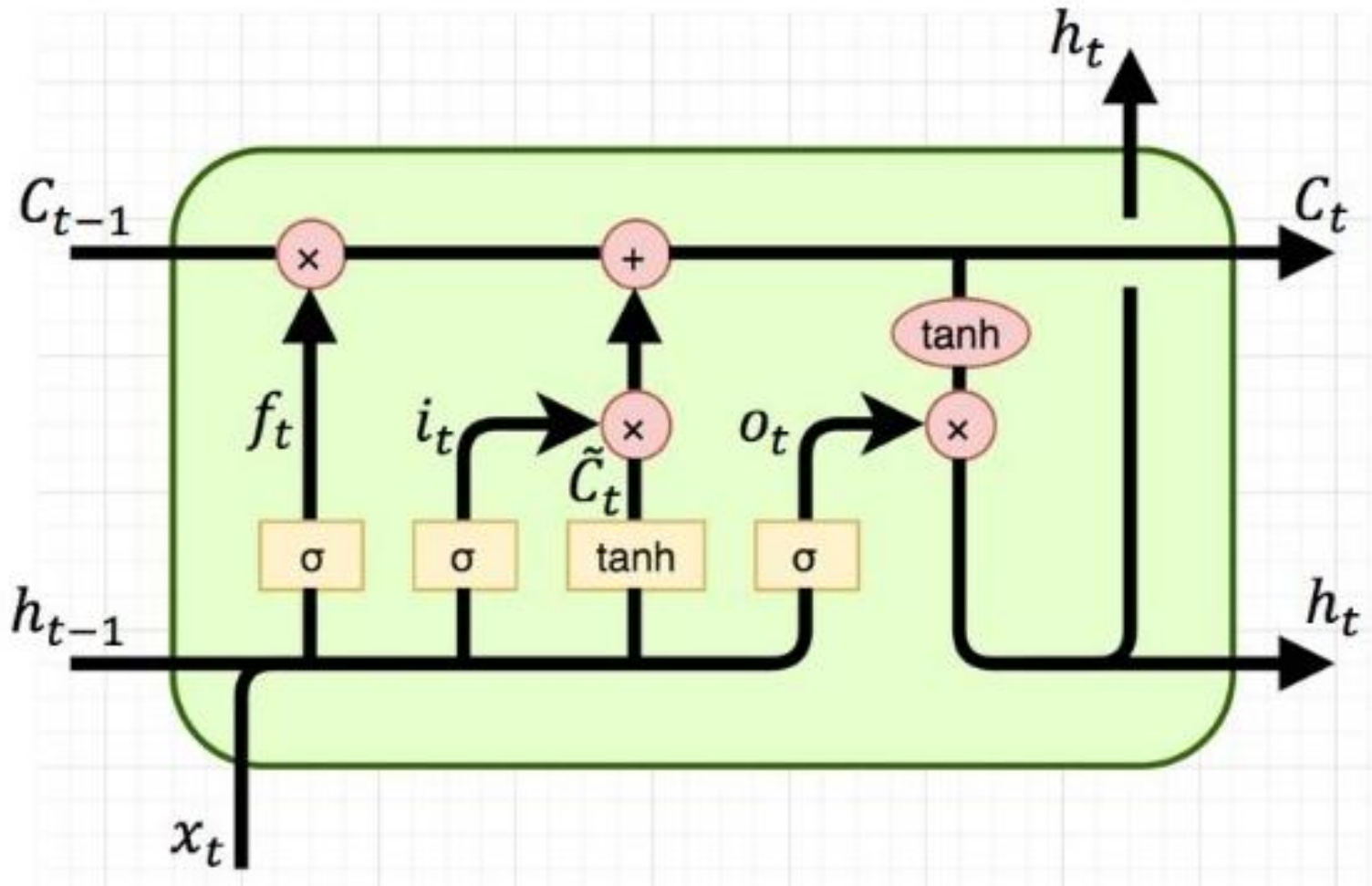
Input Gate

- Decides what new information to store
- Updates memory cell

Output Gate

- Decides what information to output
- Produces hidden state

LSTM Cell



Advantages of LSTM

- Learns long-term dependencies
- Avoids vanishing gradient
- Better performance on text

Applications of RNN & LSTM

- POS Tagging
- Named Entity Recognition
- Machine Translation
- Speech Recognition

POS Tagging

Definition:

- Assign grammatical labels to words

Example:

She/PRON likes/VERB coffee/NOUN

Named Entity Recognition (NER)

Definition:

- Identifies named entities in text

Exam Keywords:

PERSON, LOCATION, ORGANIZATION

Neural Machine Translation (NMT)

- Translates one language to another
- Uses Seq2Seq architecture

Sequence-to-Sequence Model

- Encoder reads input
- Decoder generates output
- Uses RNN/LSTM

Seq2Seq Architecture

Input Sentence → Encoder → Context Vector
→ Decoder → Output Sentence

Problems with Traditional Seq2Seq

- Fixed-length context vector
- Information loss
- Poor long sentence translation

Why Attention Mechanism?

- Removes fixed-length bottleneck
- Focuses on relevant input words

Attention Mechanism

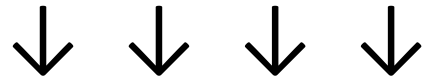
- Decoder step → looks at all encoder states
- More focus on important words

Attention – Definition

- Technique that allows model to focus on relevant parts of input
- Different focus at each decoding step

Attention Working – Diagram

Encoder States: h_1 h_2 h_3 h_4



Attention Weights



Context Vector

Advantages of Attention

- Better translation quality
- Handles long sentences
- Improves interpretability

Overall Summary

- • RNN introduced sequence learning
- • LSTM improved memory
- • Seq2Seq enabled translation
- • Attention solved bottleneck problem

Seq2Seq to Transformers and LLMs

Evolution of Neural Sequence Models

Why Sequence Models in NLP

- Language is sequential in nature
- Meaning depends on word order
- Input and output lengths can differ
- Classical models fail to capture long dependencies

Seq2Seq Model – Definition

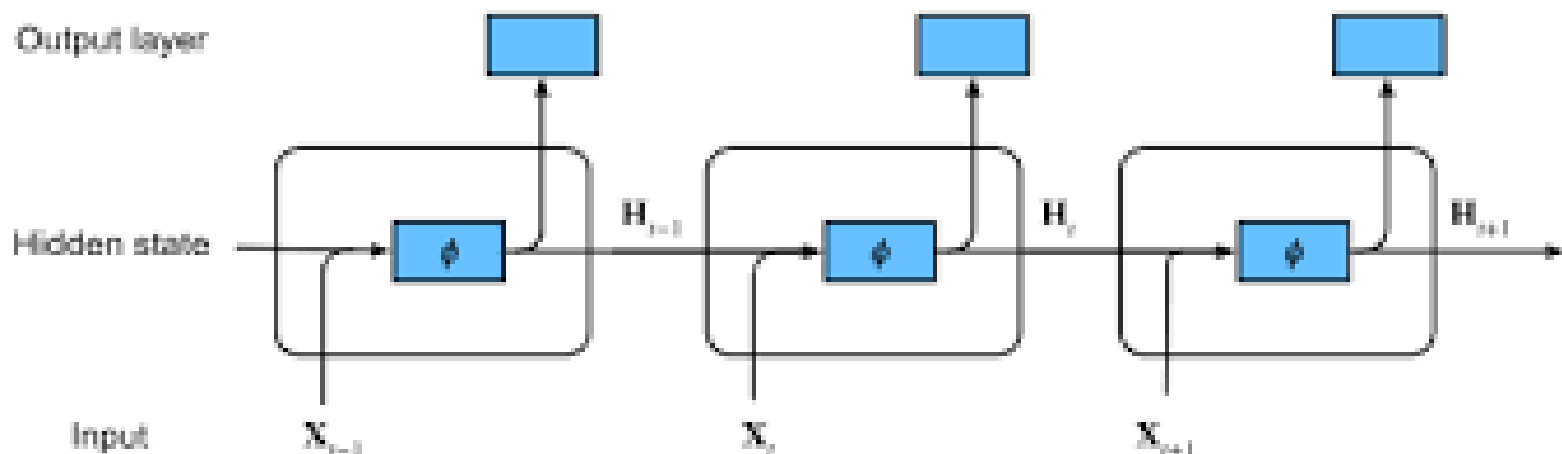
- Sequence-to-Sequence maps input sequence to output sequence
- Uses encoder–decoder architecture
- Originally implemented using RNN / LSTM
- Popular in early machine translation systems

Seq2Seq Architecture – Explanation

- Encoder reads input tokens sequentially
- Encoder compresses input into a context vector
- Decoder generates output sequence token by token
- Same model handles variable-length input and output

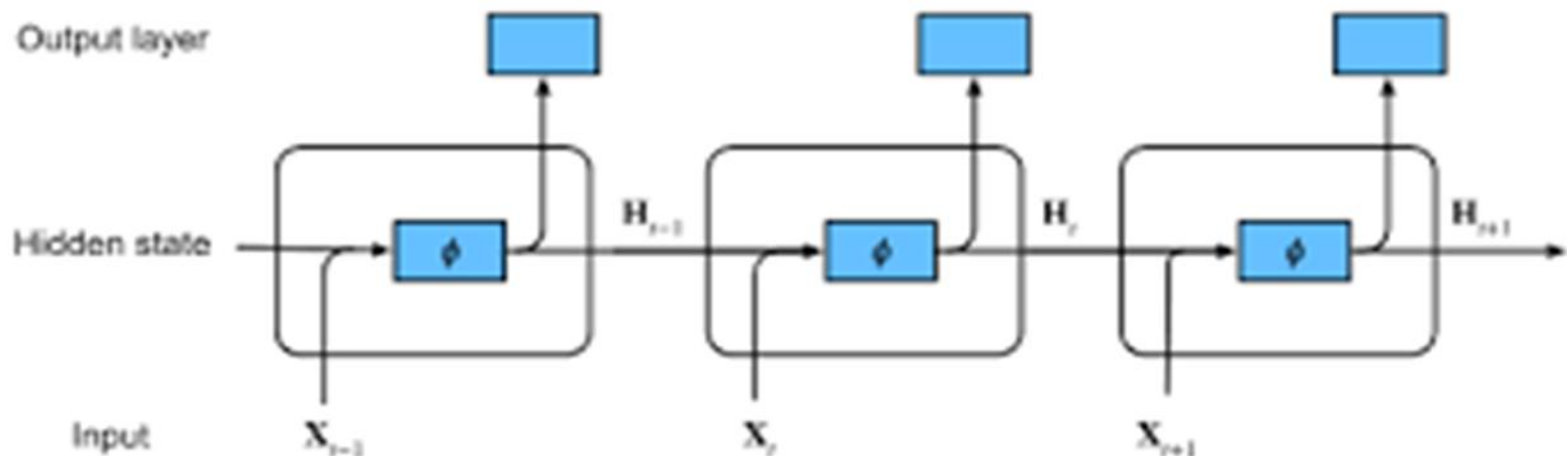
Encoder – Detailed View

- Processes input one word at a time
- Maintains hidden state
- Final hidden state summarizes entire input
- Information is stored implicitly in hidden state



Decoder – Detailed View

- Initialized using encoder final state
- Generates output autoregressively
- Uses previous output to predict next token
- Stops generation at <EOS> token



Fixed-Length Context Vector

- Final encoder hidden state is called context vector
- Vector size is fixed irrespective of sentence length
- Acts as information bottleneck
- Decoder relies fully on this vector

Why Fixed-Length Vector is a Problem

- Short and long sentences mapped to same vector size
- Important information may be lost
- Early words get forgotten
- Performance drops for long sentences

Bottleneck Problem – Intuition

- Too much information compressed into one vector
- Hard for decoder to recover full meaning
- Limits translation quality
- Major drawback of vanilla Seq2Seq

Need for Attention Mechanism

- Decoder should not rely on single vector
- Important words differ for each output step
- Model should focus dynamically
- Leads to attention mechanism
- **A new context vector is built at every decoding step**

Attention Mechanism – Definition

- Attention allows model to focus on relevant input parts
- Context vector becomes dynamic
- Different attention weights at each time step
- Solves information bottleneck

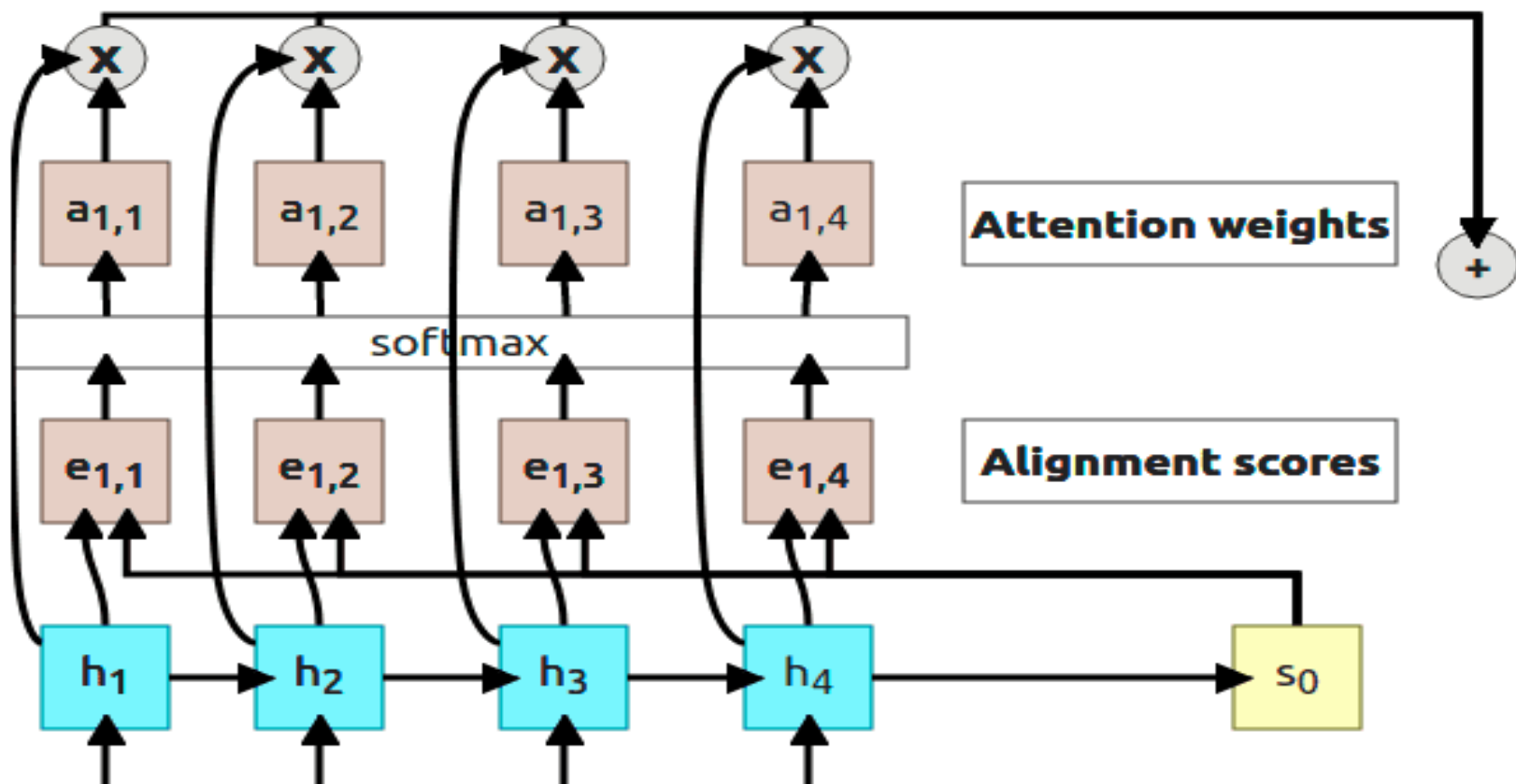
Input Sentence:

I $\rightarrow$ h_1 am $\rightarrow$ h_2 eating $\rightarrow$ h_3 food $\rightarrow$ h_4

How Attention Works

- Encoder produces hidden states for each input word
- Decoder computes relevance scores
- Softmax converts scores to attention weights
- Weighted sum produces context vector

Encoder Hidden States (h_1, h_2, h_3, h_4)



Compute **context vector**

$$c_t = \sum_i a_{t,i} h_i$$

Dynamic context vector

- The encoder generates a hidden state for each input word.
- At every decoding step, the decoder computes relevance scores between its current state and all encoder states.
- These scores are normalized using softmax to obtain attention weights.
- A weighted sum of encoder states then produces a dynamic context vector.

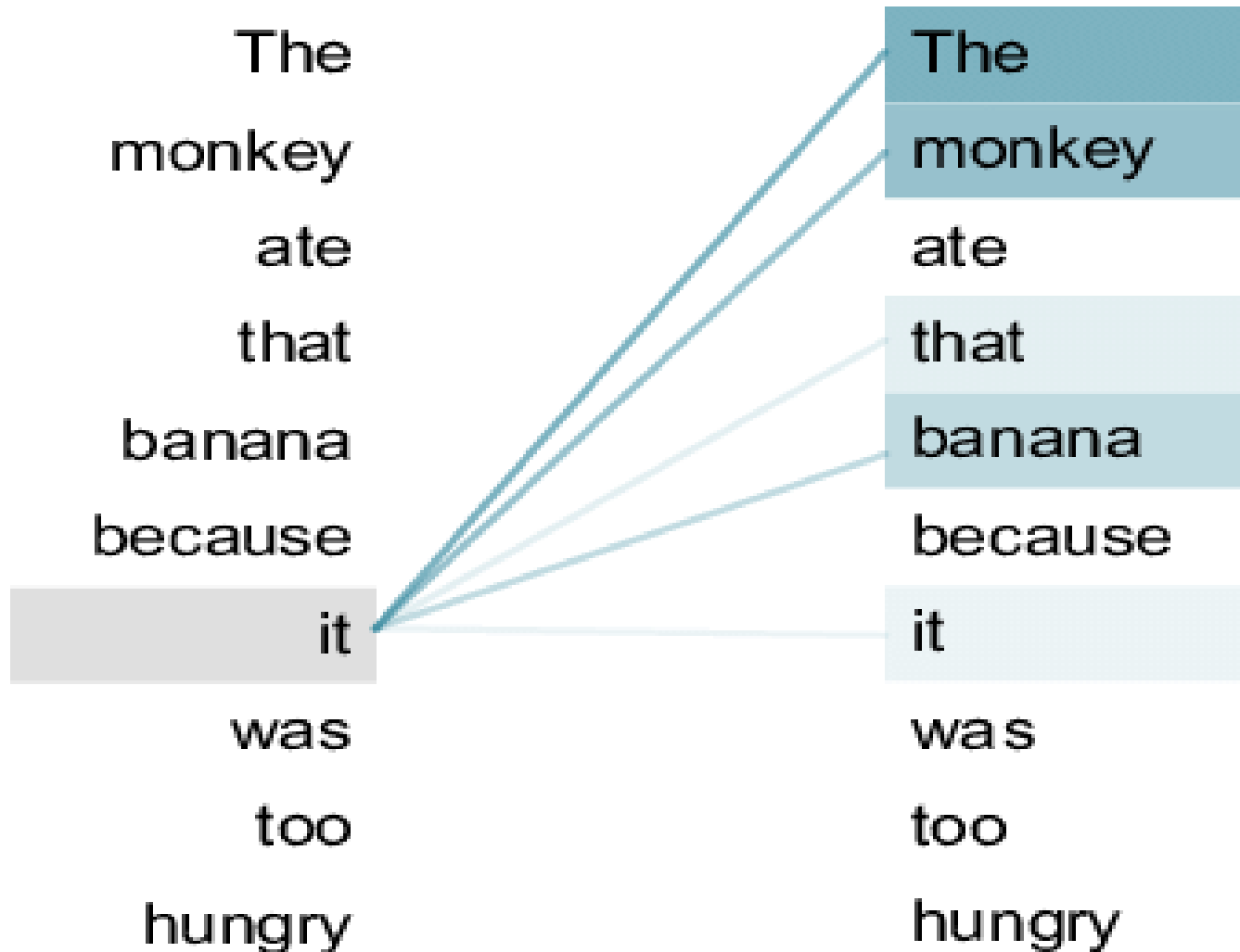
Types of Attention – Overview

- Global Attention: attends to all input tokens
- Local Attention: attends to nearby tokens
- Self-Attention: tokens attend to each other
- Cross Attention: decoder attends to encoder outputs

Self-Attention – Explanation

- Each word attends to all other words
- Captures long-range dependencies
- Does not require sequential processing
- Core idea behind Transformers

Self-Attention: How Tokens Understand Each Other



Self-Attention Example

- Sentence: The animal did not cross the road because it was tired
- 'it' correctly refers to 'animal'
- Learns context automatically
- No handcrafted rules required

Limitations of RNN-Based Models

- Sequential computation is slow
- Difficult to parallelize
- Struggles with very long dependencies
- Motivation for Transformers

Transformer – Definition

- Transformer is an attention-based architecture
- Does not use RNN or LSTM
- Processes entire sequence in parallel
- Introduced in 2017

Transformer Architecture

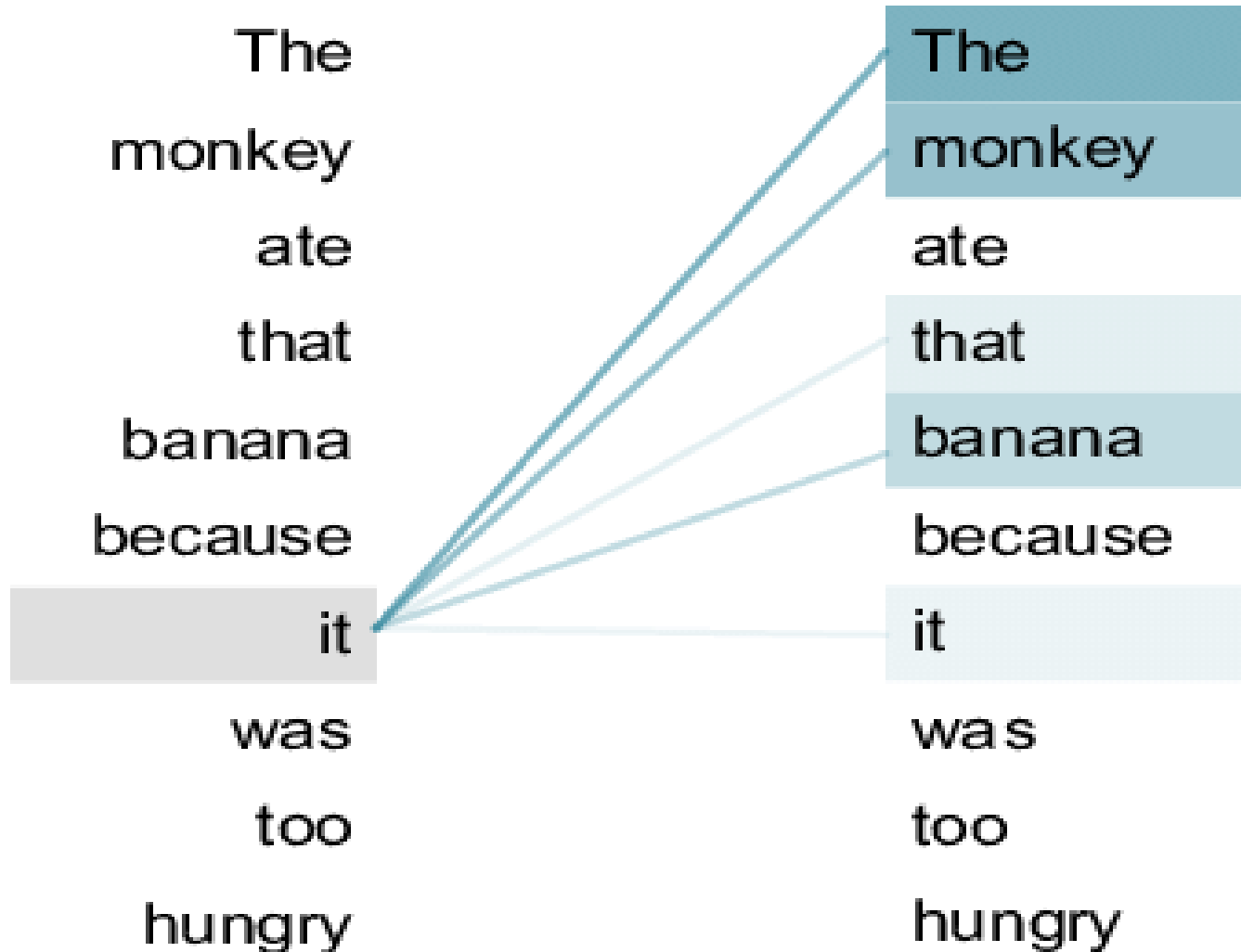
- Consists of encoder and decoder stacks
- Encoder handles input understanding
- Decoder handles output generation
- Built using attention and feed-forward layers

**“Transformer is not one encoder or decoder
— it is a stack of layers.”**

Transformer Encoder – Details

- Multi-head self-attention layer
- Feed-forward neural network
- Residual connections and layer normalization
- Stacked multiple times

Self-Attention: How Tokens Understand Each Other



Multi-head self-attention

- Allows the Transformer to attend to information from different representation subspaces simultaneously.
- Each attention head computes self-attention independently using separate projection matrices, and their outputs are concatenated and linearly transformed to produce the final representation.

Transformer Decoder – Details

- Masked self-attention to prevent future access
- Encoder–decoder attention
- Feed-forward network
- Autoregressive generation

Positional Encoding

- Transformers lack sequence order information
- Positional encodings add order awareness
- Added to input embeddings
- Can be sinusoidal or learned

Why Transformers Are Powerful

- Parallel processing improves speed
- Captures long-range dependencies
- Scales to large datasets
- Foundation of modern NLP models

Transformer Model Categories

- Encoder-only models
- Decoder-only models
- Encoder–decoder models
- Chosen based on task requirements

BERT – Model Overview

- Encoder-only Transformer
- Bidirectional self-attention
- Designed for language understanding
- Pretrained using masked language modeling

BERT Applications

- Sentiment analysis
- Named Entity Recognition
- POS tagging
- Extractive question answering

GPT – Model Overview

- Decoder-only Transformer
- Autoregressive text generation
- Uses masked self-attention
- Predicts next token

GPT Applications

- Chatbots and dialogue systems
- Story and content generation
- Code generation
- Text completion

T5 – Model Overview

- Encoder–decoder Transformer
- All tasks framed as text-to-text
- Unified task formulation
- Highly flexible architecture

T5 Example

- Input: summarize: Transformers are powerful models
- Output: Transformers are powerful
- Same model handles many tasks
- Task specified using prefix

BART – Model Overview

- Encoder–decoder Transformer
- Trained as denoising autoencoder
- Corrupt input and reconstruct original
- Combines BERT and GPT ideas

BART Applications

- Abstractive summarization
- Paraphrasing
- Text correction
- Question generation

Large Language Models (LLMs)

- Very large Transformer-based models
- Trained on massive text corpora
- Exhibit emergent abilities
- Foundation of modern AI assistants

LLM Applications

- Conversational AI
- Research assistance
- Programming support
- Multimodal reasoning

Evolution Timeline

- 2014: Seq2Seq
- 2015: Attention
- 2017: Transformer
- 2018+: BERT, GPT, T5, BART, LLMs

Key Takeaways

- Seq2Seq introduced encoder–decoder
- Attention solved bottleneck
- Transformers removed recurrence
- LLMs scale everything

Discussion

- Why attention is important?
- Why Transformers outperform RNNs?
- Which model for which task?
- Questions?

Building Models & Case Studies

- Question Answering and Chatbots
- Using Transformer Models

Question Answering (QA)

- QA systems answer natural language questions automatically.
- Types: Extractive, Abstractive, Open-domain, Closed-domain

Types of QA

- Extractive QA
- Abstractive QA
- Open-domain QA
- Closed-domain QA

Transfer Learning

- Reuse pre-trained language models
- Fine-tune for QA tasks
- Less data, better performance

Transfer Learning Workflow

- Pre-training
- Fine-tuning
- Inference

BERT for Question Answering

- Primarily extractive QA
- Predicts start and end positions

BERT Input Format

- [CLS] Question [SEP] Context [SEP]
- Bidirectional attention

Advantages of BERT

- Strong contextual understanding
- High accuracy on reading comprehension

Limitations of BERT

- Cannot generate new text
- Answer must exist in passage

T5 Model Overview

- Text-to-Text Transfer Transformer
- All tasks as text generation

T5 for QA

- Input: question + context
- Output: generated answer

Advantages of T5

- Supports abstractive QA
- Unified framework

QA Model Architecture

- Input Processing
- Transformer Encoder
- Task-Specific Head

Chatbots Overview

- Conversational agents
- Rule-based, Retrieval-based, Generative

Transformer Chatbots

- Use self-attention
- Generate human-like responses

Challenges in Transformer Chatbots

- Long context
- High memory usage
- Latency
- Hallucination

Solutions to Challenges

- Efficient attention
- Memory augmentation
- Retrieval-augmented generation

Reformer Model

- Optimized Transformer
- Designed for long sequences

Reformer Key Features

- LSH Attention
- Reversible layers
- Reduced memory usage

Reformer-based Chatbot

- Handles long conversations efficiently
- Faster and scalable

Model Comparison

- BERT: Extractive QA
- T5: Abstractive QA
- Reformer: Efficient Chatbots

Conclusion

- BERT for reading comprehension
- T5 for flexible QA
- Reformer for long-context chatbots